“시간이 아깝지 않은”

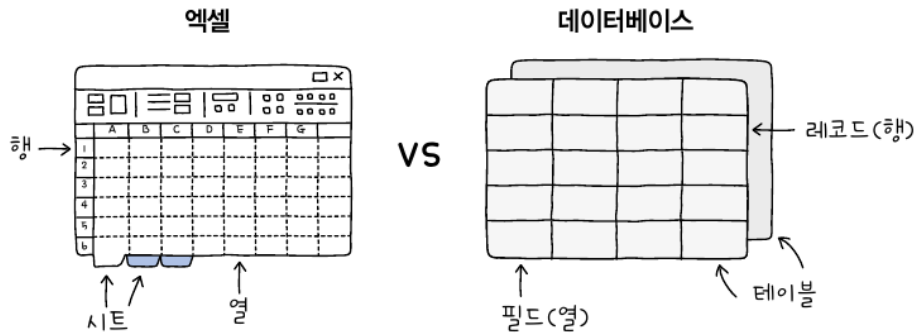
# SK네트웍스 Family AI캠프 교재2

강사 이영배

# 04. Database

# 데이터베이스(DB)

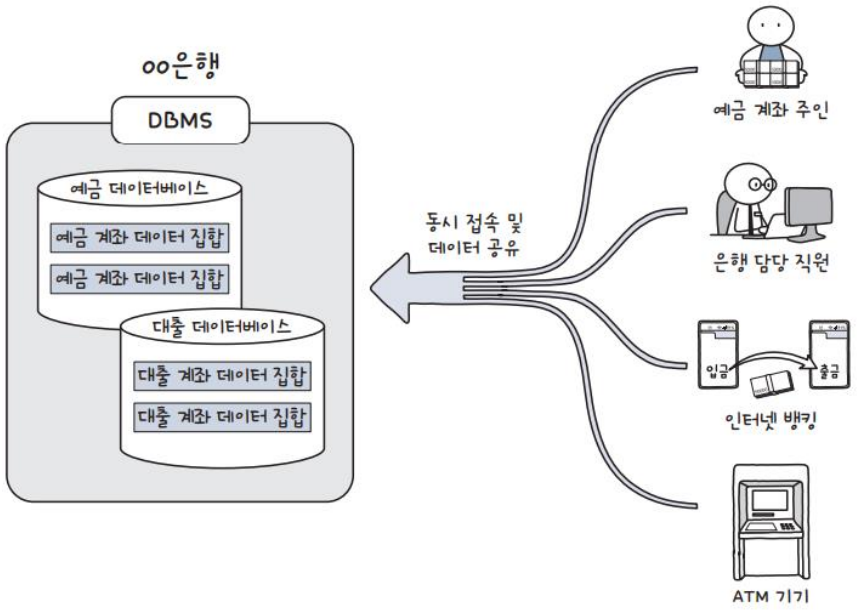
- 데이터베이스(DB) : 컴퓨터를 사용하여 저장하고 관리하는 데이터의 집합
- DBMS(Database Management System, 데이터베이스 관리 시스템)
  1) 데이터베이스를 운영하고 관리하는 소프트웨어,  예) MySQL, MariaDB, PostgreSQL, Oracle, SQLite
  2) 관계형 DBMS : 정보를 표 형식(table)으로 저장, 테이블은 레코드(row)와 필드(column)로 구성, 행과 열은 데이터 간의 다양한 관계를 나타냄




| 내용 | 엑셀 | 데이터베이스 |
|---|---|---|
| 데이터 구조 | • 행과 열로 구성<br>• 시트 | • 레코드(행)과 필드(열)로 구성<br>• 테이블 |
| 관리 프로그램 | • 스프레드시트 프로그램 예) 엑셀 | • DBMS |

# 엑셀과 데이터베이스 비교

- 데이터베이스(DB)
  1) 장점 : 여러 명의 사용자나 응용 프로그램과 공유하고 동시에 접근이 가능하며, 사용 중에 데이터가 안전하게 저장되고 보호됨
  2) 단점 : 전문가(관리자) 필요



# MySQL 개요

- MySQL 특징
  1) 오픈소스 기반의 세계에서 가장 인기 있는 관계형 데이터베이스 관리 시스템(RDBMS)
  2) 뛰어난 성능, 높은 신뢰성, 쉬운 사용성
- MySQL 에디션 비교
  1) Community Edition (오픈소스 / 무료)
     : 개발자, 교육기관, 중소 규모 서비스에서 사용하는 무료 에디션 (본 과정 실습 버전)
  2) Enterprise Edition (상용 / 유료)
     : 대기업용 고급 보안, 모니터링, 기술 지원이 포함된 에디션

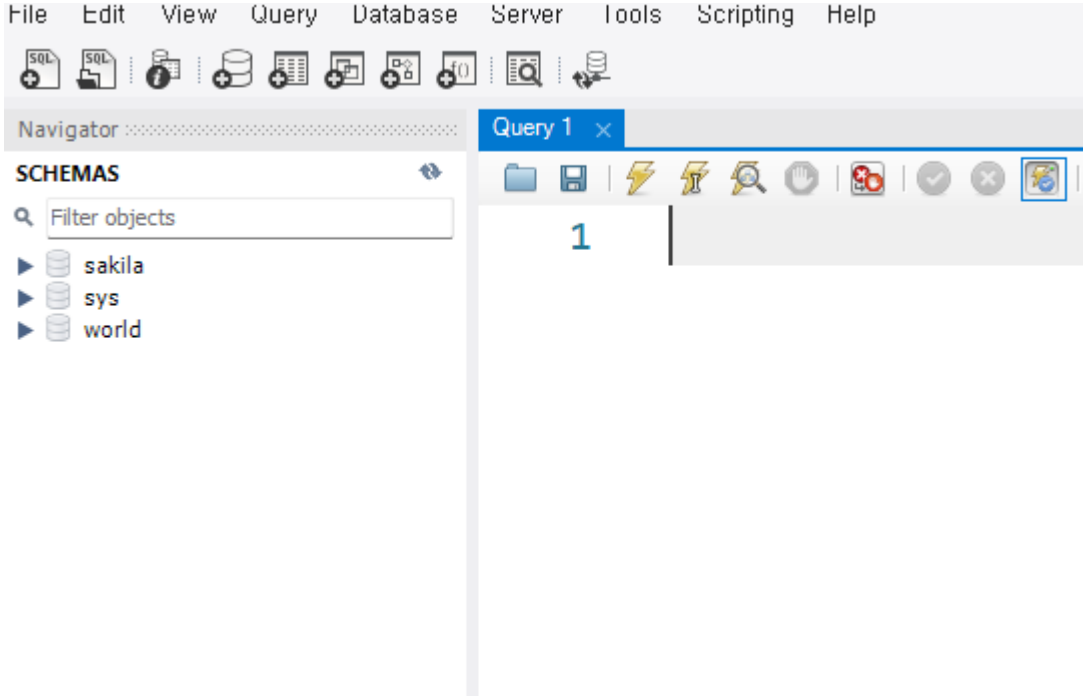
# MySQL Workbench 접속 확인

- Workbench 접속 및 샘플 Database 확인

  1) Workbench 실행 후 `Local instance MySQL` 클릭 (root 계정 로그인)

  2) 화면 왼쪽 상단 Navigator의 `SCHEMAS` 탭 확인

  3) 사전 설치된 `sakila`, `world`, `sys` 데이터베이스 확인



# 정보시스템 구축 5단계 (SDLC)

- 소프트웨어 생명주기 (Software Development Life Cycle)

1) 요구 사항 분석 (Requirements Analysis)

: 고객 및 업무 담당자의 요구 사항을 수집하고 정리하는 단계 (무엇을 만들 것인가?)

2) 시스템 설계 (System Design) [핵심]

: 요구 사항을 바탕으로 화면 설계 및 데이터베이스 설계도(ERD)를 작성하는 단계

3) 프로그램 구현 (Implementation)

: 설계도를 바탕으로 SQL 테이블을 생성하고, 실제 코딩하는 단계

4) 테스트 (Testing)

: 만들어진 시스템의 버그 및 요구사항 충족 여부를 검증하는 단계

5) 유지보수 (Maintenance)

: 시스템 오픈 후 발생하는 오류 수정 및 기능 제어

# 모델, 모델링, ERD 개념 정리

- 모델, 모델링, ERD라는 복잡한 개념이 필요한 이유

  1) 관계형 데이터베이스(RDB)가 데이터를 엄격하게 세분화(정규화)하여 찢어 놓기 때문이다.

  2) 예시: 쇼핑몰에서 '주문 테이블'에 회원 이름, 주소, 상품명, 가격을 다 넣으면 데이터 중복이 심해지므로, 이를 회원, 상품, 주문 테이블로 잘게 쪼갠다.

  3) 데이터를 잘게 쪼개는 이유: 홍길동 회원이 이사를 가서 주소를 변경했는데, 만일 데이터 관리자가 깜빡하고 1번 행만 수정하고 2, 3번 행을 수정하지 않으면, 동일한 회원의 주소가 행마다 달라지는 데이터 불일치가 발생한다.

  4) 해결 방안: 데이터 세분화 → 회원 테이블, 상품 테이블, 주문 테이블

| 주문번호 | 회원ID | 회원이름 | 회원주소 | 상품명 | 가격 |
|---|---|---|---|---|---|
| 1 | user01 | **홍길동** | **서울시 서초구...** | 맥북 | 2,000,000 |
| 2 | user01 | **홍길동** | **서울시 서초구...** | 마우스 | 50,000 |
| 3 | user01 | **홍길동** | **서울시 서초구...** | 키보드 | 150,000 |

# 모델, 모델링, ERD 개념 정리

- 모델, 모델링, ERD라는 복잡한 개념이 필요한 이유

| 주문번호 | 회원ID | 회원이름 | 회원주소 | 상품명 | 가격 |
|---|---|---|---|---|---|
| 1 | user01 | **홍길동** | **서울시 서초구...** | 맥북 | 2,000,000 |
| 2 | user01 | **홍길동** | **서울시 서초구...** | 마우스 | 50,000 |
| 3 | user01 | **홍길동** | **서울시 서초구...** | 키보드 | 150,000 |

**1. 회원 테이블 (Member Table)**

회원의 고유 정보만 보관합니다. 홍길동 회원의 이름과 주소는 전체 시스템에서 **딱 한 줄(1개 행)**만 저장되어 중복이 완전히 제거됩니다.

| 회원ID (PK) | 회원이름 | 회원주소 |
|---|---|---|
| user01 | 홍길동 | 서울시 서초구... |

**2. 상품 테이블 (Product Table)**

판매하는 상품의 고유 정보만 보관합니다. 각 상품의 이름과 가격이 안전하게 분리됩니다.

| 상품ID (PK) | 상품명 | 가격 |
|---|---|---|
| prod01 | 맥북 | 2,000,000 |
| prod02 | 마우스 | 50,000 |
| prod03 | 키보드 | 150,000 |

**3. 주문 테이블 (Order Table)**

"누가(회원ID), 무엇을(상품ID), 어떤 주문번호로" 샀는지에 대한 **연결 관계와 행위**만 기록합니다.

| 주문번호 (PK) | 회원ID (FK) | 상품ID |
|---|---|---|
| 1 | user01 | prod01 |
| 2 | user01 | prod02 |
| 3 | user01 | prod03 |

# 모델, 모델링, ERD 개념 정리

- 데이터베이스 구축은 현실의 복잡한 비즈니스를 컴퓨터 속 데이터 공간으로 옮기는 작업이고, 세 개념은 각각 [결과물], [과정], [시각화 도면] 관계를 맺고 있다.

1) 모델 (Model) → [결과물]

① 개념: 현실 세계의 복잡한 데이터와 비즈니스 규칙을 단순화하여 정제한 '데이터 구조의 틀'이다.

② 실체: 관계형 데이터베이스에서 구체화된 모델의 실체는 바로 '테이블(Table)'과 테이블 정의서이다.

2) 모델링 (Modeling) → [과정 / 행위]

① 개념: 사용자의 요구사항을 분석하여 데이터베이스에 저장할 수 있는 '모델'로 변환해 나가는 전체적인 프로세스이다.

② 행위: 어떤 데이터를 테이블로 쪼갤지, 핵심 식별자(PK)는 무엇으로 할지, 테이블 간의 연관 관계를 어떻게 맺을지 고민하고 정규화하는 작업 일체를 의미한다.

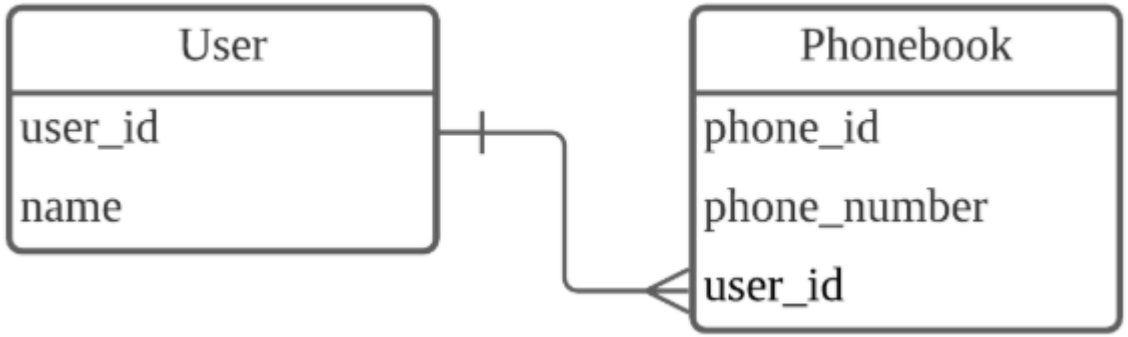
# 모델, 모델링, ERD 개념 정리

- 데이터베이스 구축은 현실의 복잡한 비즈니스를 컴퓨터 속 데이터 공간으로 옮기는 작업이고, 세 개념은 각각 [결과물], [과정], [시각화 도면] 관계를 맺고 있다.

3) ERD (Entity-Relationship Diagram) → [시각화 설계도]

① 개념: 모델링 과정에서 도출된 테이블들의 내부 구조와 그들 간의 관계를 한눈에 볼 수 있도록 시각화한 설계도이다.

② 포함 요소: 테이블 내부의 컬럼명, 자료형 등이 사각형 안에 명시되며, 이 사각형들이 서로 1:1, 1:N, N:M 중 어떤 관계로 얽혀 있는지 선과 기호로 표현된다.



# DB 모델링 필수 용어

- 엔티티 (Entity / 개체)
  1) 데이터로 관리하고자 하는 현실 세계의 대상 (명사 형태: 회원, 상품, 주문 등)
  2) DB 적용: 테이블(Table)이 됨
- 속성 (Attribute)
  1) 엔티티가 가질 수 있는 세부적인 특징이나 항목 (회원 이름, 아이디, 이메일 등)
  2) DB 적용: 테이블의 컬럼(Column / 필드)이 됨

# DB 모델링 필수 용어

- 관계 (Relationship)
  1) 엔티티와 엔티티 간의 업무적 규칙 및 연관성을 의미하며, 주로 '주문한다', '소속된다'와 같은 동사 형태로 표현된다.
  2) 1:1, 1:N, N:M 세 가지기 관계가 존재한다.
  3) 1:1 관계
     ① 개념: A 테이블의 1개 행이 B 테이블의 1개 행과만 연결되는 관계이다.
     ② 예시: [국민] 1명은 1개의 [주민등록증]만 보유한다.
  4) 1:N 관계
     ① 개념: A 테이블의 1개 행이 B 테이블의 N개 행과 연결될 수 있지만, B 테이블의 각 행은 반드시 A 테이블의 1개 행과만 연결되는 관계이다.
     ② 예시: [부서] 1개에는 여러 명의 [사원]이 소속지만, 사원은 한 부서에만 소속된다.

# DB 모델링 필수 용어

- 관계 (Relationship)

  5) N:M 관계

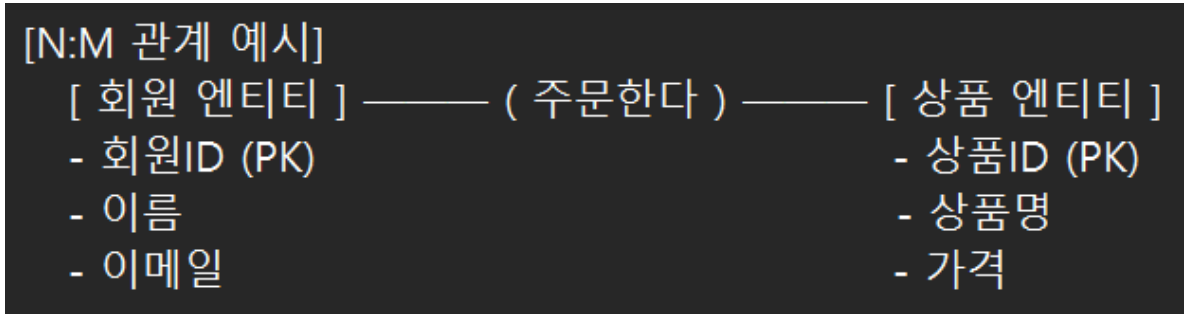
  ① 개념: A 테이블의 행들과 B 테이블의 행들이 서로 다수 대 다수로 연결되는 관계이다.

  ② 예시: 한 명의 회원은 여러 개(N개)의 상품을 주문할 수 있으며, 하나의 상품은 여러 명(M명)의 회원에게 판매(주문)될 수 있다.

- 주식별자 (PK: Primary Key)

  1) 테이블(엔티티) 내 수많은 데이터(행) 중 단 하나를 유일하게 구별할 수 있는 대표 속성이다.

  2) 예시: 학번, 회원 ID, 상품 코드, 사원 번호



# 데이터베이스 모델링의 3단계 진행 과정

- 데이터베이스 모델링
  1) 개념:  현실 세계의 업무 프로세스와 데이터를 RDBMS의 테이블 구조로 변환하는 과정
  2) 모델링 3단계 진행 과정
     ① 개념적 모델링: 업무 분석 후 핵심 주체(Entity)와 관계(Relationship)를 추출하여 ERD(개념도)라는 그림으로 표현하는 단계
     ② 논리적 모델링: 그림으로 그린 ERD를 테이블 형태로 변환하고, 속성과 기본키(PK) 등을 명확히 정의하는 단계
     ③ 물리적 모델링: 사용할 DBMS(MySQL)의 특성에 맞춰 INT와 같은 구체적인 데이터 형식을 지정하는 단계

# SQL 개요 및 명령어 종류

- SQL (Structured Query Language)

  : 관계형 데이터베이스에 정보를 저장, 수정, 삭제, 조회하기 위한 표준 프로그래밍 언어

- 주요 SQL 명령어 분류

1) DDL (Data Definition Language / 데이터 정의어)

  : 데이터베이스, 테이블 등 데이터베이스 스키마(구조)를 생성, 변경, 삭제하는 명령어

| 명령어 | 대상 | 설명 |
|---|---|---|
| SHOW | 데이터베이스 | MySQL 서버에 존재하는 모든 데이터베이스 목록을 보여줍니다. |
| | 테이블 | 특정 데이터베이스 또는 현재 선택된 데이터베이스의 모든 테이블 목록을 보여줍니다. |
| CREATE | 데이터베이스 | 새로운 데이터베이스를 생성합니다. |
| | 테이블 | 새로운 테이블을 생성합니다. |
| ALTER | 데이터베이스 | 기존 데이터베이스의 속성을 수정합니다. |
| | 컬럼 | 테이블의 기존 컬럼의 데이터 타입이나 제약 조건을 수정합니다. |
| DROP | 데이터베이스 | 기존 데이터베이스를 삭제합니다. |
| | 테이블 | 기존 테이블을 삭제합니다. |

# SQL 개요 및 명령어 종류

- 주요 SQL 명령어 분류

2) DML (Data Manipulation Language / 데이터 조작어)

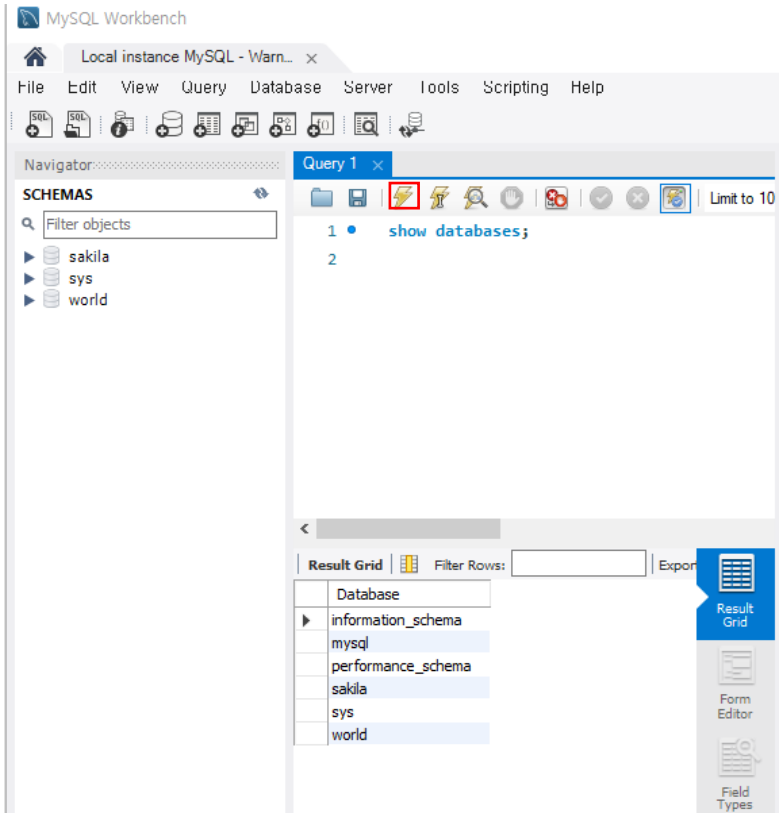
: 테이블 내의 실제 데이터를 조회, 입력, 수정, 삭제하는 명령어

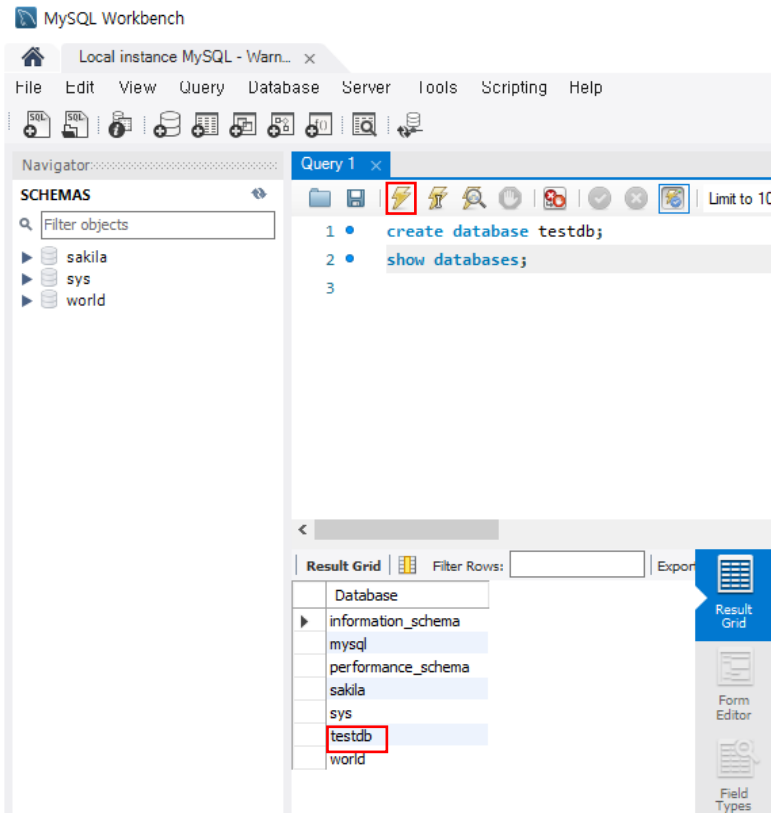
| 명령어 | 설명 |
|---|---|
| select | 데이터 테이블에서 데이터를 조회합니다. |
| insert | 데이터 테이블에 데이터를 삽입합니다. |
| update | 데이터 테이블의 데이터를 변경합니다. |
| delete | 데이터 테이블의 데이터를 삭제합니다. |

# 데이터베이스 생성

- CREATE DATABASE [데이터베이스 이름];
- 실습 코드
  1) 기존 DB 목록 확인: SHOW DATABASES;
  2) 실습용 testdb 생성 및 확인
     ① CREATE DATABASE testdb;
     ② SHOW DATABASES;

# 데이터베이스 생성 실습


MySQL Workbench
Local instance MySQL - Warn...
File Edit View Query Database Server Tools Scripting Help
Navigator
SCHEMAS
Filter objects
sakila
sys
world
Query 1
Limit to 10
1 show databases;
2
Result Grid
Filter Rows:
Export
Database
information_schema
mysql
performance_schema
sakila
sys
world
Result Grid
Form Editor
Field Types


MySQL Workbench
Local instance MySQL - Warn...
File Edit View Query Database Server Tools Scripting Help
Navigator
SCHEMAS
Filter objects
sakila
sys
world
Query 1
Limit to 10
1 create database testdb;
2 show databases;
3
Result Grid
Filter Rows:
Export
Database
information_schema
mysql
performance_schema
sakila
sys
testdb
world
Result Grid
Form Editor
Field Types

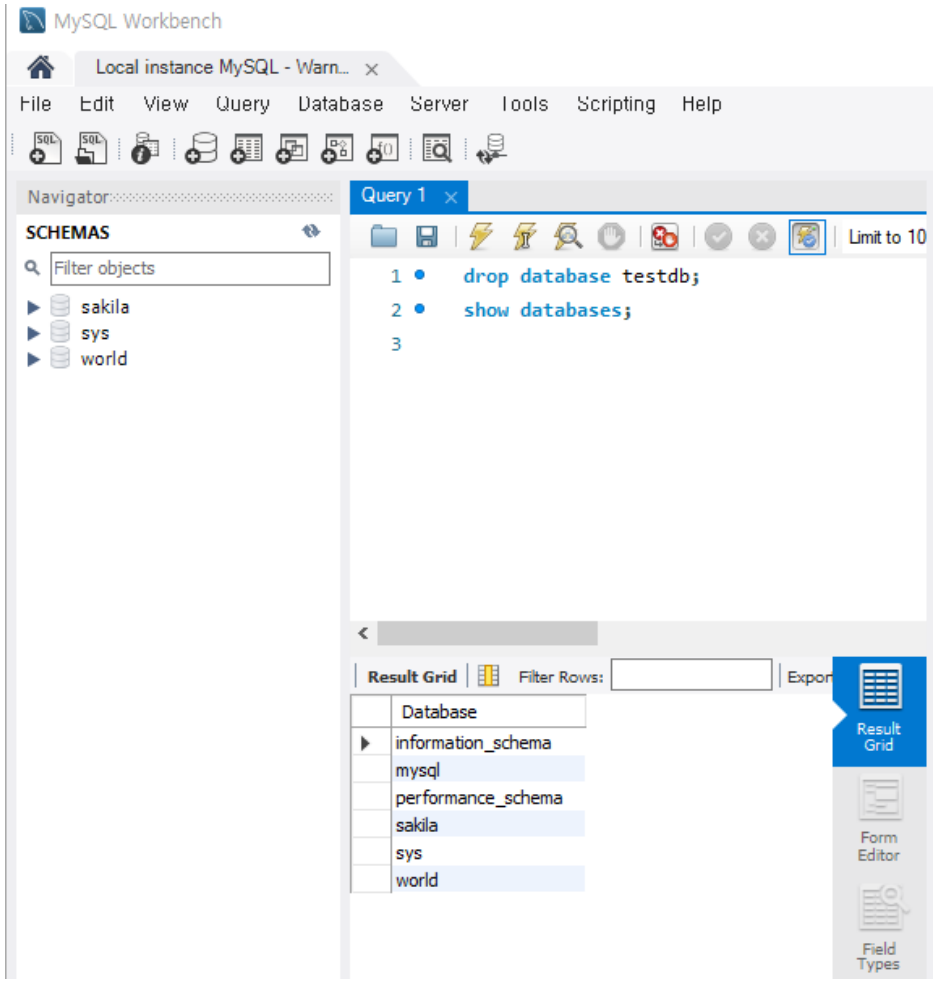
# 데이터베이스 삭제

- DROP DATABASE [데이터베이스 이름];
- 실습 코드

1) 실습용 testdb 삭제: DROP DATABASE testdb;

2) 확인: SHOW DATABASES;

# 데이터베이스 삭제 실습


MySQL Workbench
Local instance MySQL - Warn...
File Edit View Query Database Server Tools Scripting Help
Navigator:
SCHEMAS
Filter objects
sakila
sys
world
Query 1
Limit to 10
1 drop database testdb;
2 show databases;
3
Result Grid
Filter Rows:
Export
Database
information_schema
mysql
performance_schema
sakila
sys
world
Result Grid
Form Editor
Field Types

# 테이블 생성을 위한 주요 데이터 유형

| 데이터 유형 | 목적 | 특징 | 일반적인 사용 사례 |
|---|---|---|---|
| INT (정수형) | 부호 있는 또는 부호 없는 정수 저장 | -2,147,483,648 ~ 2,147,483,647 정수를 저장 | 고유 ID, 수량, 횟수 등 |
| DECIMAL (정밀 십진수형) | 정확한 십진수 값 저장 | DECIMAL(전체 자리수, 소수점 이하 자리수) 형식으로 정확한 십진수 값을 저장 | 가격, 금액, 세율 등 정확한 측정값 |
| VARCHAR (가변 길이 문자열형) | 가변 길이 문자열 저장 | VARCHAR저장 가능 최대 문자수) 형식으로 최대 길이 지정, 실제 데이터 길이에 맞춰 저장 공간 사용 | 이름, 제목, 주소, 설명 등 |
| DATETIME (날짜 및 시간형) | 날짜 및 시간 정보 저장 | YYYY-MM-DD HH:MM:SS 형식으로 날짜와 시간 정보를 저장 | 생성 시간, 수정 시간, 이벤트 발생 시간 등 |

# 테이블 생성

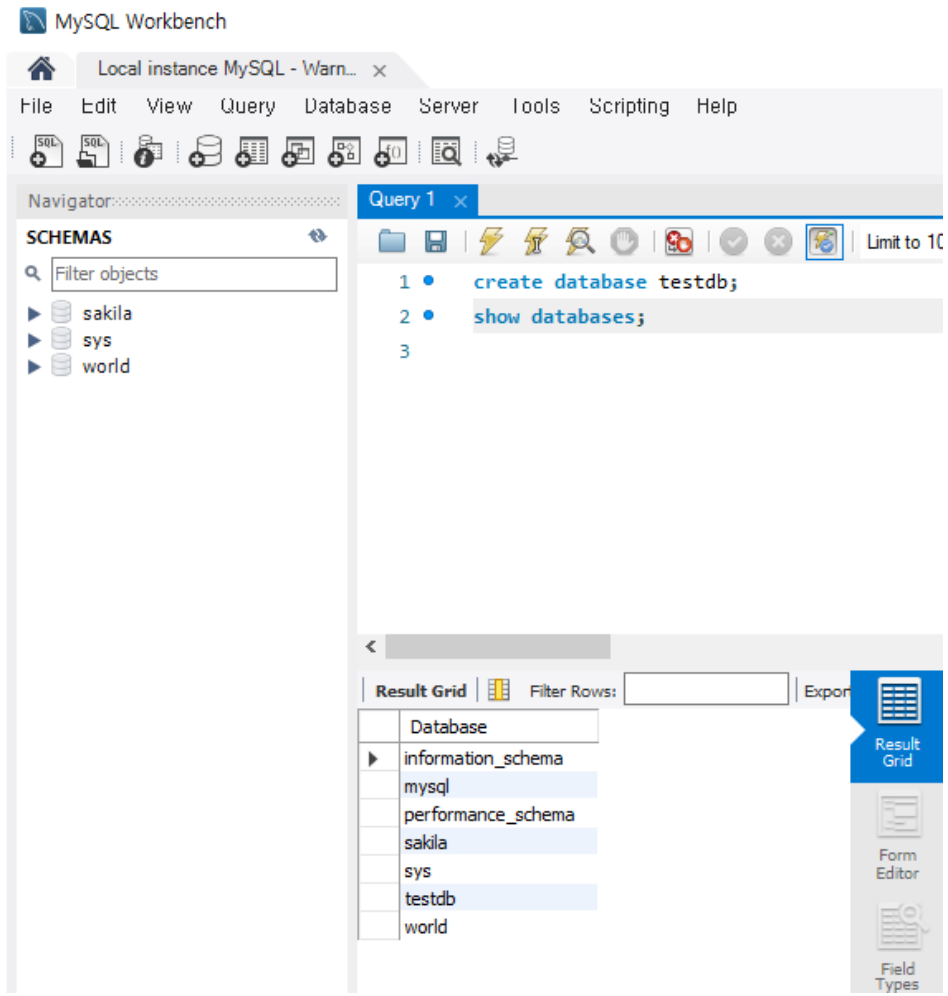
- 테이블 생성 기본 문법

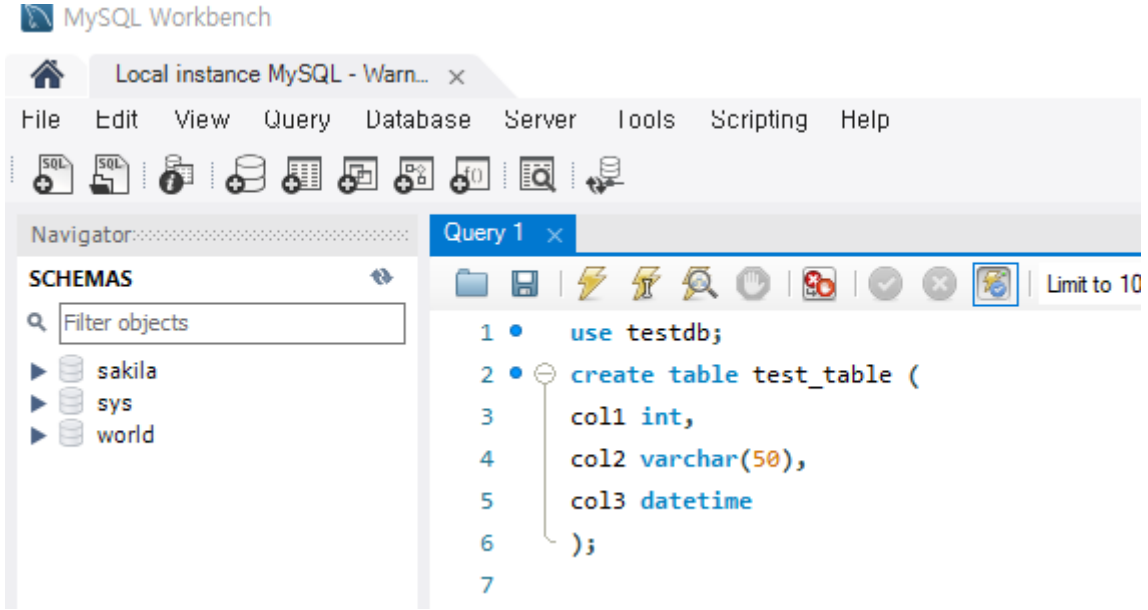
```
USE [데이터베이스 이름];
CREATE TABLE [테이블 이름] (
[열 이름1 데이터 유형 ],
[열 이름2 데이터 유형 ],
        (... 생략 ...)
);
```

- 실습 코드

```
CREATE DATABASE testdb;
USE testdb;
# test_table 생성
CREATE TABLE test_table (
col1 INT,
col2 VARCHAR(50),
col3 DATETIME
);
# 생성된 테이블 목록 확인
SHOW TABLES;
```

# 테이블 생성 실습

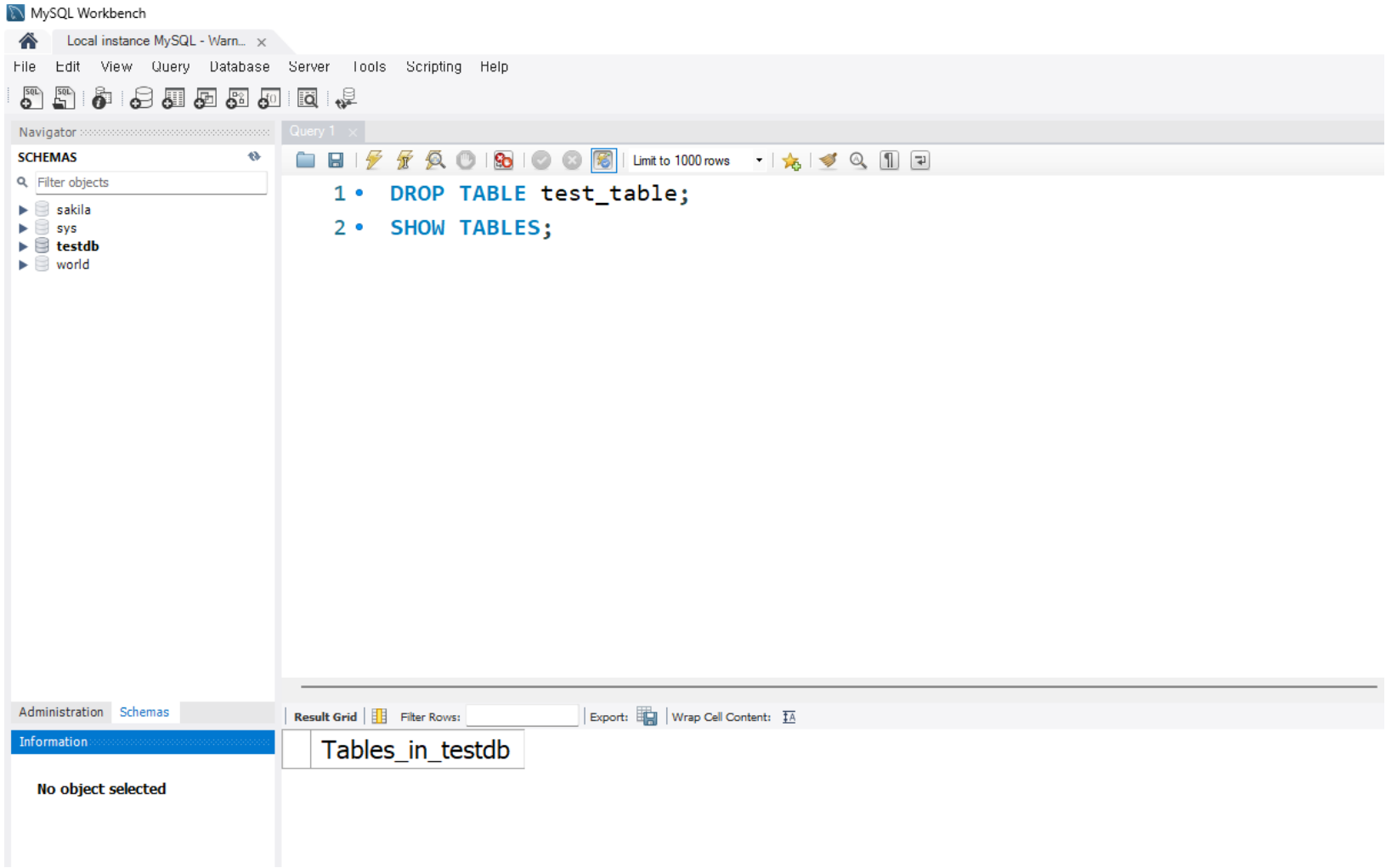

MySQL Workbench
Local instance MySQL - Warn...
File Edit View Query Database Server Tools Scripting Help
Navigator:
SCHEMAS
Filter objects
sakila
sys
world
Query 1
Limit to 10
1 create database testdb;
2 show databases;
3
Result Grid
Filter Rows:
Export:
Database
information_schema
mysql
performance_schema
sakila
sys
testdb
world
Result Grid
Form Editor
Field Types


MySQL Workbench
Local instance MySQL - Warn...
File Edit View Query Database Server Tools Scripting Help
Navigator:
SCHEMAS
Filter objects
sakila
sys
world
Query 1
Limit to 10
1 use testdb;
2 create table test_table (
3 col1 int,
4 col2 varchar(50),
5 col3 datetime
6 );
7

# 테이블 삭제

- 테이블 삭제 기본 문법: drop table [테이블 이름];
- 실습 코드
  1) test_table 삭제: DROP TABLE test_table;
  2) 확인: SHOW TABLES;

# 테이블 삭제 실습


MySQL Workbench
Local instance MySQL - Warn...
File Edit View Query Database Server Tools Scripting Help
Navigator
SCHEMAS
Filter objects
sakila
sys
testdb
world
Query 1
Limit to 1000 rows
1 • DROP TABLE test_table;
2 • SHOW TABLES;
Administration Schemas
Information
No object selected
Result Grid
Filter Rows:
Export:
Wrap Cell Content:
Tables_in_testdb

# 주요 속성: PRIMARY KEY & NOT NULL

- PRIMARY KEY (기본 키)
  1) 테이블 내의 각 행(Row)을 유일하게 식별하는 컬럼 (테이블당 단 1개만 존재)
  2) 특징: Unique(중복 불가) + NOT NULL(누락 불가)의 특성을 자동 보유
- NOT NULL
  1) 해당 컬럼에 NULL(누락) 상태가 들어오는 것을 금지하는 제약조건
  2) 데이터 누락을 방지하고 데이터 무결성(Integrity)을 보장함
- [실습 구문]

```
CREATE TABLE users (
    user_id INT PRIMARY KEY,
    username VARCHAR(50) NOT NULL,
    email VARCHAR(100)
);
```

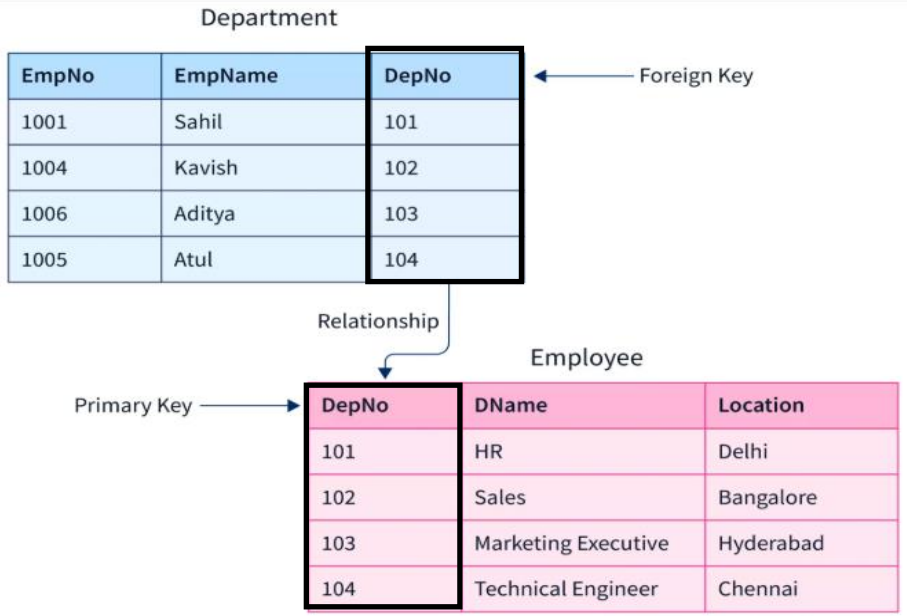
# 주요 속성: AUTO_INCREMENT

- AUTO_INCREMENT (자동 순차 증가)
  1) 주로 `PRIMARY KEY` 정수형 컬럼(고유 ID)에 사용됨
  2) 기능: 새로운 행이 추가될 때마다 고유한 정수 번호가 1씩 자동으로 증가하여 할당됨
  3) 특징
    ① 개발자가 직접 번호를 부여할 필요가 없어 관리가 용이함(고유 ID 자동 생성)
    ② 정수형(`INT`) 컬럼에만 지정 가능하며, 테이블당 단 1개의 컬럼만 설정 가능
- [실습 구문]

```
CREATE TABLE users (
    user_id INT PRIMARY KEY AUTO_INCREMENT,
    username VARCHAR(50) NOT NULL,
    email VARCHAR(100)
);
```

# 주요 속성: FOREIGN KEY (외래 키)

- FOREIGN KEY (외래 키)
  1) 한 테이블의 컬럼이 다른 테이블의 `PRIMARY KEY` 컬럼을 참조하도록 설정하는 키
  2) 테이블 간의 부모-자식 관계를 형성함
- 참조 무결성 (Referential Integrity)
  : 부모 테이블에 존재하지 않는 임의의 값을 자식 테이블에 넣을 수 없도록 엄격히 제한함



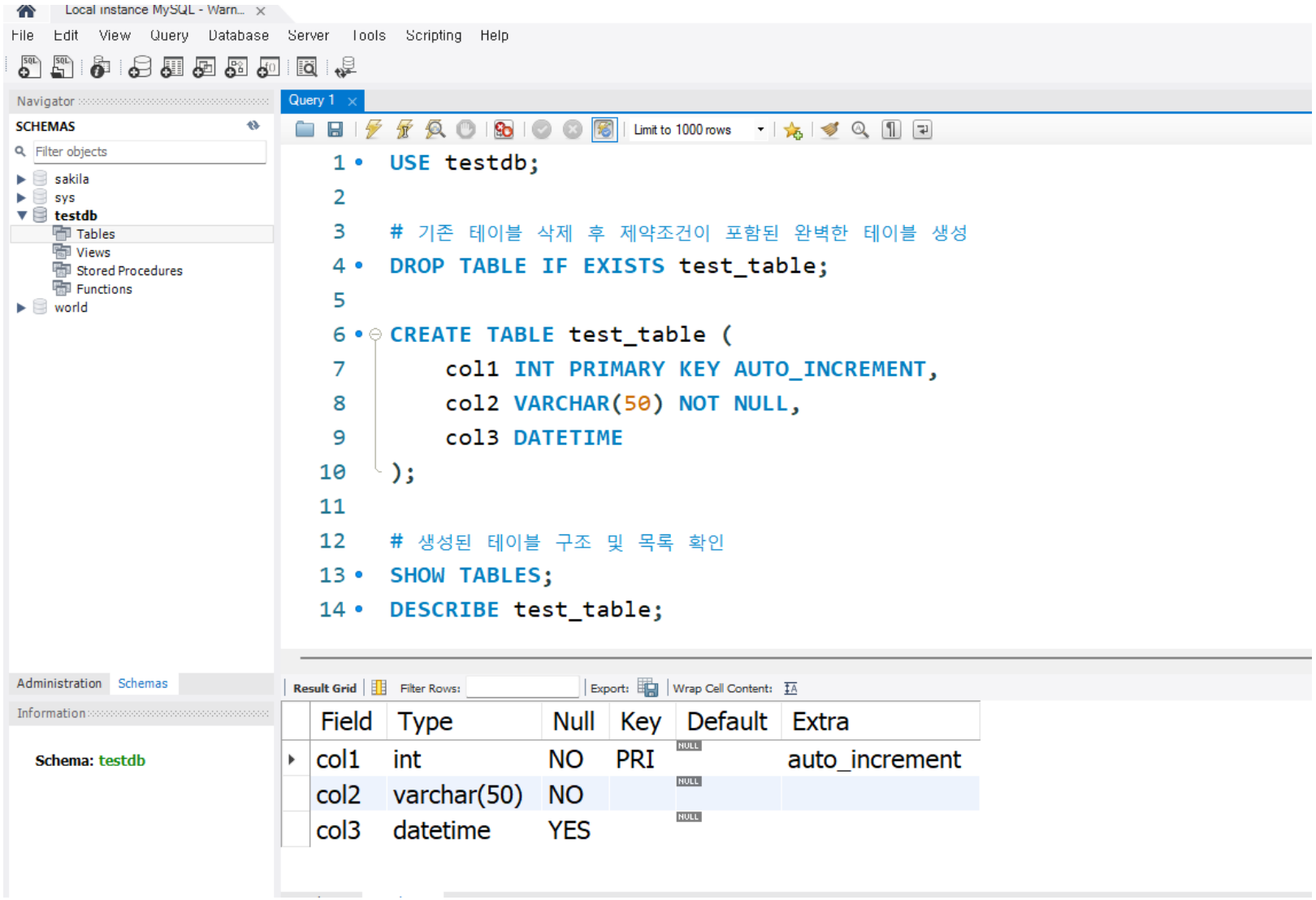

자식 테이블의 컬럼에 들어가는 값은 반드시 부모 테이블의 PK에 실제로 존재하는 값 중에서만 가져오겠다"고 보증하는 행위

# 제약조건 종합 적용 테이블 생성 실습

- 실습 스크립트

```
USE testdb;
# 기존 테이블 삭제 후 제약조건이 포함된 완벽한 테이블 생성
DROP TABLE IF EXISTS test_table;
CREATE TABLE test_table (
    col1 INT PRIMARY KEY AUTO_INCREMENT,
    col2 VARCHAR(50) NOT NULL,
    col3 DATETIME
);
# 생성된 테이블 구조 및 목록 확인
SHOW TABLES;
DESCRIBE test_table;
```

# 제약조건 종합 적용 테이블 생성 실습



# 테이블에 데이터 저장하기 (INSERT)

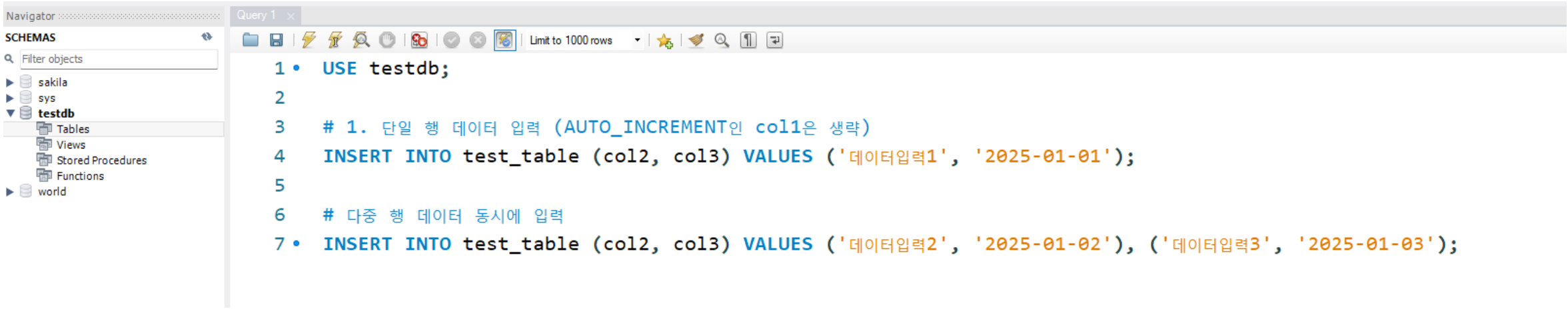
- 데이터 입력 문법

  : INSERT INTO 테이블명 (컬럼1, 컬럼2, ...) VALUES (값1, 값2, ...);

- 실습 스크립트

```
USE testdb;
# 1. 단일 행 데이터 입력 (AUTO_INCREMENT인 col1은 생략)
INSERT INTO test_table (col2, col3) VALUES ('데이터입력1', '2025-01-01');
# 다중 행 데이터 동시에 입력
INSERT INTO test_table (col2, col3) VALUES ('데이터입력2', '2025-01-02'), ('데이터입력3', '2025-01-03');
```

# 테이블에 데이터 저장하기 (INSERT) 실습



# 저장된 데이터 검색 및 확인 (SELECT)

- 데이터 조회 기본 문법

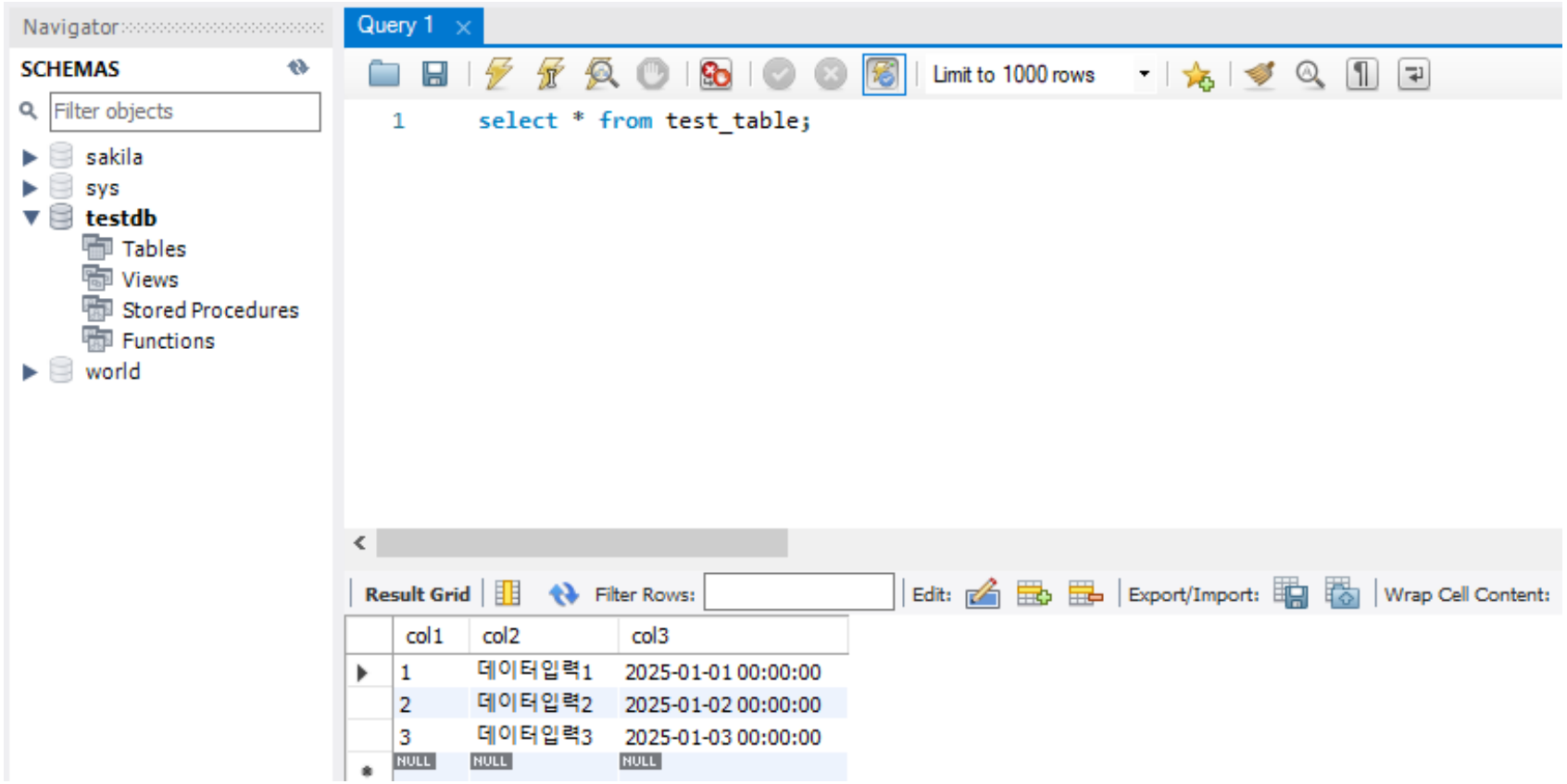
  1) 특정 컬럼 조회:  SELECT 컬럼1, 컬럼2 FROM 테이블명;

  2) 전체 컬럼 조회: SELECT * FROM 테이블명;

- 실습 스크립트 및 결과 확인

```
# test_table의 모든 데이터 조회
SELECT * FROM test_table;
```

# 저장된 데이터 검색 및 확인 (SELECT) 실습



# 데이터 수정 및 Safe Mode 해제

- 데이터 수정 기본 문법

: UPDATE 테이블명 SET 컬럼명 = 변경할값 WHERE 조건절;

- 주의사항

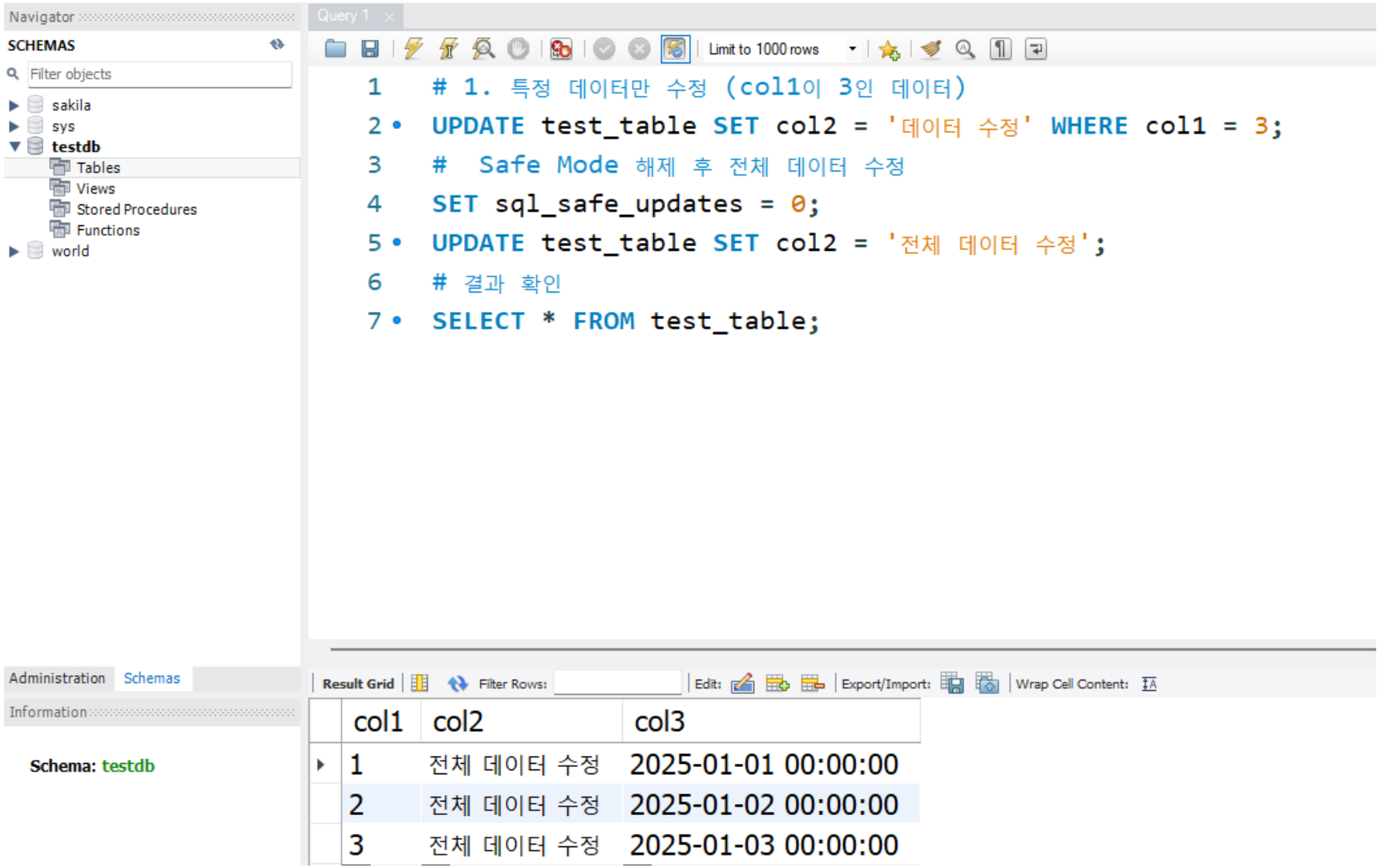
1) WHERE절 없이 UPDATE/DELETE를 실행하거나 PK가 아닌 조건으로 수정 시 에러 발생

2) 해제 명령: SET sql_safe_updates = 0;

- 실습 스크립트

```
# 1. 특정 데이터만 수정 (col1이 3인 데이터)
UPDATE test_table SET col2 = '데이터 수정' WHERE col1 = 3;
#  Safe Mode 해제 후 전체 데이터 수정
SET sql_safe_updates = 0;
UPDATE test_table SET col2 = '전체 데이터 수정';
# 결과 확인
SELECT * FROM test_table;
```

# 데이터 수정 및 Safe Mode 해제 실습



# 데이터 삭제 (DELETE)

- 데이터 삭제 기본 문법

  : DELETE FROM 테이블명 WHERE 조건절;

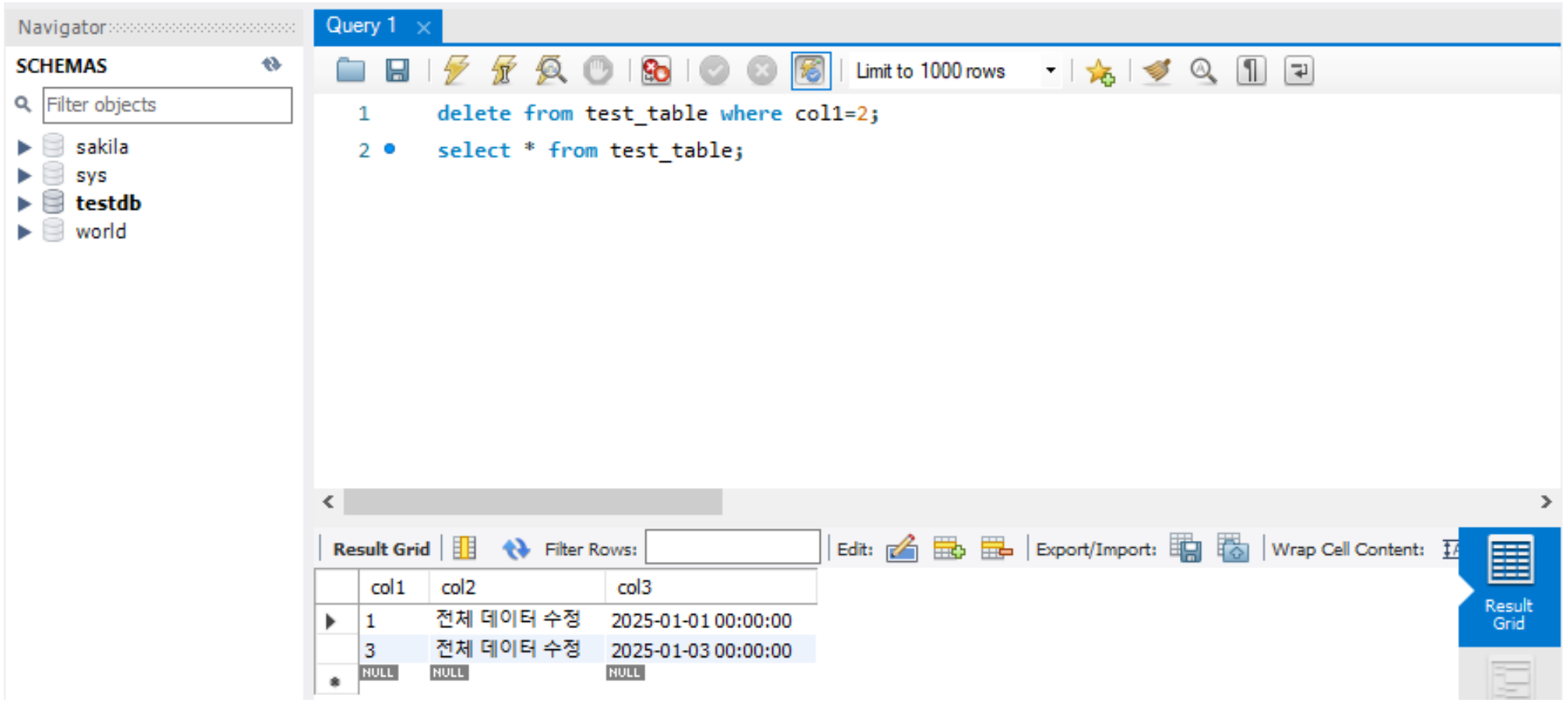
- 실습 스크립트

```
# 조건에 맞는 특정 행 삭제 (col1이 2인 행 삭제)
DELETE FROM test_table WHERE col1 = 2;
SELECT * FROM test_table;

# 테이블 내 전체 데이터 삭제 (주의!)
DELETE FROM test_table;
SELECT * FROM test_table;
```

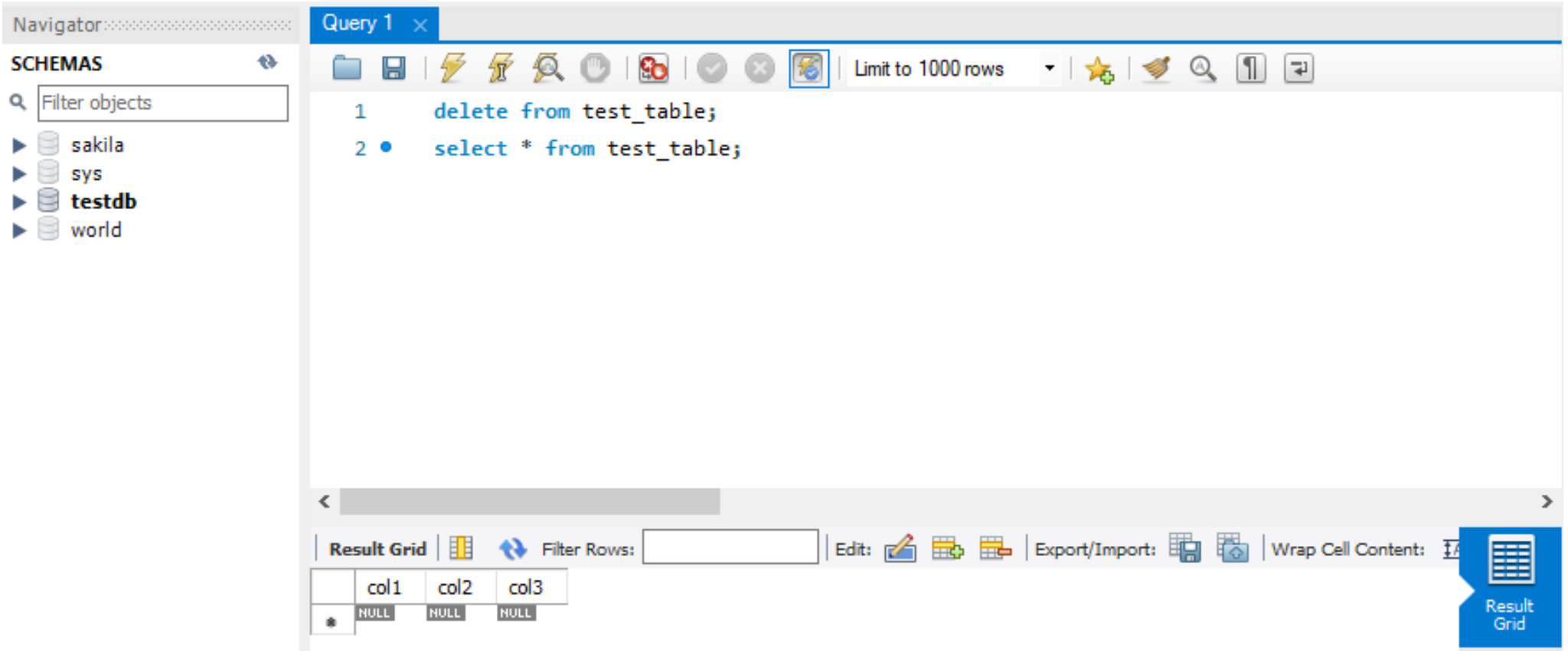
# 데이터 삭제 (DELETE) 실습

- 조건에 맞는 특정 행 삭제 (col1이 2인 행 삭제)



# 데이터 삭제 (DELETE) 실습

- 테이블 내 전체 데이터 삭제



# [종합 실습] 쇼핑몰 회원 DB 구축 및 CRUD 다지기

- 과제: 아래 조건에 맞는 SQL 스크립트를 작성하고 실행하시오.

1. DB 생성: `shopdb` 생성 및 사용 지정
2. 테이블 생성: member_tbl
    - member_id : 정수형, 기본키, 자동증가
    - member_name : 문자형(20), 필수입력
    - member_age : 정수형
    - reg_date : 날짜/시간형
3. 데이터 3건 입력 (INSERT)
    - ('홍길동', 20, '2026-01-01')
    - ('이순신', 45, '2026-01-02')
    - ('강감찬', 50, '2026-01-03')
4. 홍길동의 나이를 21세로 수정 (UPDATE)
5. 강감찬 회원 데이터 삭제 (DELETE)
6. 최종 데이터 조회 (SELECT)

# [종합 실습] 쇼핑몰 회원 DB 구축 및 CRUD 다지기

## - 종합 실습 정답 스크립트

```
CREATE DATABASE IF NOT EXISTS shopdb;
USE shopdb;

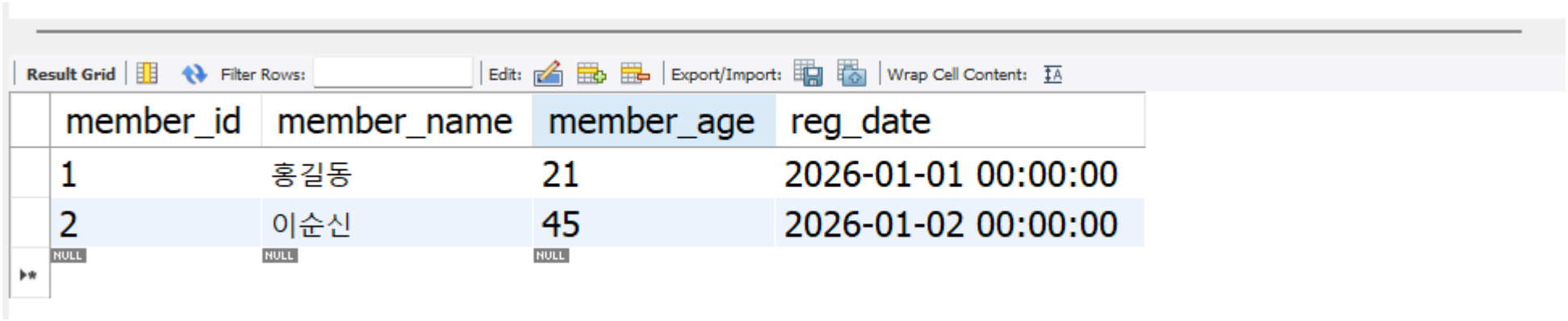
CREATE TABLE member_tbl (
    member_id INT PRIMARY KEY AUTO_INCREMENT,
    member_name VARCHAR(20) NOT NULL,
    member_age INT,
    reg_date DATETIME
);

INSERT INTO member_tbl (member_name, member_age, reg_date) VALUES ('홍길동', 20, '2026-01-01'), ('이순신', 45, '2026-01-02'),
('강감찬', 50, '2026-01-03');

UPDATE member_tbl SET member_age = 21 WHERE member_id = 1;
DELETE FROM member_tbl WHERE member_id = 3;

SELECT * FROM member_tbl;
```

# [종합 실습] 쇼핑몰 회원 DB 구축 및 CRUD 다지기 결과




| member_id | member_name | member_age | reg_date |
|---|---|---|---|
| 1 | 홍길동 | 21 | 2026-01-01 00:00:00 |
| 2 | 이순신 | 45 | 2026-01-02 00:00:00 |
| NULL | NULL | NULL | |

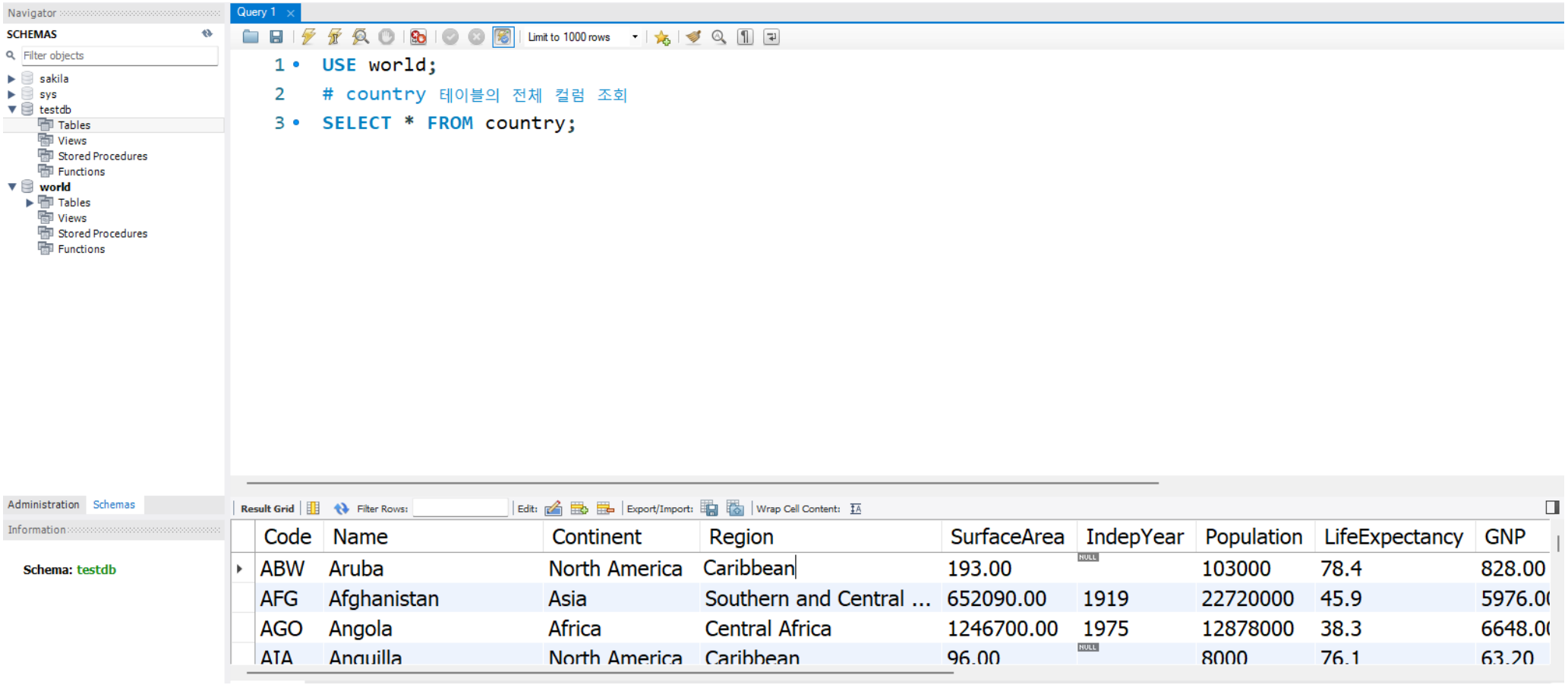
# 데이터 조회 기본 구조 및 샘플 DB 탐색

- SELECT 구문 기본 뼈대:  SELECT [컬럼명1, 컬럼명2 ...] FROM [테이블명];
- world 샘플 데이터베이스 구조
  1) country : 세계 각국의 정보 (국가코드, 국가명, 대륙, 인구, GDP 등)
  2) city : 세계 주요 도시 정보 (도시ID, 도시명, 국가코드, 인구 등)
  3) countrylanguage : 각 국가별 사용 언어 정보
- 실습 스크립트

```
USE world;
# country 테이블의 전체 컬럼 조회
SELECT * FROM country;
# 특정 컬럼만 지정하여 조회 (국가명, 대륙, 인구)
SELECT Name, Continent, Population FROM country;
```

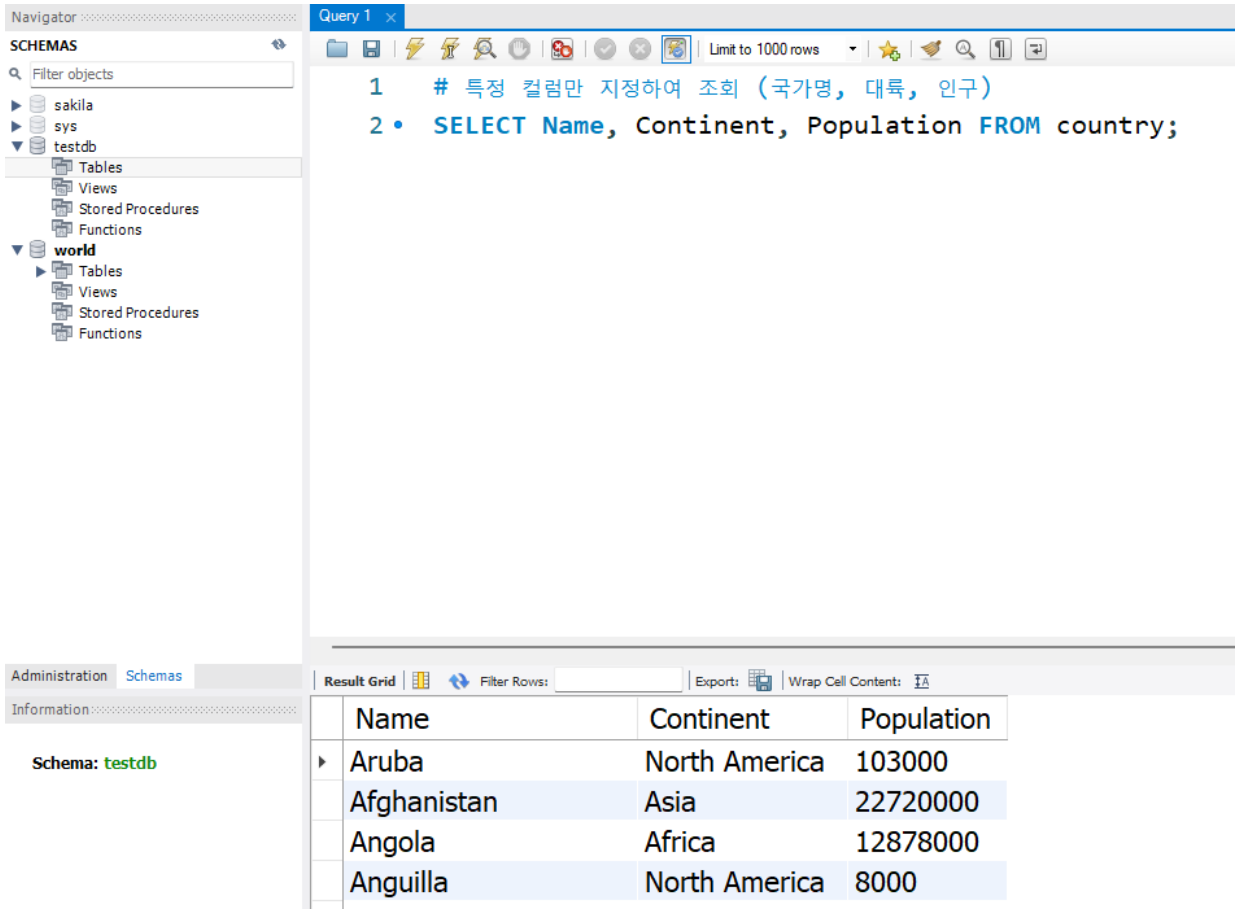
# 데이터 조회 기본 구조 및 샘플 DB 탐색 실습

- country 테이블의 전체 컬럼 조회



# 데이터 조회 기본 구조 및 샘플 DB 탐색 실습

- 특정 컬럼만 지정하여 조회 (국가명, 대륙, 인구)



# WHERE 조건절과 비교/논리 연산자

- WHERE 절

  : 테이블 전체 데이터 중 특정 조건에 맞는 행(Row)만 필터링할 때 사용

- 주요 연산자

  1) 비교 연산자: `=`, `!=`, `>`, `<`, `>=`, `<=`

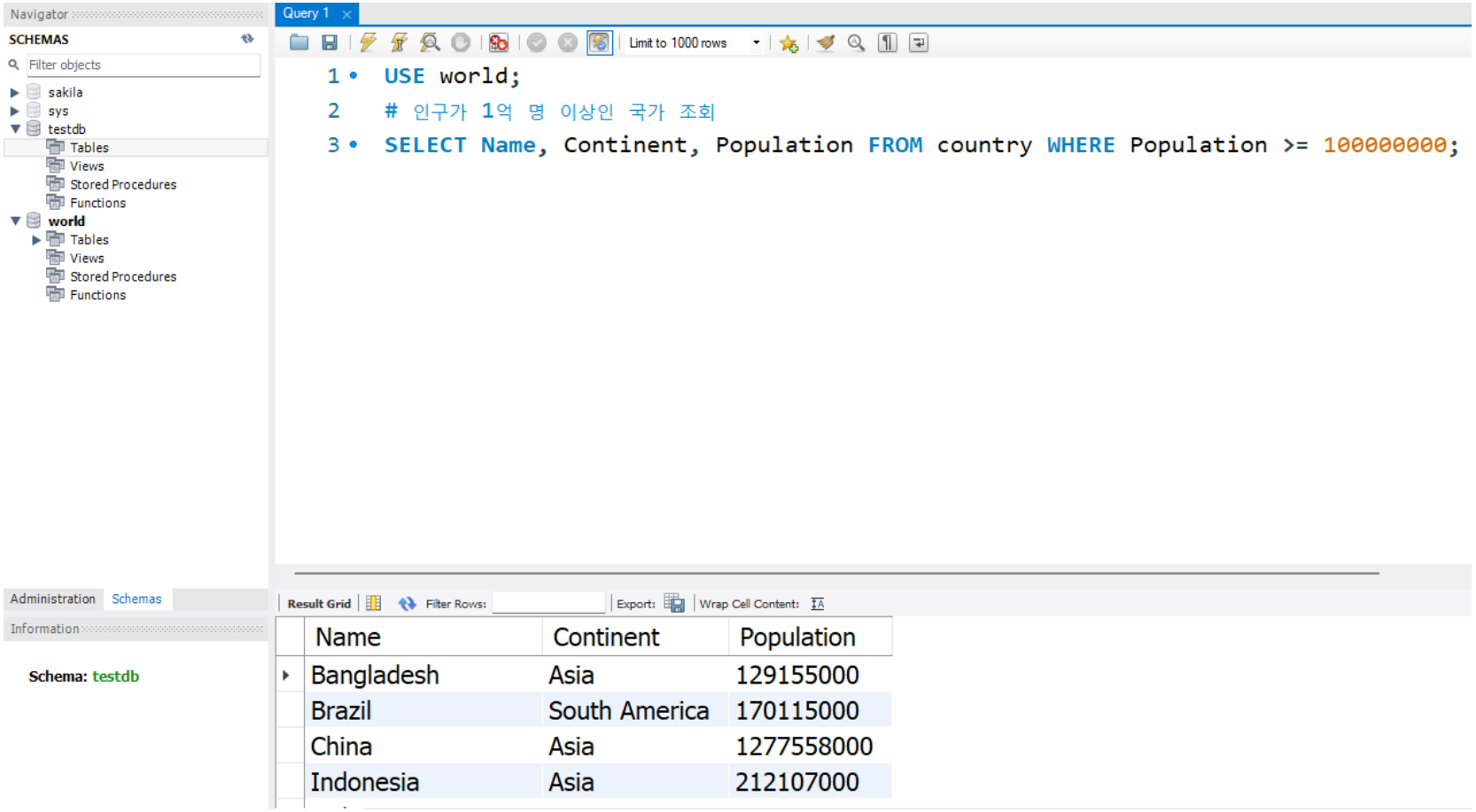
  2) 논리 연산자: `AND` (둘 다 참), `OR` (하나라도 참), `NOT` (조건 부정)

- 실습 스크립트

```
USE world;
# 인구가 1억 명 이상인 국가 조회
SELECT Name, Continent, Population FROM country WHERE Population >= 100000000;
# 대륙이 'Asia'이면서 인구가 5,000만 명 이상인 국가 조회 (AND)
SELECT Name, Continent, Population
FROM country
WHERE Continent = 'Asia' AND Population >= 50000000;
```

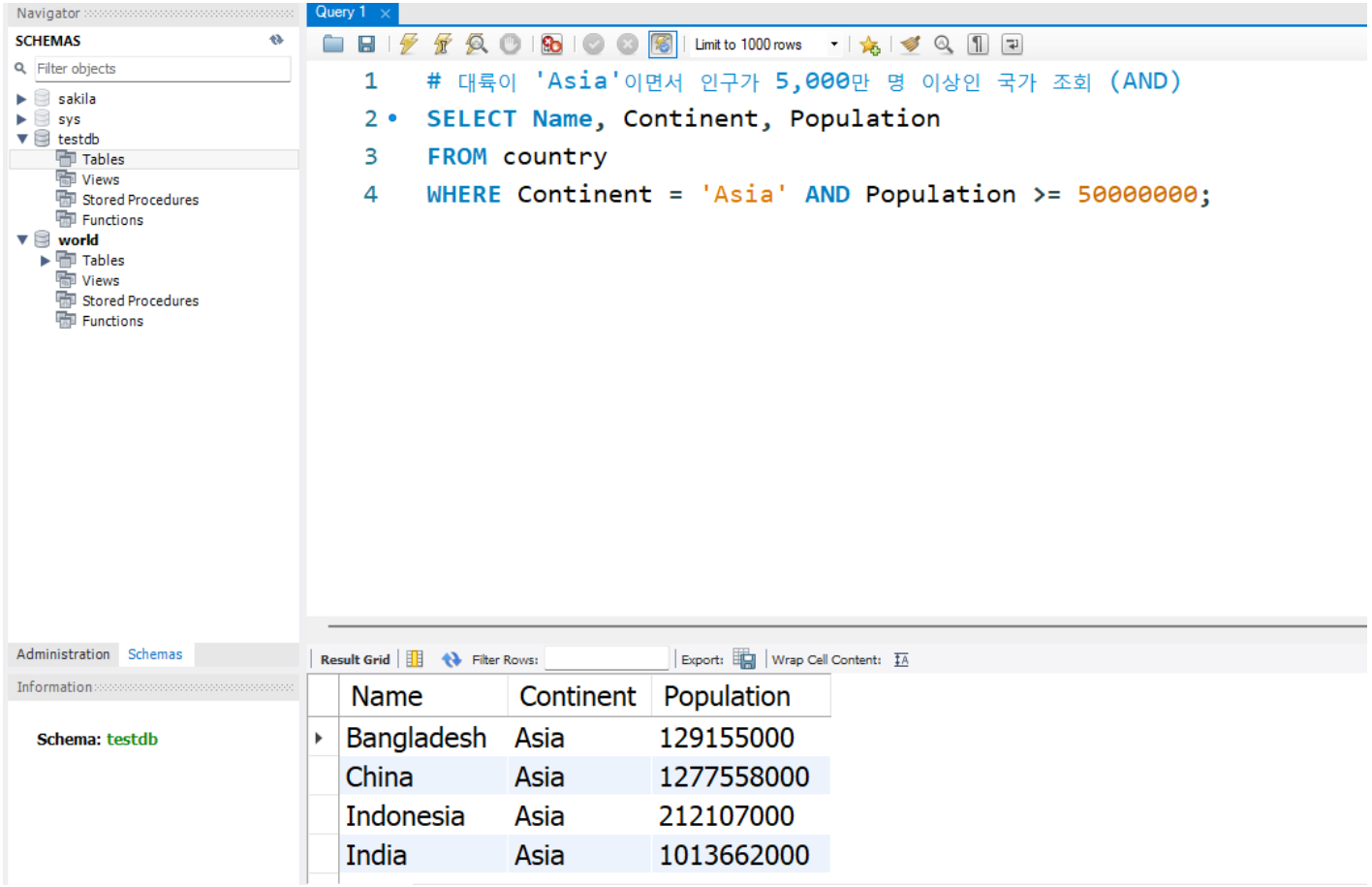
# WHERE 조건절과 비교/논리 연산자 실습

- 인구가 1억 명 이상인 국가 조회



# WHERE 조건절과 비교/논리 연산자 실습

- 대륙이 'Asia'이면서 인구가 5,000만 명 이상인 국가 조회 (AND)



# 범위, 목록, 패턴 검색 (BETWEEN, IN, LIKE)

- BETWEEN A AND B : 연속된 범위 검색 (A 이상 B 이하)
- IN (값1, 값2 ...) : 지정한 목록 중 하나라도 일치하면 검색
- LIKE : 문자열 부분 일치 패턴 검색 (와일드카드 활용)

1) 와일드카드: 문자열의 특정 패턴을 유연하게 검색하기 위해 사용하는 특수 문자로, LIKE 연산자와 함께 사용되어 '정확히 일치하지 않는' 부분 검색이나 형태 검색을 수행함

2)· `%`: 글자 수에 제한 없이 어떤 문자가 와도 허용하는 와일드카드

① LIKE '김%' : '김'으로 시작하는 모든 문자 (예: 김철수, 김, 김민준)

② LIKE '%김%' : '김'이 포함된 모든 문자 (예: 홍길동, 김철수, 서울김치)

③ LIKE '%김' : '김'으로 끝나는 모든 문자 (예: 허균김, 김)

3) `_` : 글자 수가 정확히 일치해야 하며, 언더바 개수만큼의 글자 자리를 강제함

① LIKE '김_' : '김'으로 시작하는 총 2글자 문자 (예: 김민, 김수 / '김철수'는 3글자이므로 조회 안 됨)

② LIKE '_김' : '김'으로 끝나는 총 2글자 문자 (예: 홍김, 박김)

③ LIKE '김__' : '김'으로 시작하는 총 3글자 문자 (예: 김철수, 김민준)

# 범위, 목록, 패턴 검색 (BETWEEN, IN, LIKE) 실습

- 실습 스크립트

```
USE world;

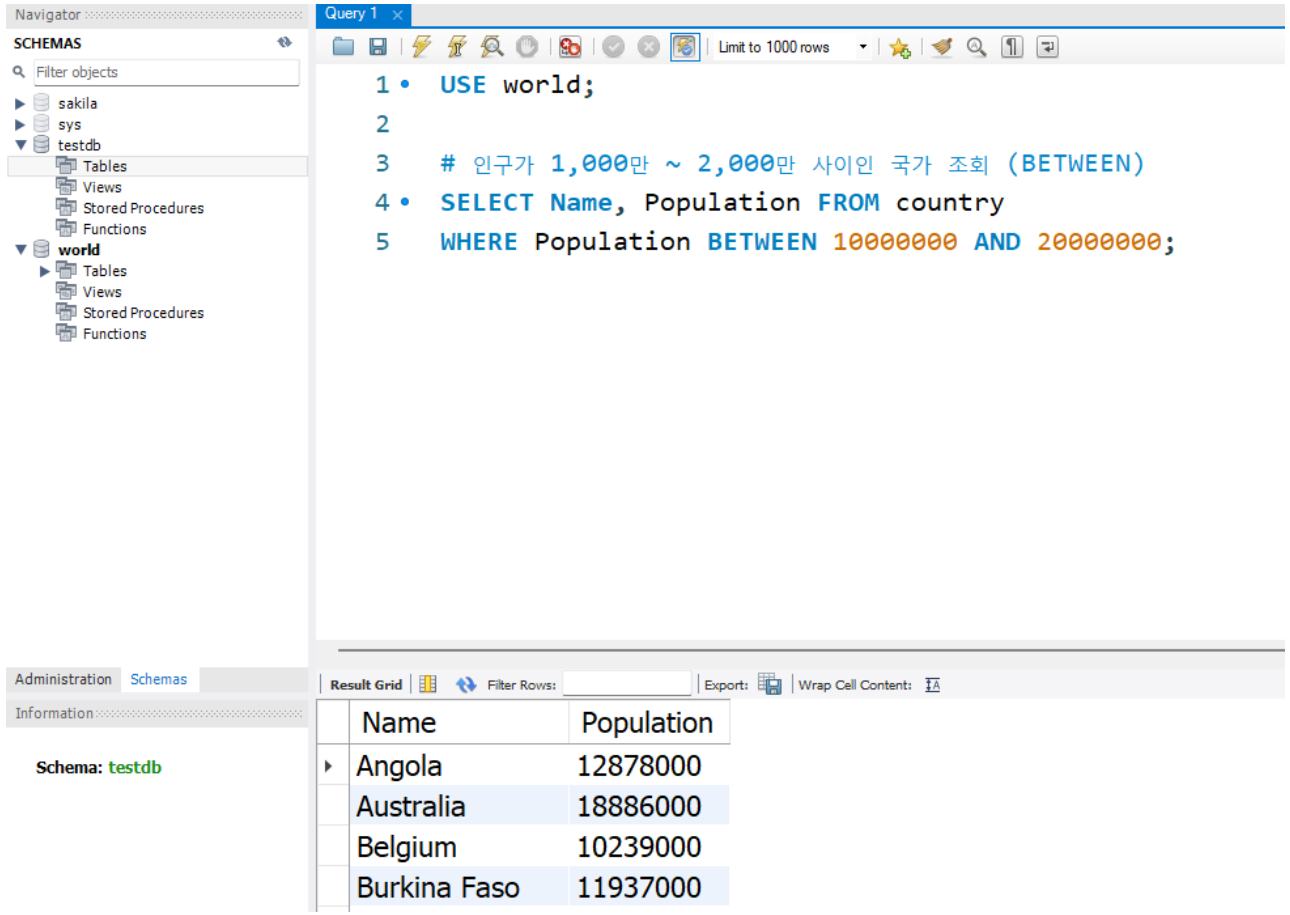
# 인구가 1,000만 ~ 2,000만 사이인 국가 조회 (BETWEEN)
SELECT Name, Population FROM country
WHERE Population BETWEEN 10000000 AND 20000000;

# 대륙이 'Asia', 'Europe', 'North America' 중 하나인 국가 조회 (IN)
SELECT Name, Continent FROM country
WHERE Continent IN ('Asia', 'Europe', 'North America');

# 국가명이 'South'로 시작하는 국가 조회 (LIKE)
SELECT Name FROM country WHERE Name LIKE 'South%';
```

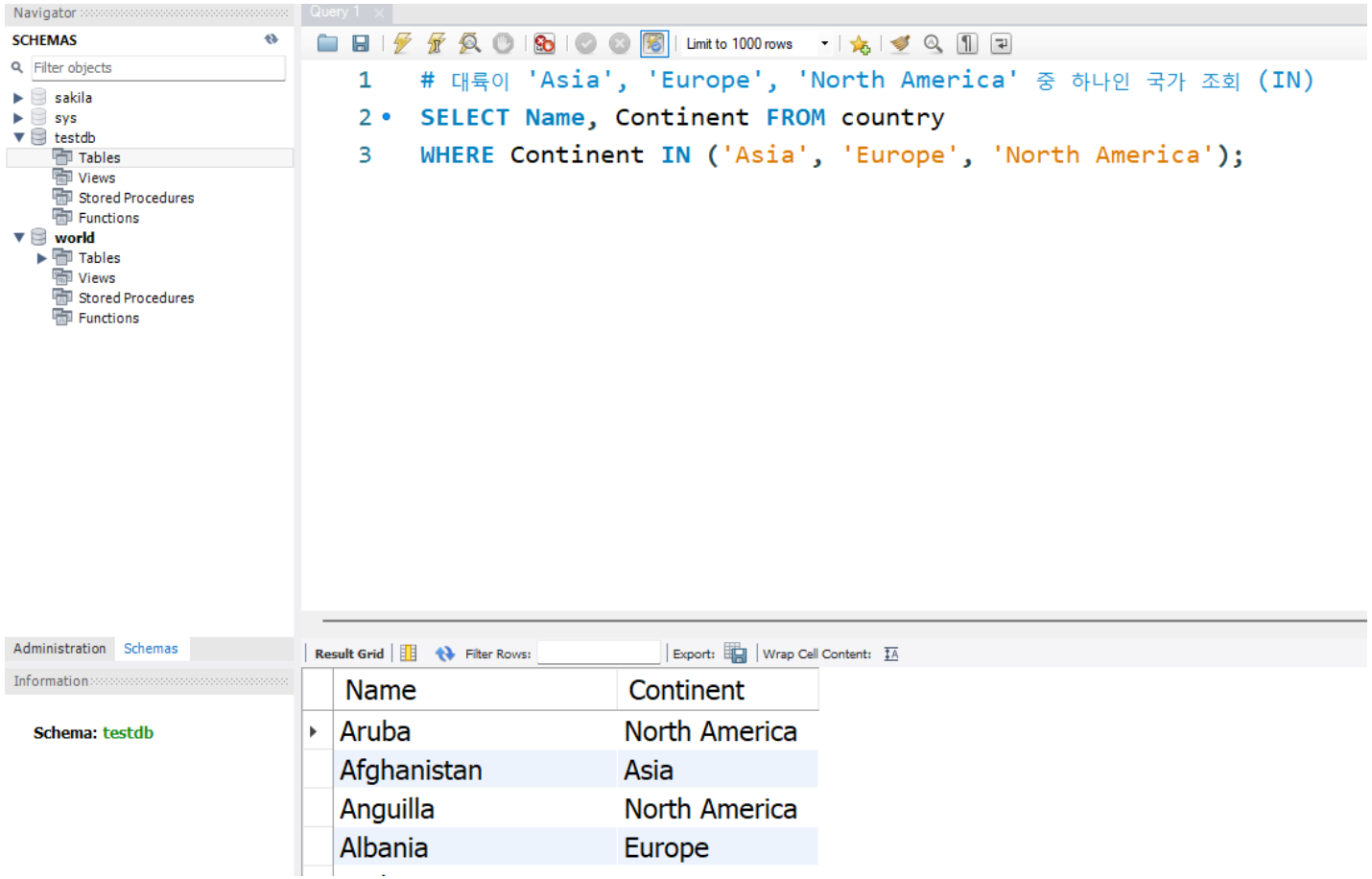
# 범위, 목록, 패턴 검색 (BETWEEN, IN, LIKE) 실습

- BETWEEN



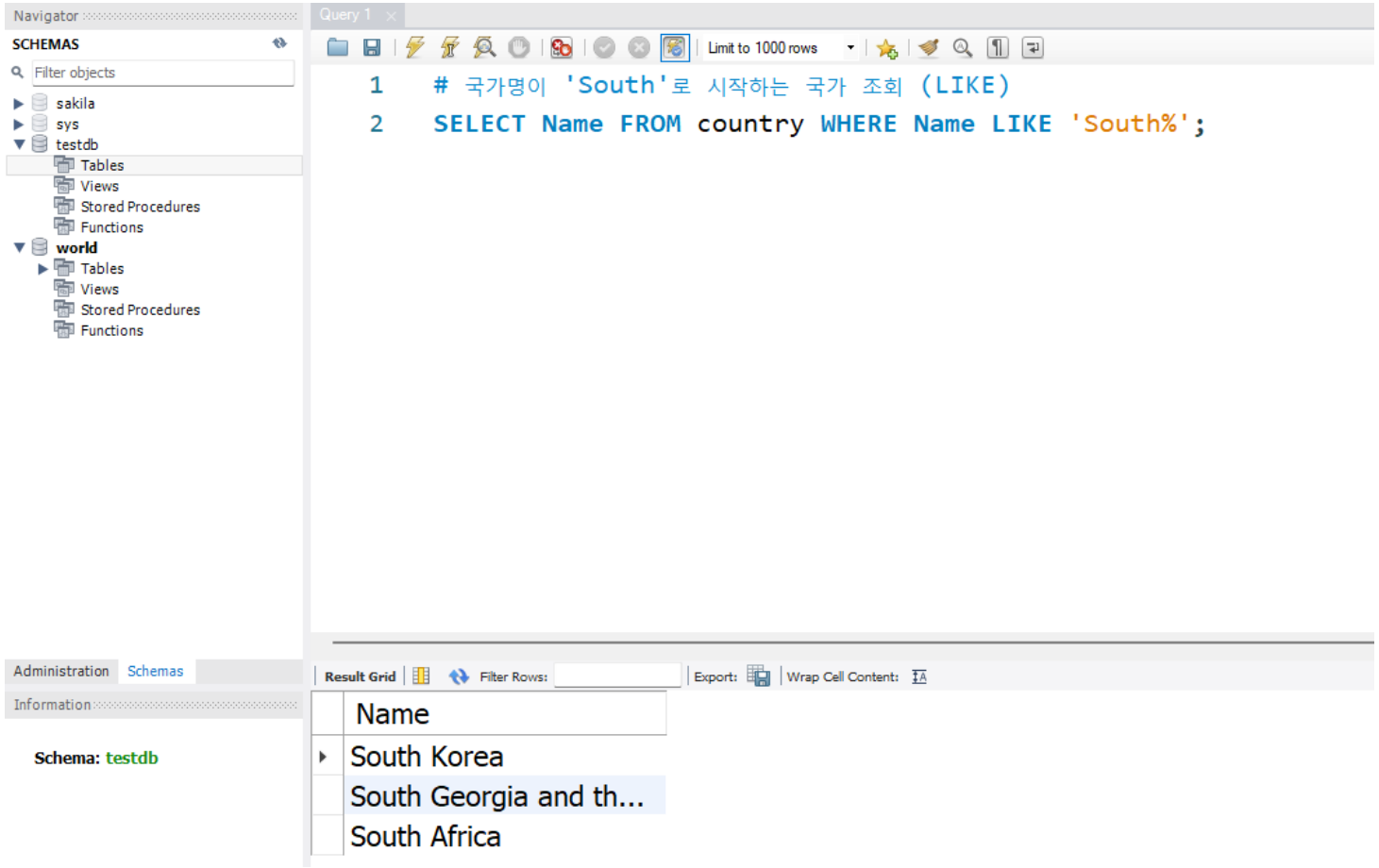
# 범위, 목록, 패턴 검색 (BETWEEN, IN, LIKE) 실습

- IN



# 범위, 목록, 패턴 검색 (BETWEEN, IN, LIKE) 실습

- LIKE



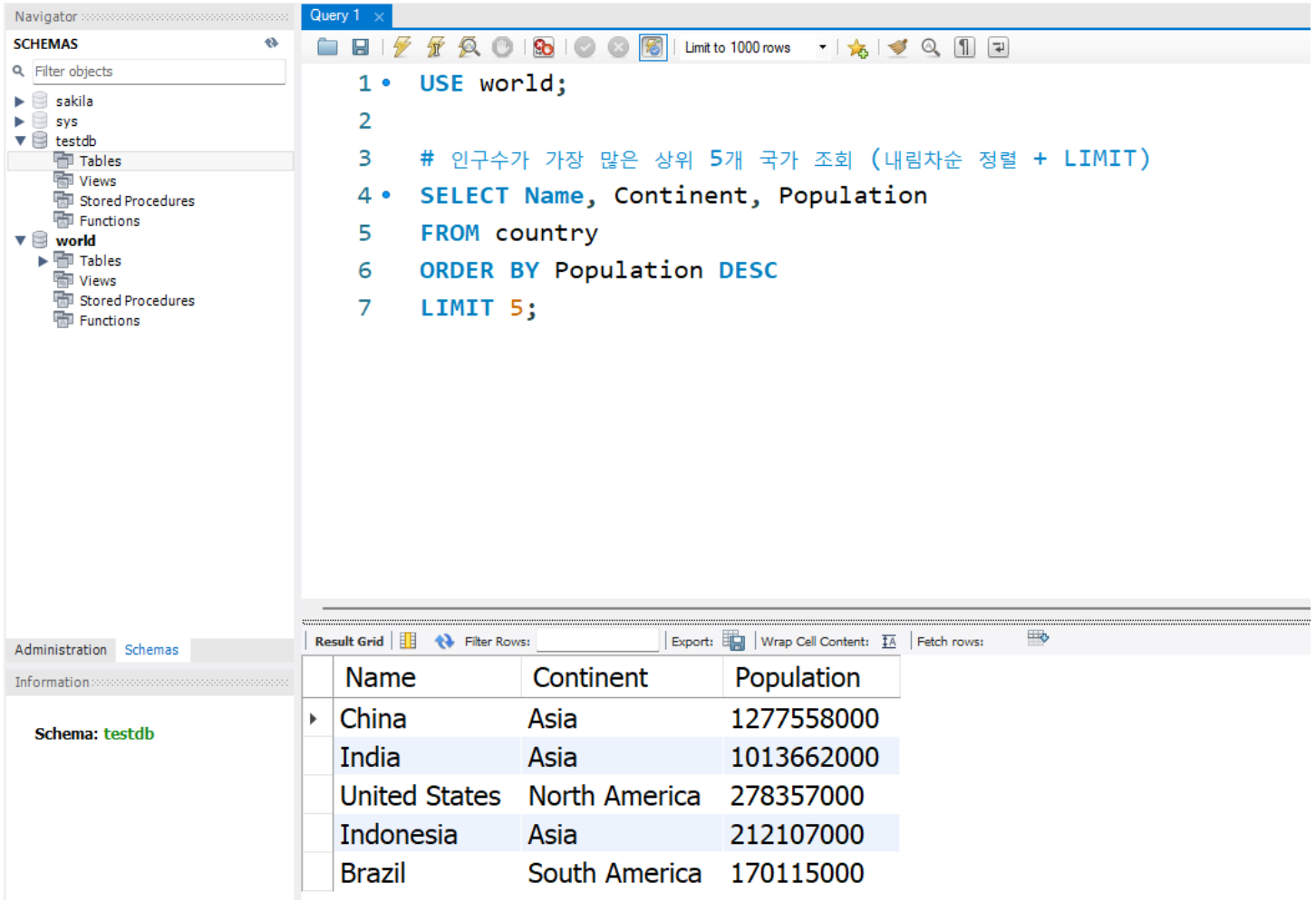
# 조회 결과 정렬(ORDER BY) 및 개수 제한(LIMIT)

- ORDER BY 절
  : 조회된 결과를 특정 컬럼 기준으로 오름차순(`ASC`, 기본값) 또는 내림차순(`DESC`) 정렬
- LIMIT 절:
  : 출력할 결과의 개수를 제한 (Top-N 추출)
- 실습 스크립트

```
USE world;

# 인구수가 가장 많은 상위 5개 국가 조회 (내림차순 정렬 + LIMIT)
SELECT Name, Continent, Population
FROM country
ORDER BY Population DESC
LIMIT 5;
```

# 조회 결과 정렬(ORDER BY) 및 개수 제한(LIMIT) 실습



# world 데이터베이스 조건별 실전 조회 실습

- 실습 문제

1) `city` 테이블에서 한국(CountryCode = 'KOR') 도시 중 인구가 100만 명 이상인 도시의 이름(Name)과 인구(Population)를 인구수 내림차순으로 조회하시오.

2) `country` 테이블에서 기대수명(LifeExpectancy)이 80세 이상인 국가의 Name, Continent, LifeExpectancy를 조회하시오.

# world 데이터베이스 조건별 실전 조회 실습

- 실습 정답 스크립트

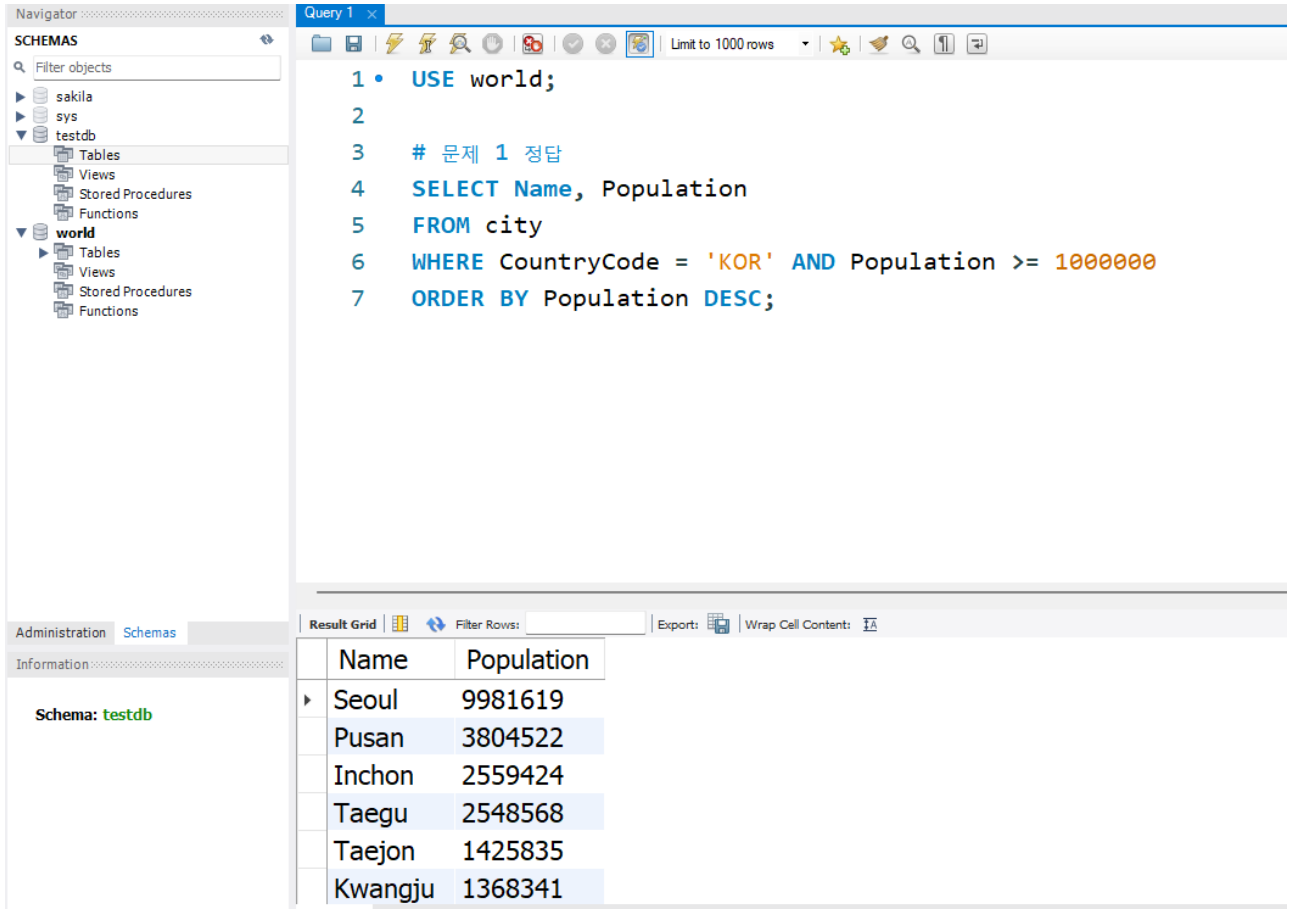
```
USE world;

# 문제 1 정답
SELECT Name, Population
FROM city
WHERE CountryCode = 'KOR' AND Population >= 1000000
ORDER BY Population DESC;

# 문제 2 정답
SELECT Name, Continent, LifeExpectancy
FROM country
WHERE LifeExpectancy >= 80;
```

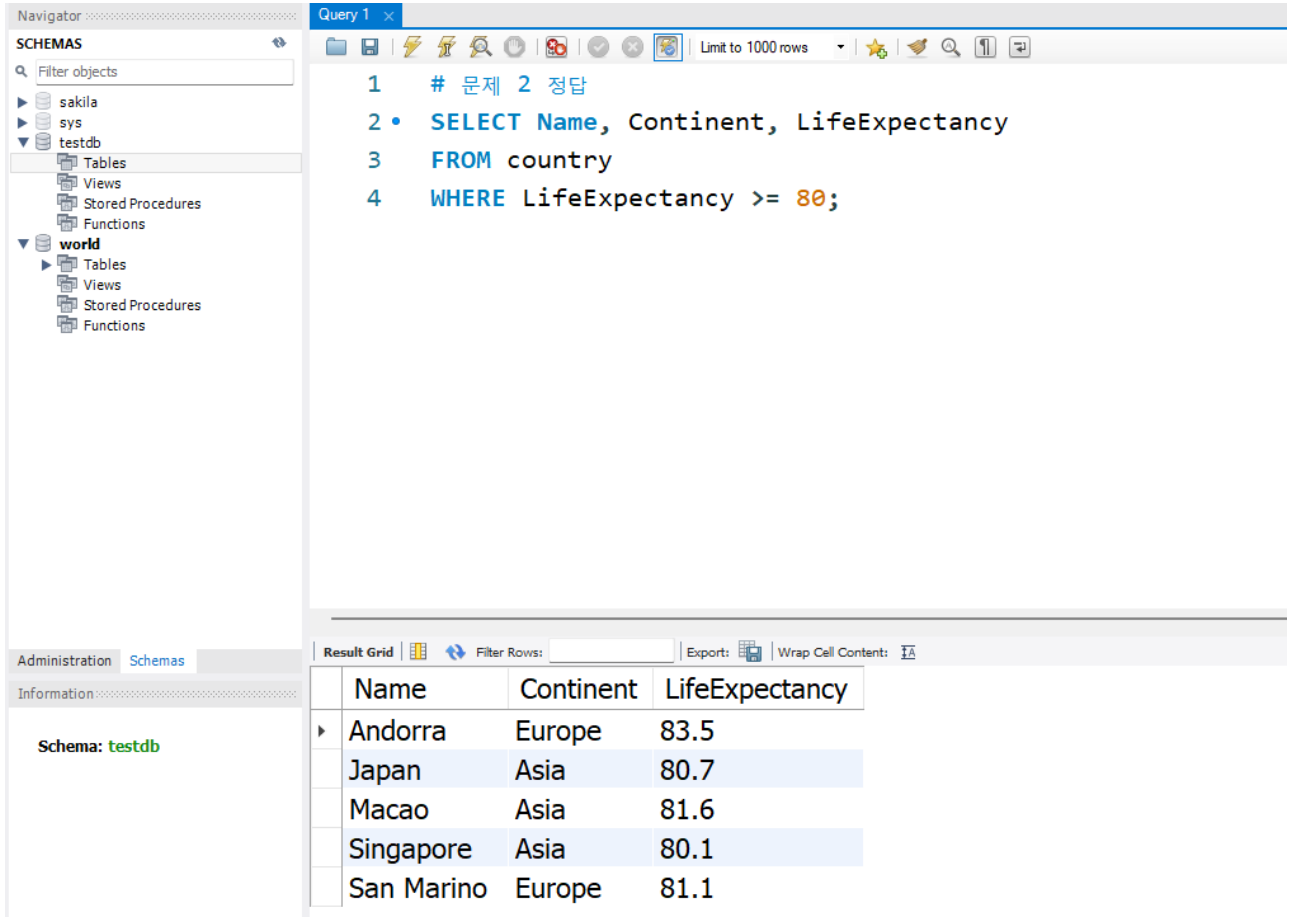
# world 데이터베이스 조건별 실전 조회 실습

- 문제 1 정답



# world 데이터베이스 조건별 실전 조회 실습

- 문제 2 정답



# 주요 집계 함수 (SUM, AVG, COUNT, MAX, MIN)

- 집계 함수란?
  1) 여러 행의 데이터를 입력받아 단 하나의 요약된 결과값을 계산하는 함수
  2) NULL 값은 자동으로 계산 대상에서 제외됨

| 함수 | 설명 | NULL 값 처리 |
|---|---|---|
| **COUNT()** | 행(레코드)의 개수를 반환 | COUNT(*)는 포함 |
| **SUM()** | 컬럼 값들의 총합을 계산 | 제외 |
| **AVG()** | 컬럼 값들의 평균을 계산 | 제외 |
| **MAX()** | 컬럼 값 중 최대값을 반환 | 제외 |
| **MIN()** | 컬럼 값 중 최소값을 반환 | 제외 |

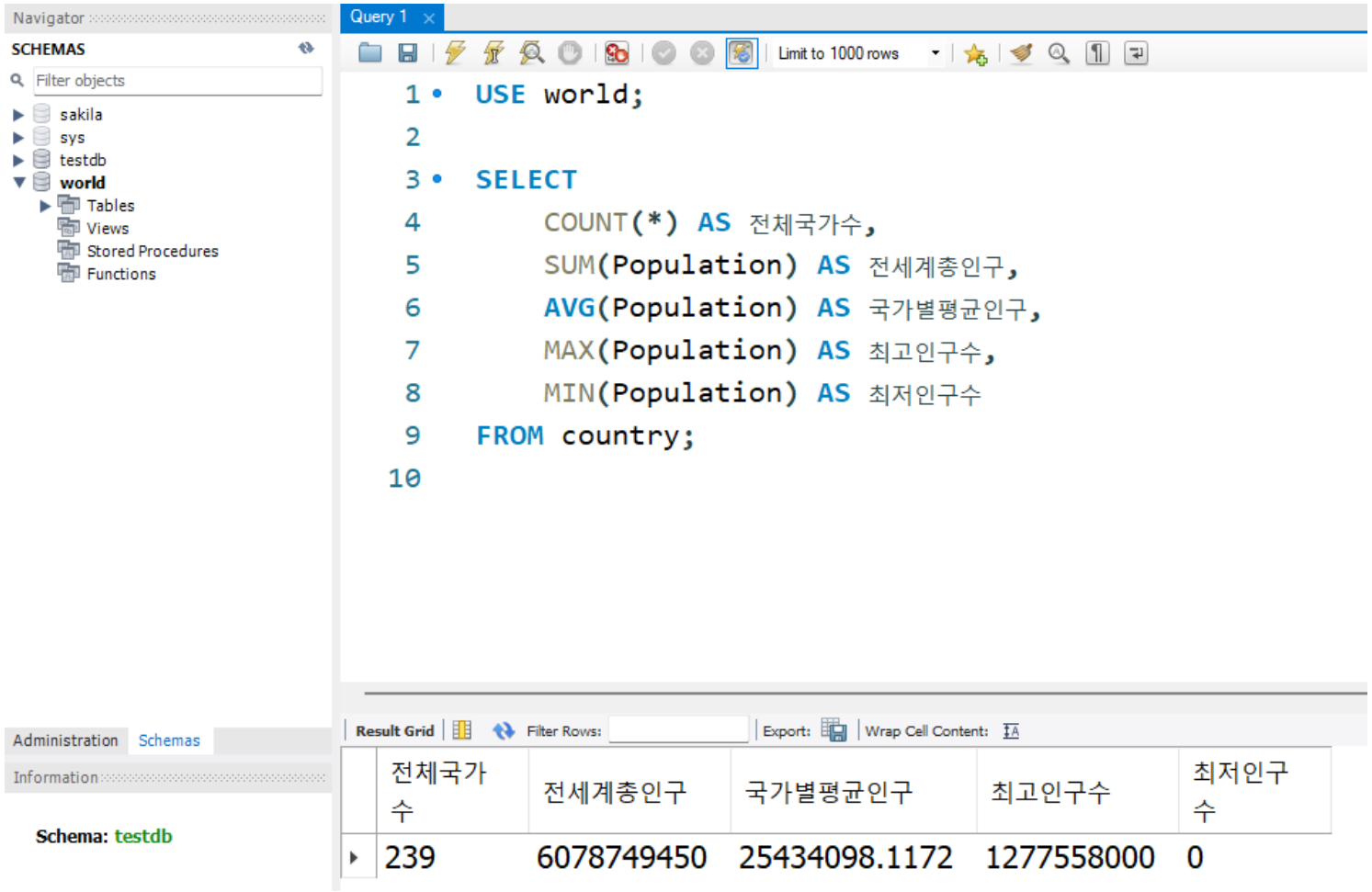
# 주요 집계 함수 (SUM, AVG, COUNT, MAX, MIN) 실습

- 실습 스크립트

```
USE world;


SELECT
    COUNT(*) AS 전체국가수,
    SUM(Population) AS 전세계총인구,
    AVG(Population) AS 국가별평균인구,
    MAX(Population) AS 최고인구수,
    MIN(Population) AS 최저인구수
FROM country;
```

# 주요 집계 함수 (SUM, AVG, COUNT, MAX, MIN) 실습



# 데이터 그룹화 (GROUP BY)

- GROUP BY 절
  1) 특정 컬럼의 값이 같은 데이터끼리 묶어서 그룹별 집계 통계(합계, 평균 등)를 낼 때 사용
  2) Pandas groupby 함수와 같은 기능

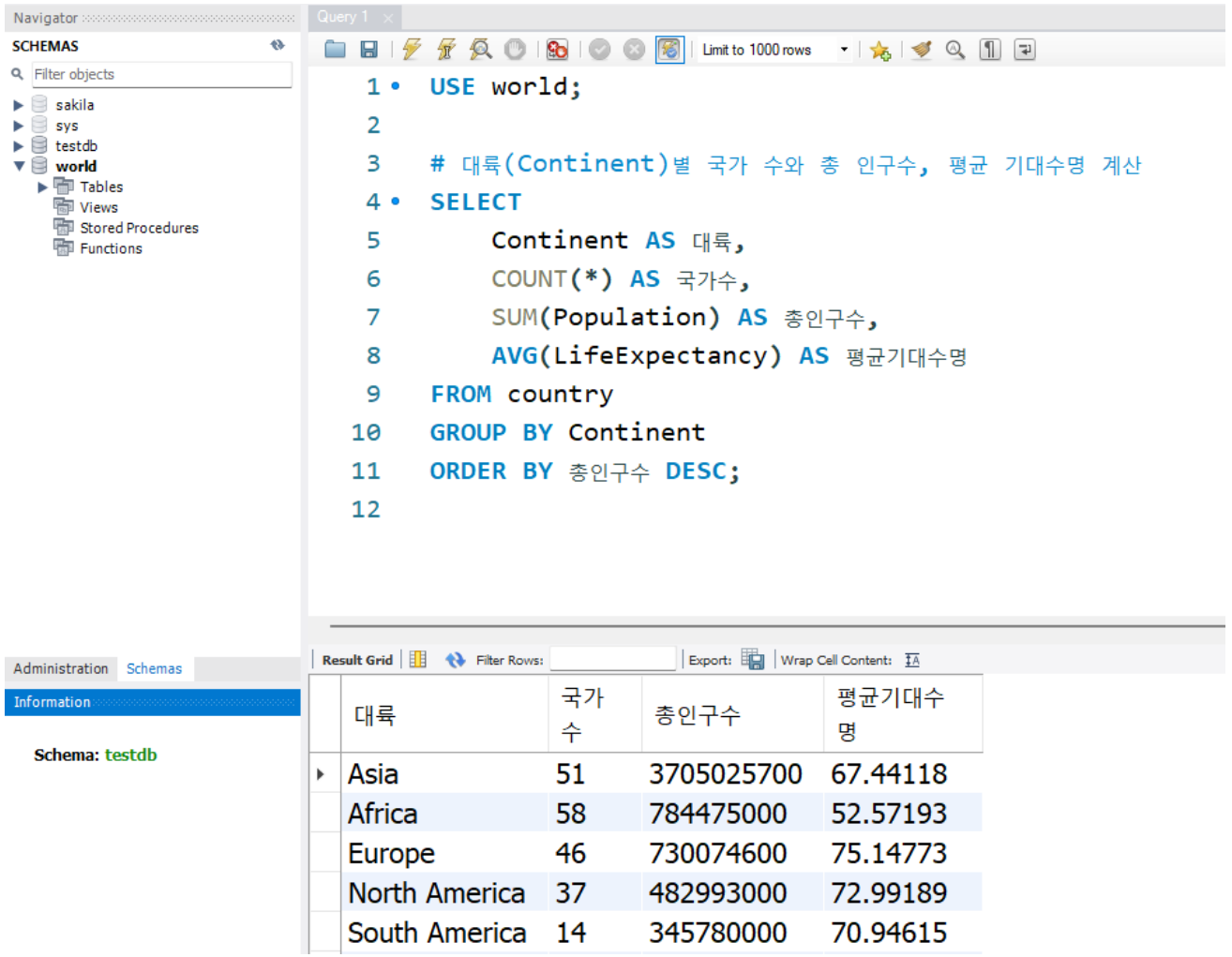
# 데이터 그룹화 (GROUP BY) 실습

- 실습 스크립트

```
USE world;


# 대륙(Continent)별 국가 수와 총 인구수, 평균 기대수명 계산
SELECT
    Continent AS 대륙,
    COUNT(*) AS 국가수,
    SUM(Population) AS 총인구수,
    AVG(LifeExpectancy) AS 평균기대수명
FROM country
GROUP BY Continent
ORDER BY 총인구수 DESC;
```

# 데이터 그룹화 (GROUP BY) 실습



# 그룹 조건절 HAVING vs WHERE 절의 차이

- WHERE 절과 HAVING 절의 핵심 차이점

1) WHERE 절

① 개별 행(Row) 단위의 1차 필터링

② 집계 함수 사용 불가

③ 데이터를 그룹으로 묶기 전에 필터링하려면 WHERE를 사용

2) HAVING` 절

① 그룹화된 결과(Group BY)에 대한  2차 필터링,

② 그룹으로 묶은 후에 집계 결과(예: 평균, 합계 등)를 기준으로 필터링하려면 HAVING을 사용

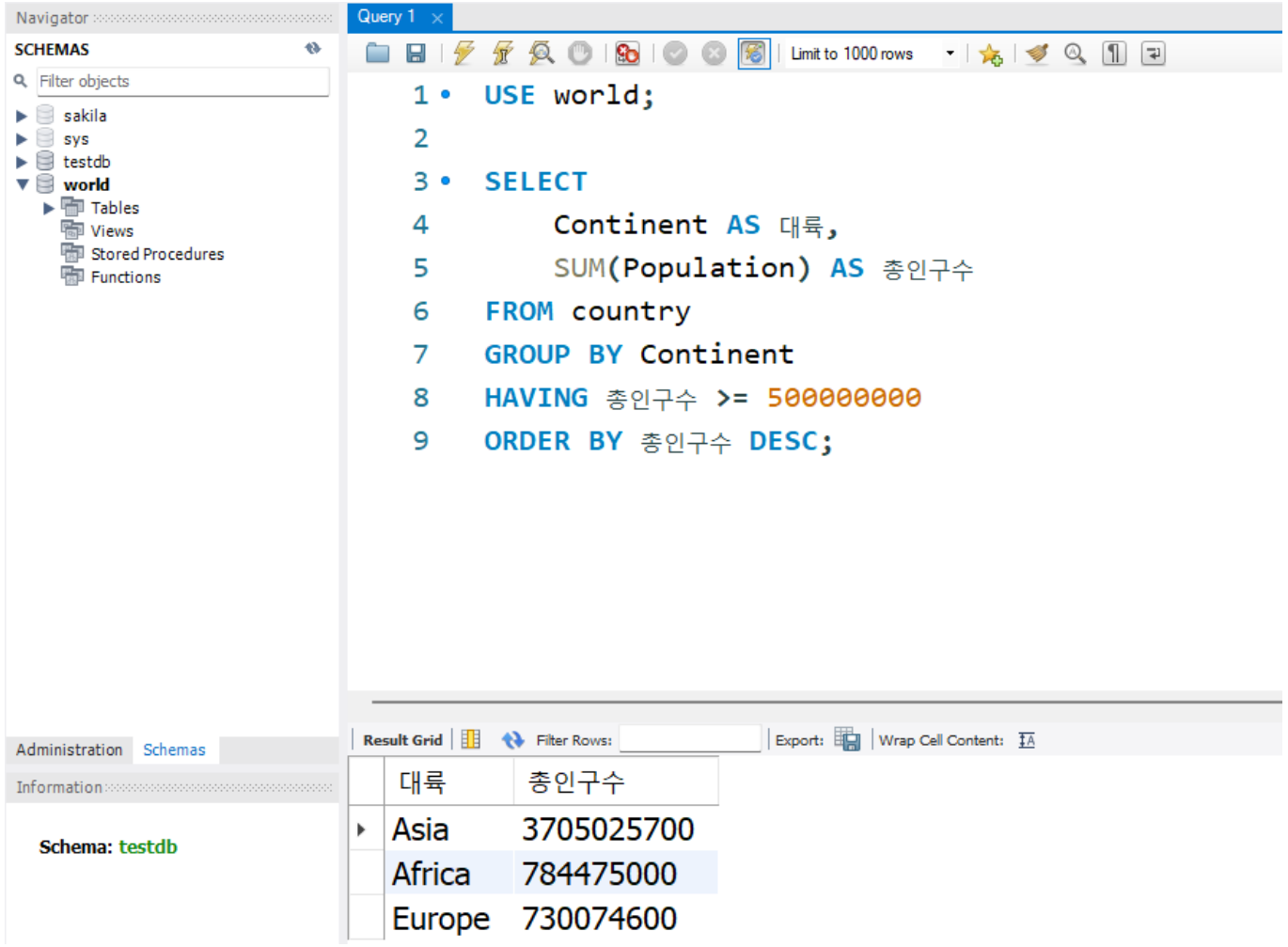
# 그룹 조건절 HAVING vs WHERE 절의 차이 실습

- 실습 스크립트

```
USE world;


# 총 인구수가 5억 명 이상인 대륙만 필터링하여 대륙과 인구수 출력 (HAVING)
SELECT
    Continent AS 대륙,
    SUM(Population) AS 총인구수
FROM country
GROUP BY Continent
HAVING 총인구수 >= 500000000
ORDER BY 총인구수 DESC;
```

# 그룹 조건절 HAVING vs WHERE 절의 차이 실습



# 조건부 데이터 입력 및 수정

- 데이터 입력 시 중복이 발생했을 때 단순히 에러를 무시하고 건너뛰고 싶다면 INSERT IGNORE를 쓰고, 기존 데이터를 새로운 값으로 고치거나 누적시키고 싶다면 ON DUPLICATE KEY UPDATE를 쓴다
- INSERT IGNORE
  : 데이터를 입력할 때 기본키(PK) 중복 에러가 발생하면 중복 에러 발생 시 에러를 무시하고 해당 행 삽입을 건너뜀 (Skip)
- ON DUPLICATE KEY UPDATE
  1) 기본키(PK)가 중복되지 않으면 신규 삽입(INSERT) 수행
  2) 기본키(PK)가 중복되면 기존 데이터를 지정한 값으로 자동 수정(UPDATE) 수행

# 조건부 데이터 입력 및 수정 실습

## - 실습 스크립트

```
USE testdb;

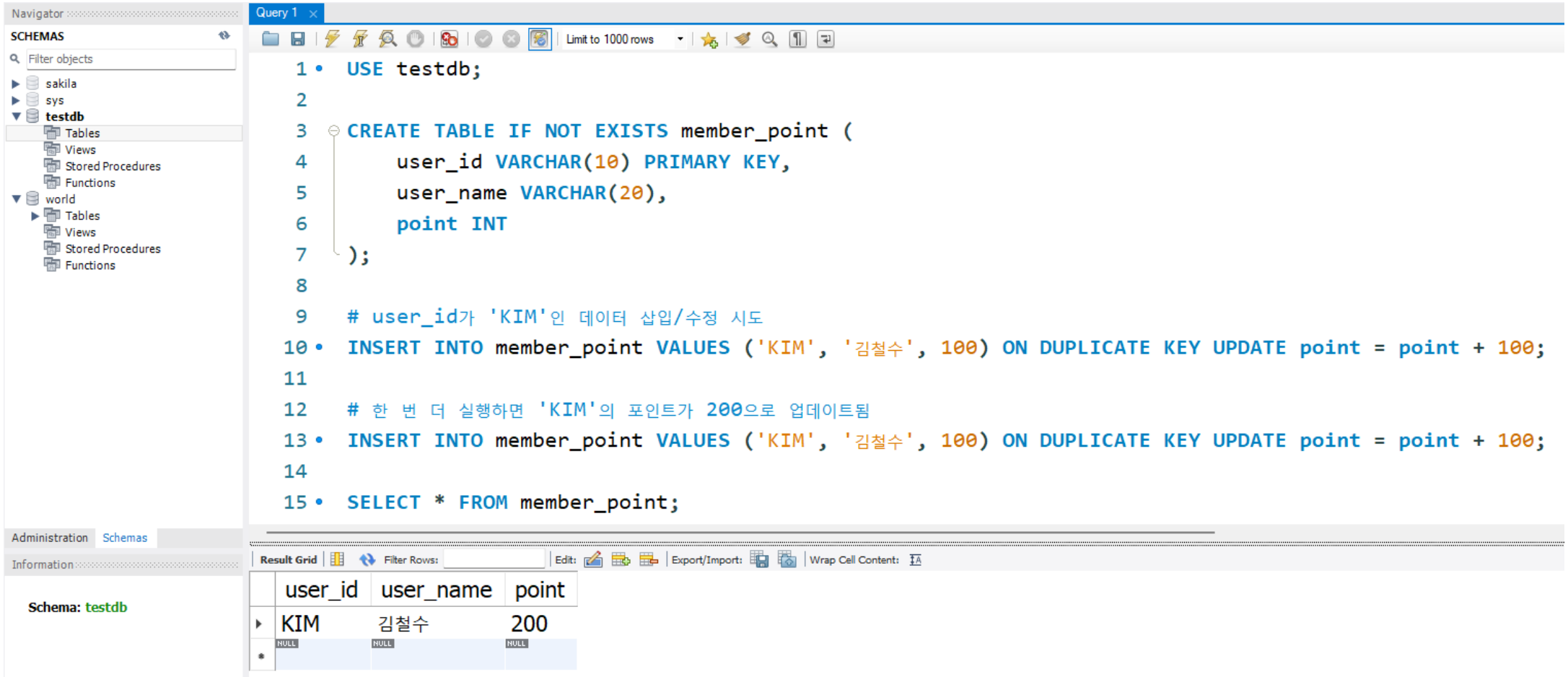
CREATE TABLE IF NOT EXISTS member_point (
    user_id VARCHAR(10) PRIMARY KEY,
    user_name VARCHAR(20),
    point INT
);

# user_id가 'KIM'인 데이터 삽입/수정 시도
INSERT INTO member_point VALUES ('KIM', '김철수', 100) ON DUPLICATE KEY UPDATE point = point + 100;

# 한 번 더 실행하면 'KIM'의 포인트가 200으로 업데이트됨
INSERT INTO member_point VALUES ('KIM', '김철수', 100) ON DUPLICATE KEY UPDATE point = point + 100;

SELECT * FROM member_point;
```

# 조건부 데이터 입력 및 수정 실습



# world & sakila 집계 분석 실습

- 실습 문제

1) `world.city` 테이블에서 국가코드(CountryCode)별 도시 개수를 구하고, 도시 개수가 10개 이상인 국가코드와 도시 수를 도시 수 내림차순으로 조회하시오.

2) `sakila.payment` 테이블에서 고객별(customer_id) 총 결제 금액(amount)을 구하고, 고객(customer_id)과 해당 고객의 총 결제 금액을 내림차순으로 조회하시오.

# world & sakila 집계 분석 실습 실습

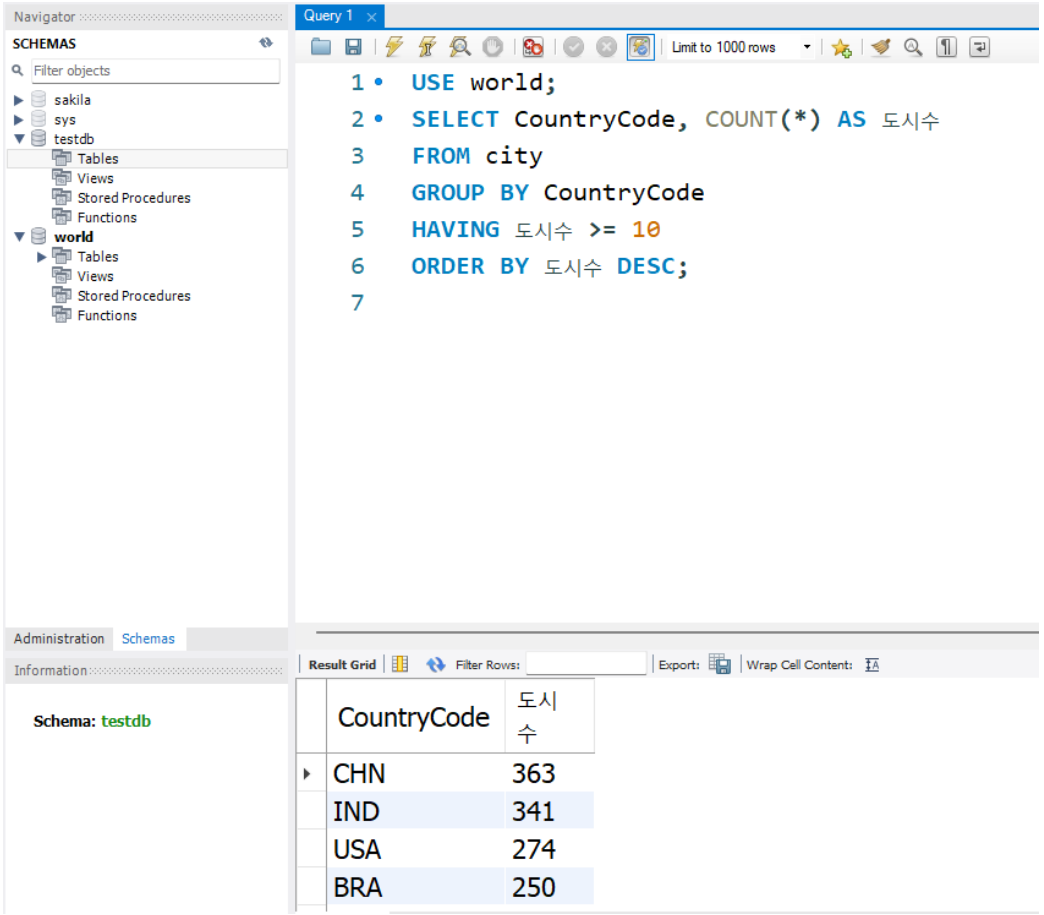
## - 실습 정답 스크립트

```
# 문제 1 정답
USE world;
SELECT CountryCode, COUNT(*) AS 도시수
FROM city
GROUP BY CountryCode
HAVING 도시수 >= 10
ORDER BY 도시수 DESC;

# 문제 2 정답
USE sakila;
SELECT customer_id, SUM(amount) AS 총결제금액
FROM payment
GROUP BY customer_id
ORDER BY 총결제금액 DESC;
```

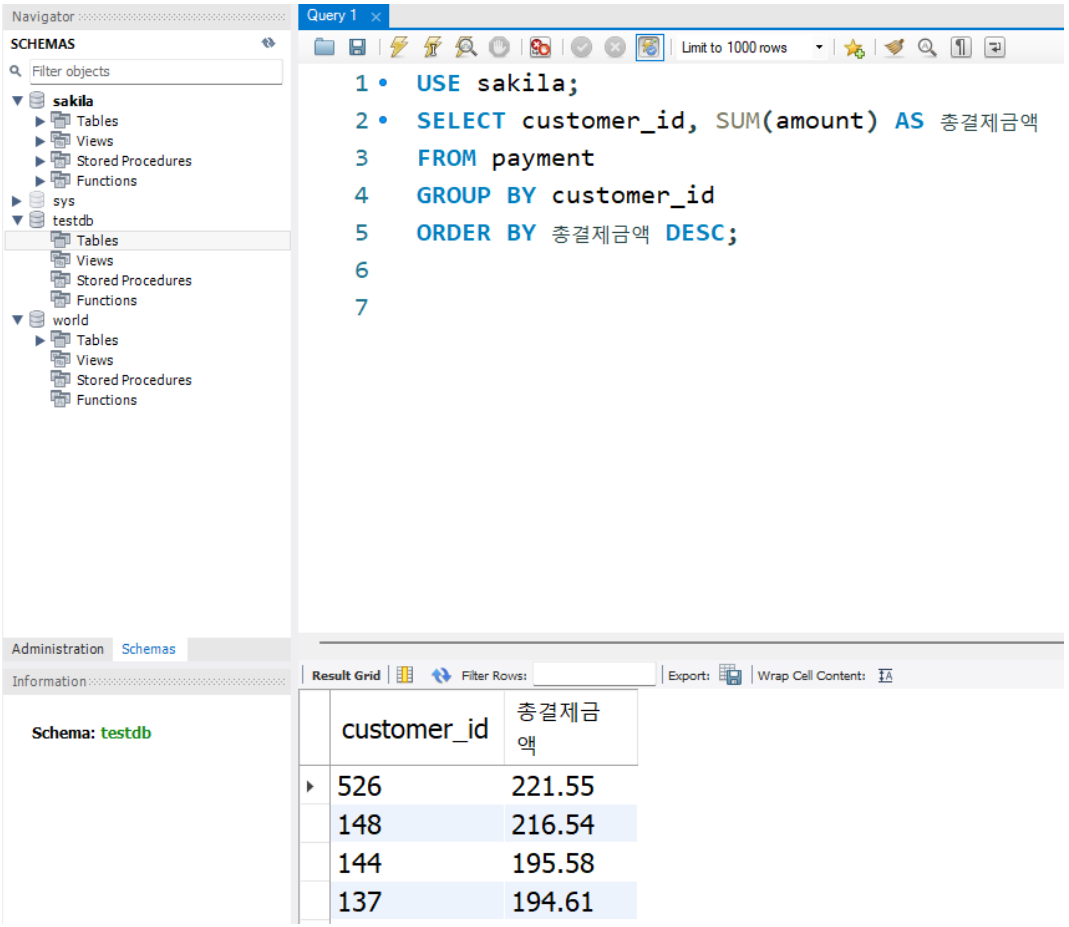
# world & sakila 집계 분석 실습

- 문제 1 정답



# world & sakila 집계 분석 실습

- 문제 2 정답



# MySQL 주요 데이터 타입 세부 정리

| 분류 | 데이터 타입 | 설명 및 범위 |
|---|---|---|
| **정수형** | TINYINT, SMALLINT, INT, BIGINT | INT(4 Byte, 약 -21억~21억)<br>BIGINT(8 Byte, 대용량 ID/금액) |
| **실수형** | FLOAT, DOUBLE, DECIMAL | DECIMAL(P, S) : 오차 없는 정확한 실수 |
| **문자형** | CHAR(N), VARCHAR(N), TEXT, LONGTEXT | CHAR(고정길이), VARCHAR(가변길이)<br>LONGTEXT(최대 4GB 대용량 텍스트) |
| **날짜/시간형** | DATE, TIME, DATETIME | DATE(YYYY-MM-DD),<br>DATETIME(시분초 포함) |

# 사용자 정의 변수 (@변수)

- 사용자 정의 변수 (@변수명)
  1) 개념: MySQL 워크벤치에서 로그아웃하기 전(세션이 유지되는 동안)까지 프로그램 전체에서 살아있는 사용자 정의 변수
  2) 사용 방법: SET @변수명 = 값;

# 사용자 정의 변수 (@변수) 실습

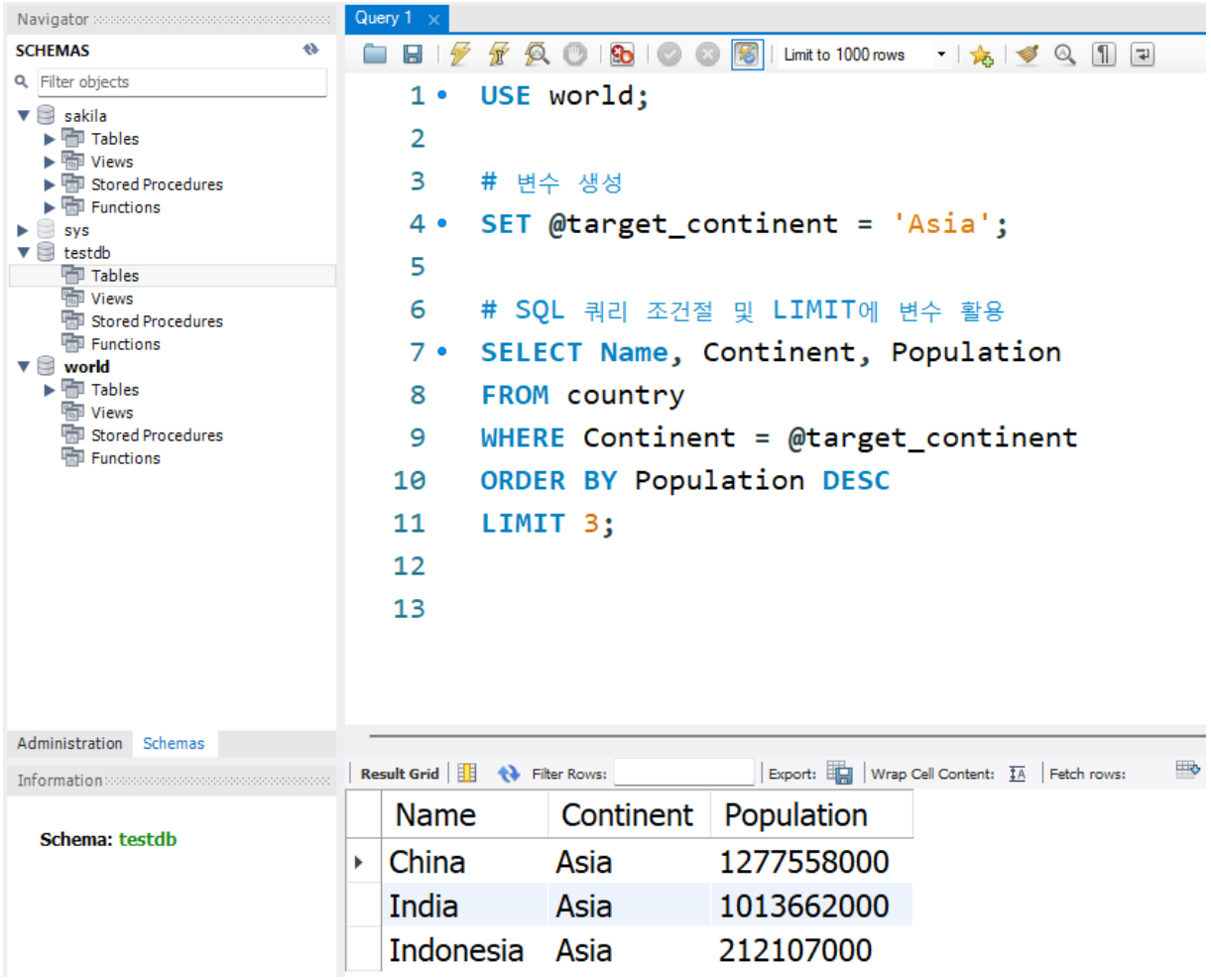
- 실습 스크립트

```
USE world;

# 변수 생성
SET @target_continent = 'Asia';

# SQL 쿼리 조건절 및 LIMIT에 변수 활용
SELECT Name, Continent, Population
FROM country
WHERE Continent = @target_continent
ORDER BY Population DESC
LIMIT 3;
```

# 사용자 정의 변수 (@변수) 실습



# 명시적 형변환과 묵시적 형변환

- 명시적 형변환 (Explicit Conversion)

  1) 개발자가 CAST 또는 CONVERT 함수를 사용하여 데이터 타입을 직접 강제로 변경하는 방식

  2) 기본 문법

  ① CAST(값 AS 데이터타입)

  ② CONVERT(값, 데이터타입)

  3) 핵심 주의사항 (타입 지정 제약)

  ① 테이블을 생성할 때 사용하는 일반 데이터 타입(INT, VARCHAR, FLOAT 등)을 형변환 함수 안에 그대로 사용하면 문법 에러(Syntax Error)가 발생함

  ② 형변환 함수 내부에서는 반드시 전용 타입을 사용해야 함

  • 정수형 변환 시: SIGNED (또는 UNSIGNED) 사용 (※ INT 사용 불가)

  • 문자열형 변환 시: CHAR (또는 CHAR(N)) 사용 (※ VARCHAR 사용 불가)

  • 실수형 변환 시: DECIMAL 사용 (※ FLOAT, DOUBLE 사용 불가)

- 묵시적 형변환 (Implicit Conversion)

  : 사용자가 직접 형변환 함수를 쓰지 않아도, 연산자나 문맥에 따라 MySQL이 데이터 타입을 알아서 판단하여 자동으로 변환해 주는 기능

# 명시적 형변환과 묵시적 형변환 실습

- 실습 스크립트

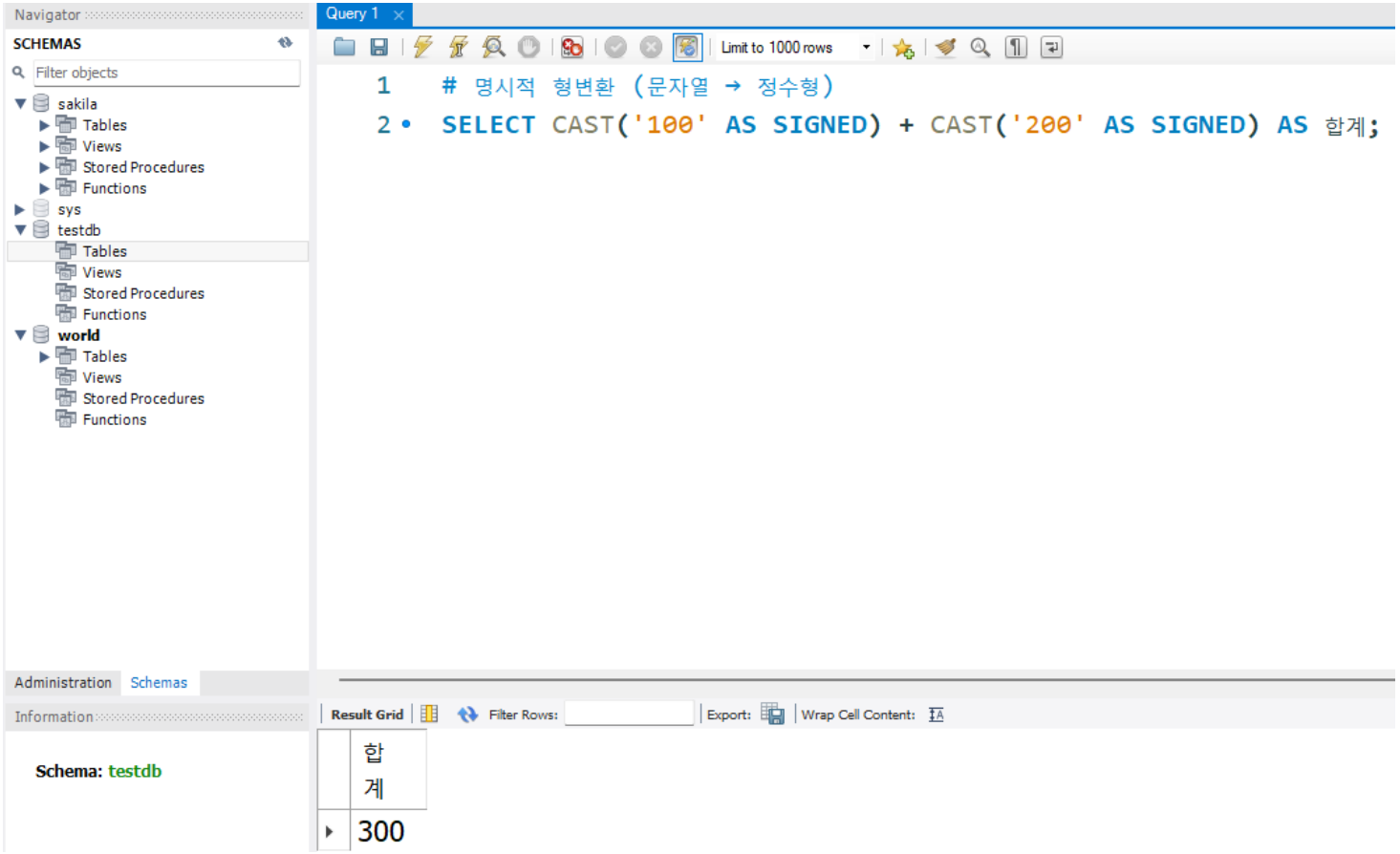
```
# 명시적 형변환 (문자열 → 정수형)
SELECT CAST('100' AS SIGNED) + CAST('200' AS SIGNED) AS 합계;

#  명시적 형변환 (실수 → 정수형)
SELECT CAST(123.456 AS SIGNED) AS 정수변환;

# 묵시적 형변환 (문자열이 자동으로 숫자로 계산됨)
SELECT '100' + '200' AS 묵시적합계;
```

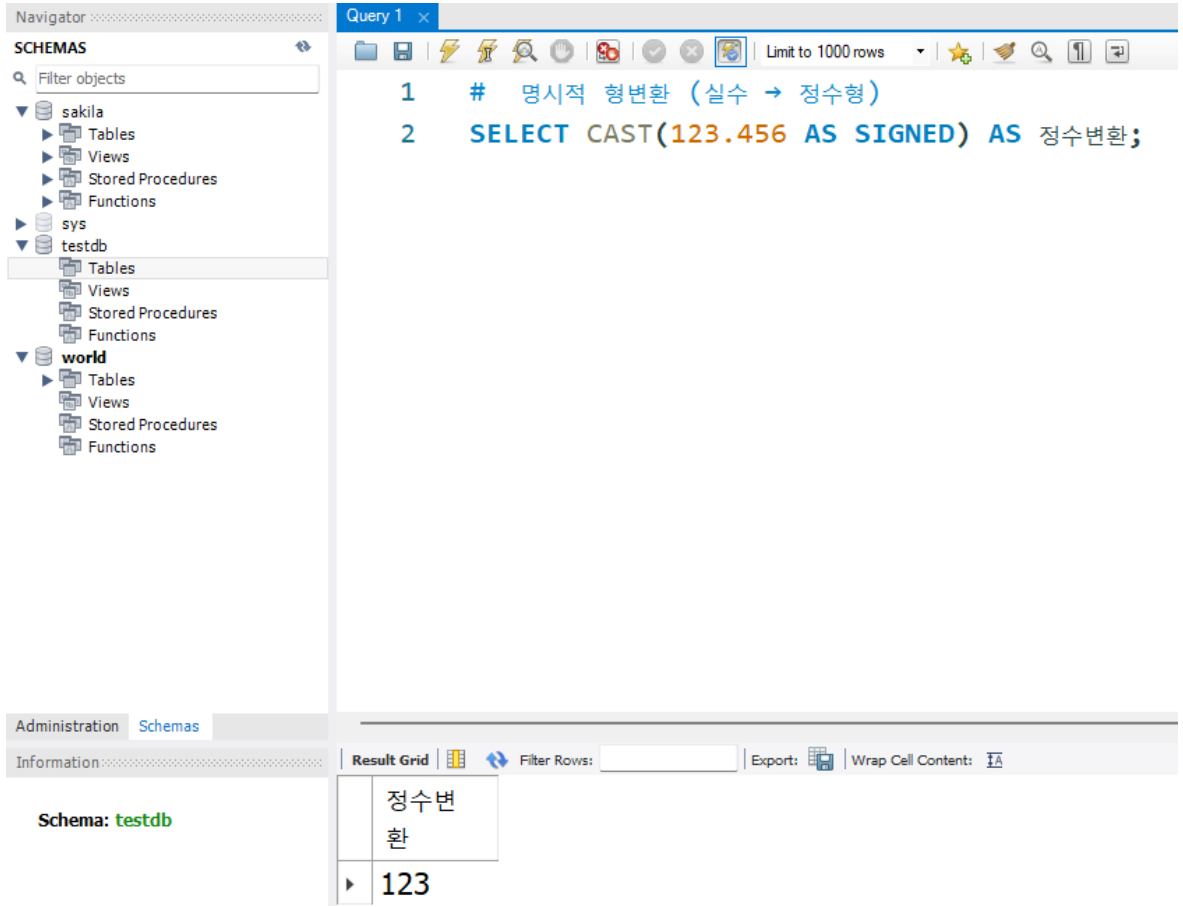
# 명시적 형변환과 묵시적 형변환 실습

- 명시적 형변환 (문자열 → 정수형)



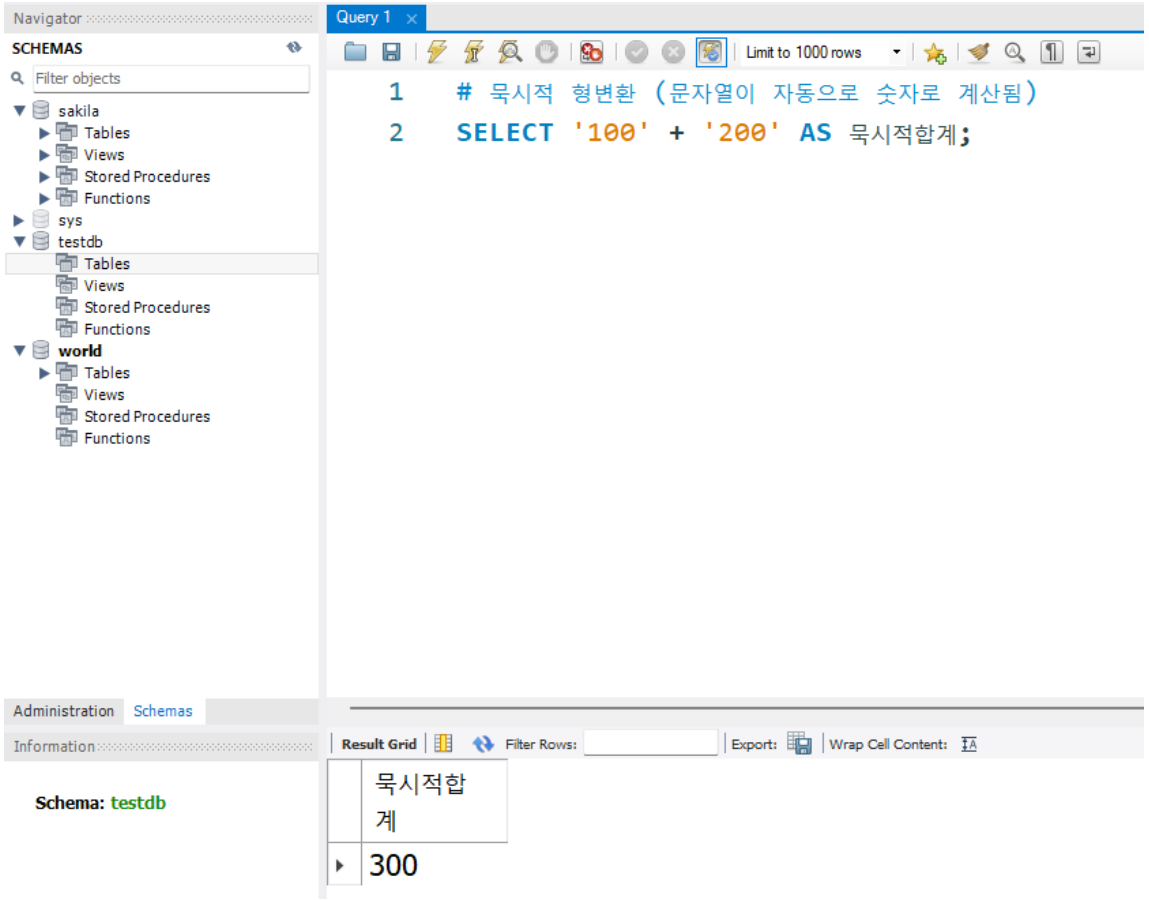
# 명시적 형변환과 묵시적 형변환 실습

- 명시적 형변환 (실수 → 정수형)



# 명시적 형변환과 묵시적 형변환 실습

- 묵시적 형변환 (문자열이 자동으로 숫자로 계산됨)



# 주요 내장 함수 ① : 문자열 처리 함수

| 함수명 | 기능 및 설명 |
|---|---|
| **CONCAT(str1, ...)** | 여러 문자열을 하나로 연결 |
| **SUBSTRING(s,p,l)** | 문자열 s의 p번째 위치부터 l개만큼 추출 |
| **LENGTH(str)** | 문자열의 바이트 수 반환 (CHAR_LENGTH는 글자 수 반환) |
| **LOWER() / UPPER()** | 소문자 / 대문자로 변환 |
| **REPLACE(s,f,t)** | 문자열 s 내의 f를 t로 치환 |

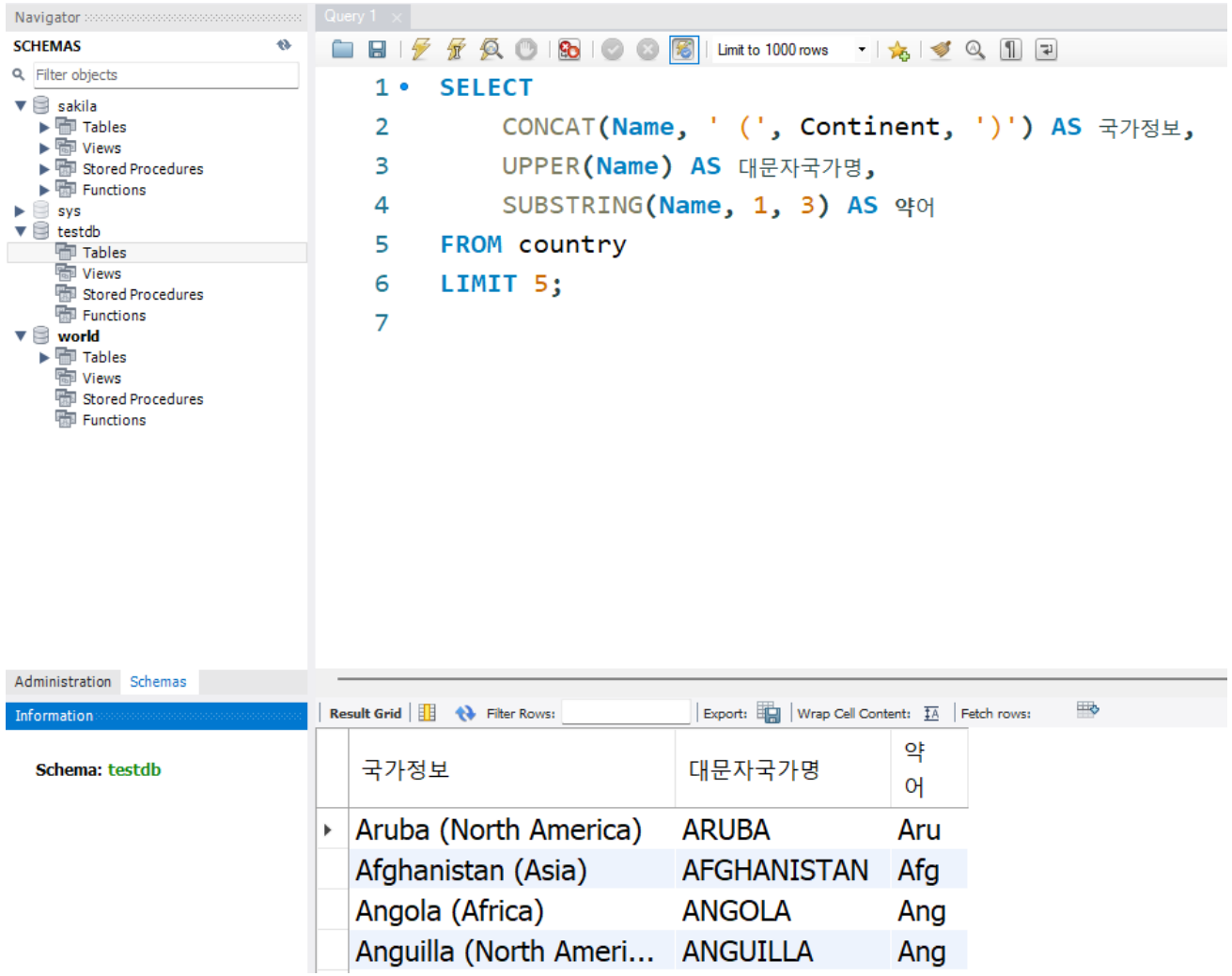
# 주요 내장 함수 ① : 문자열 처리 함수 실습

- 실습 스크립트

```
USE world;


# 국가 이름과 대륙을 하나로 결합하고 국가명을 대문자로 변환 및 약어로 표시하기
SELECT
    CONCAT(Name, ' (', Continent, ')') AS 국가정보,
    UPPER(Name) AS 대문자국가명,
    SUBSTRING(Name, 1, 3) AS 약어
FROM country
LIMIT 5;
```

# 주요 내장 함수 ① : 문자열 처리 함수 실습



# 주요 내장 함수 ② : 날짜 및 시간 처리 함수

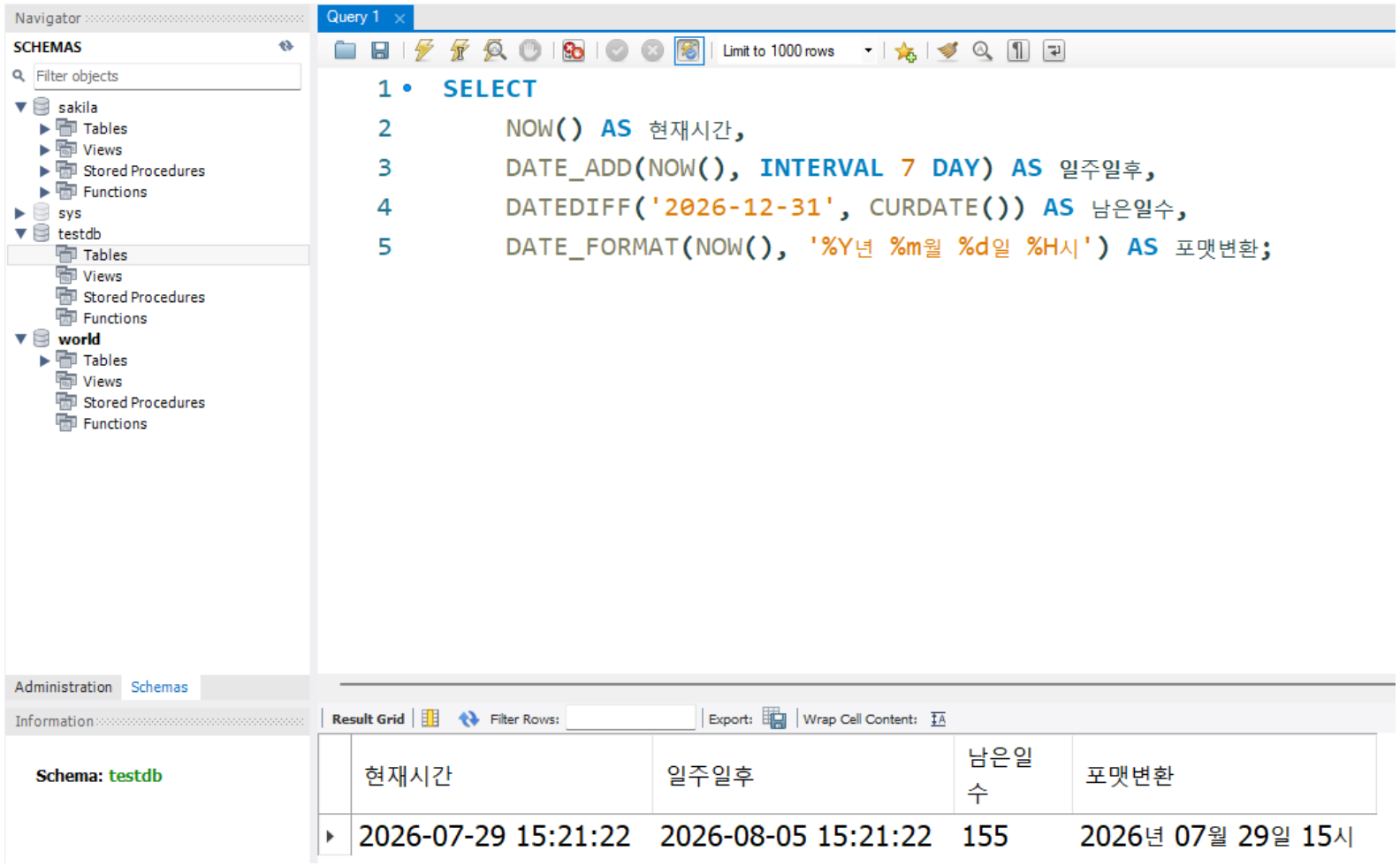
| 함수명 | 기능 및 설명 |
|---|---|
| **NOW() / SYSDATE()** | 현재 날짜와 시간 반환 (YYYY-MM-DD HH:MM:SS) |
| **CURDATE() / CURTIME()** | 현재 날짜만 반환 / 현재 시간만 반환 |
| **DATEDIFF(date1, date2)** | 두 날짜 간의 일수 차이 계산 (date1 - date2) |
| **DATE_ADD(date, INTERVAL)** | 날짜에 특정 기간을 더함 |
| **DATE_FORMAT(date, fmt)** | 날짜를 지정한 포맷의 문자열로 변환 |

# 주요 내장 함수 ② : 날짜 및 시간 처리 함수 실습

- 실습 스크립트

```
SELECT
    NOW() AS 현재시간,
    DATE_ADD(NOW(), INTERVAL 7 DAY) AS 일주일후,
    DATEDIFF('2026-12-31', CURDATE()) AS 남은일수,
    DATE_FORMAT(NOW(), '%Y년 %m월 %d일 %H시') AS 포맷변환;
```

# 주요 내장 함수 ② : 날짜 및 시간 처리 함수 실습



# 주요 내장 함수 ③ : 제어 흐름 및 NULL 처리 함수

- IF(조건식, 참일 때 값, 거짓일 때 값)
  1) 개념: 첫 번째 인자인 조건식을 평가하여 참(TRUE)일 때와 거짓(FALSE 또는 NULL)일 때 각각 다른 값을 반환하는 방식의 함수이다.
  2) 문법: IF(조건식, 참일_때_반환값, 거짓일_때_반환값)
  3) 활용: 합격/불합격, 장수/일반 등 이분법적인 조건 분기를 간단히 표현할 때 적합하다.

- IFNULL(검사할 값, 대체 값)
  1) 개념: 첫 번째 인자로 들어온 값이 NULL인지 검사하여, NULL이 아니면 원래 값을 그대로 반환하고, NULL이면 두 번째 인자로 지정한 대체 값을 반환한다.
  2) 문법: IFNULL(검사할 컬럼 또는 표현식, NULL일 경우 대체할_값)
  3) 활용: 연산 시 NULL로 인한 계산 오류(예: 숫자 + NULL = NULL)를 방지할 때 필수적으로 사용된다.

# 주요 내장 함수 ③ : 제어 흐름 및 NULL 처리 함수

- CASE 구문 (다중 조건문)

1) 개념: IF 함수와 달리 3개 이상의 복잡한 다중 조건을 처리할 때 사용하는 조건문이다.

2) 문법:

```
CASE
    WHEN 조건식1 THEN 결과1
    WHEN 조건식2 THEN 결과2
    ELSE 모든 조건에 해당하지 않을 때의 기본값
END
```

3) 핵심 특징 및 주의사항

① WHEN 조건은 위에서부터 순차적으로 평가되며, 가장 먼저 조건이 참(TRUE)이 되는 순간 해당 THEN 결과값을 반환하고 종료된다.

② ELSE 절을 생략했는데 어떤 조건도 만족하지 않으면 NULL이 반환된다. (예외 방지를 위해 ELSE 절을 명시할 것)

③ 구문 끝에는 반드시 END 키워드를 작성해야 한다.

# 주요 내장 함수 ③ : 제어 흐름 및 NULL 처리 함수 실습

- 실습 스크립트

```
USE world;


# IF 및 IFNULL 활용
SELECT
    Name,
    LifeExpectancy,
    IF(LifeExpectancy >= 75, '장수국가', '일반국가') AS 국가분류,
    IFNULL(GNPOld, 0) AS 이전GNP
FROM country LIMIT 5;
```

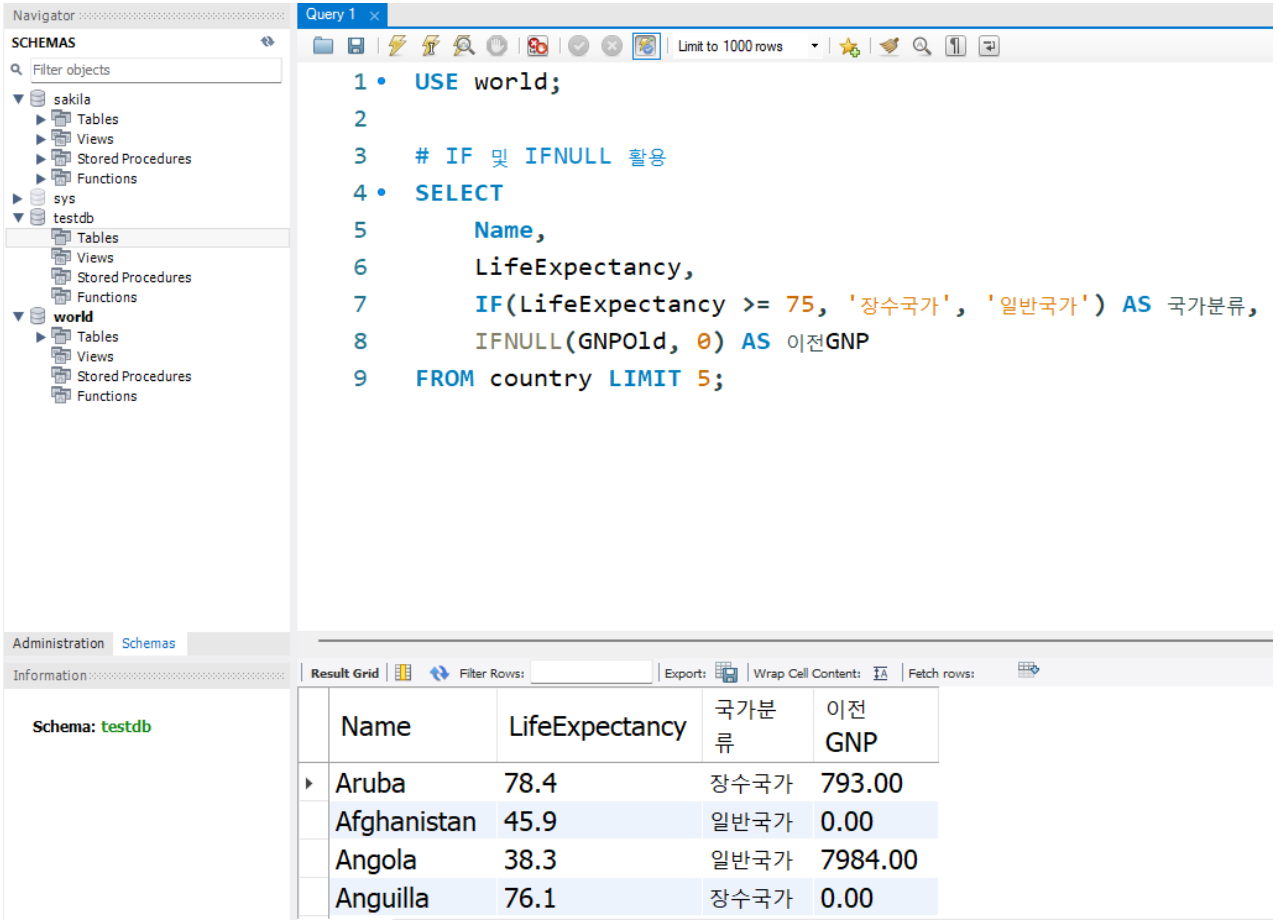
# 주요 내장 함수 ③ : 제어 흐름 및 NULL 처리 함수 실습

- 실습 스크립트

```
# CASE WHEN 활용
SELECT Name, Population,
    CASE
        WHEN Population >= 100000000 THEN '초대형 국가'
        WHEN Population >= 30000000 THEN '중대형 국가'
        ELSE '소형 국가'
    END AS 인구규모분류
FROM country LIMIT 5;
```

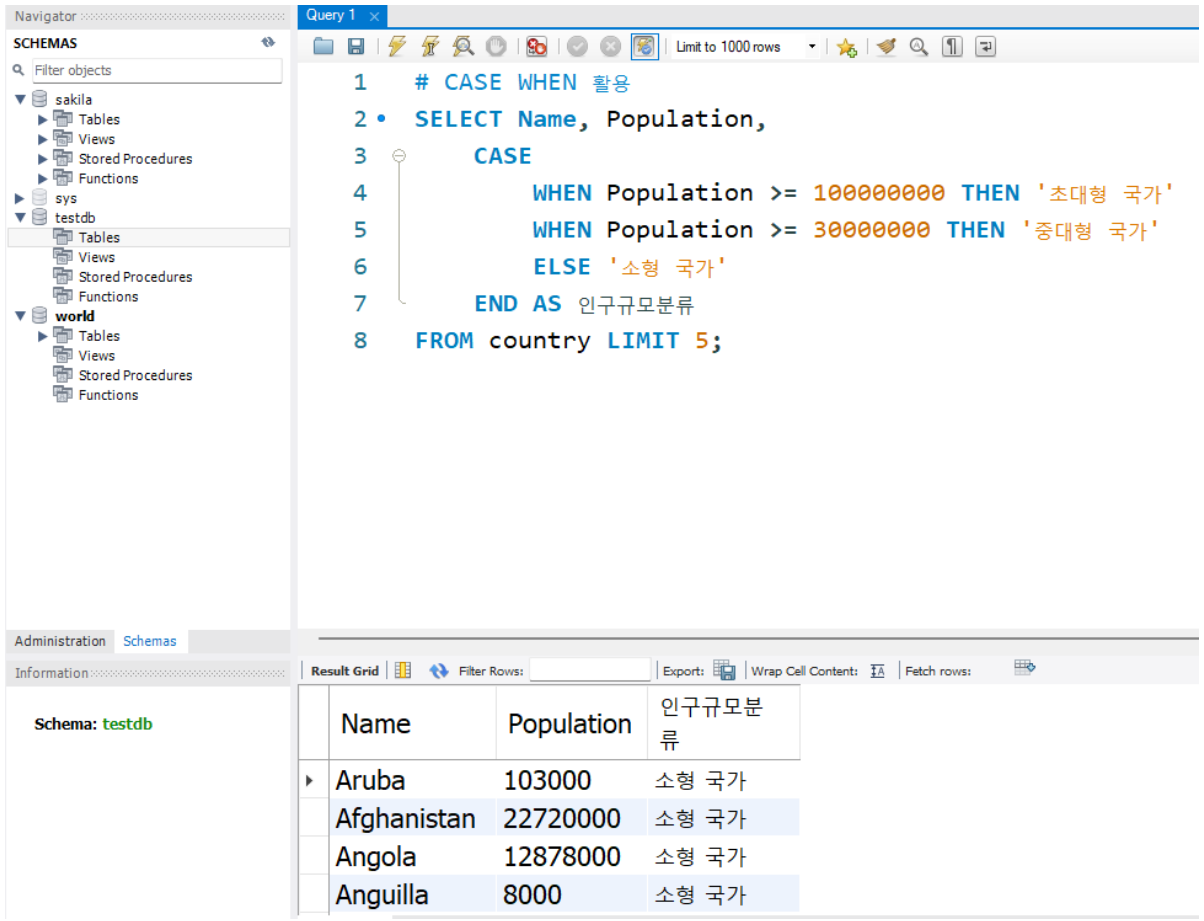
# 주요 내장 함수 ③ : 제어 흐름 및 NULL 처리 함수 실습

- IF 및 IFNULL 활용



# 주요 내장 함수 ③ : 제어 흐름 및 NULL 처리 함수 실습

- CASE WHEN 활용



# 내장 함수를 활용한 리포트 생성 실습

- 실습 문제

1) `sakila.film` 테이블에서 영화 제목(title), 대여 기간(rental_duration), 대여료(rental_rate)를 조회하되,

2) 영화 제목은 대문자로 출력할 것

3) 대여료가 3.0 이상이면 '프리미엄', 아니면 '일반'으로 분류(IF 함수)할 것

4) 상위 10개 데이터만 조회할 것

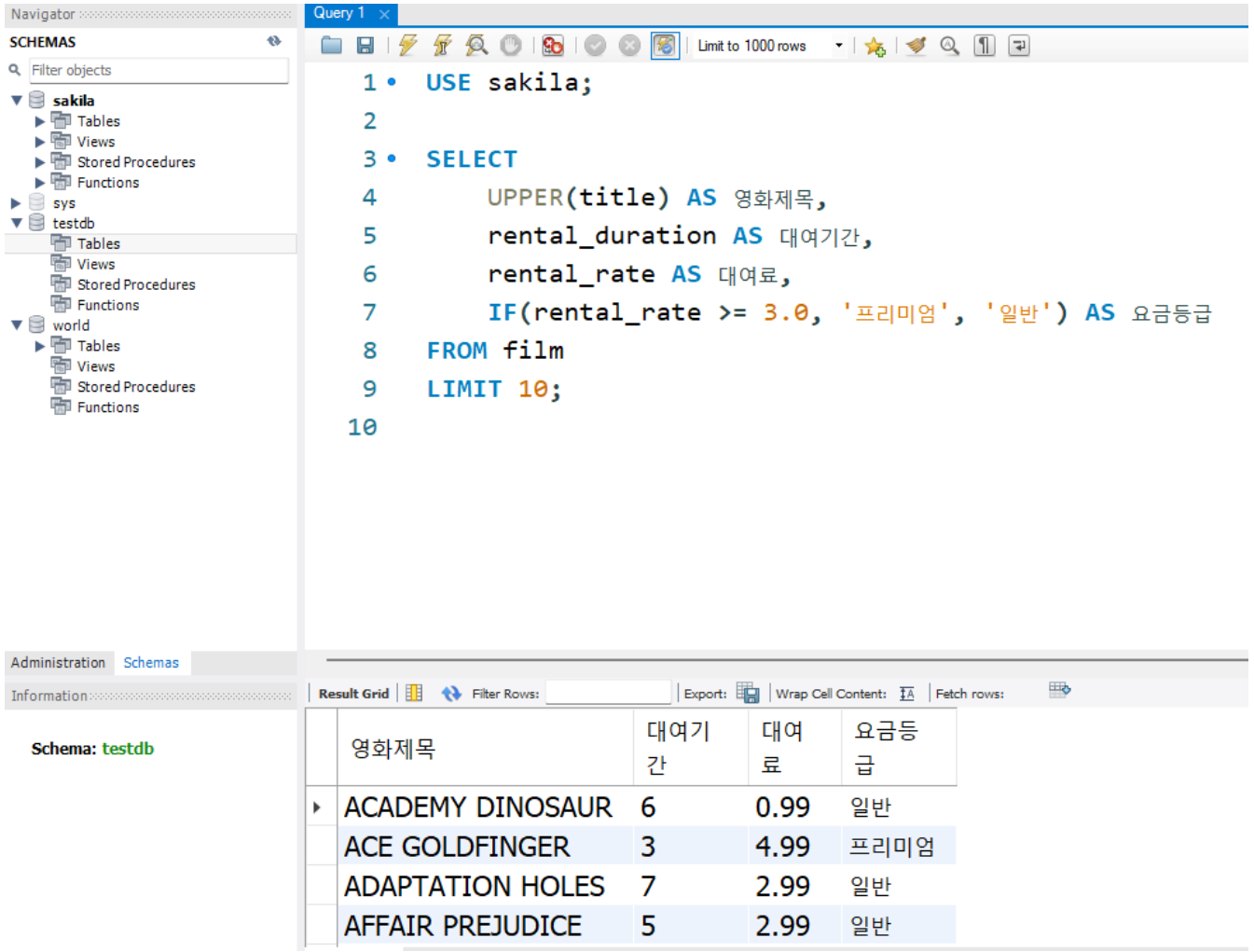
# 내장 함수를 활용한 리포트 생성 실습

- 실습 정답 스크립트

```
USE sakila;


SELECT
    UPPER(title) AS 영화제목,
    rental_duration AS 대여기간,
    rental_rate AS 대여료,
    IF(rental_rate >= 3.0, '프리미엄', '일반') AS 요금등급
FROM film
LIMIT 10;
```

# 내장 함수를 활용한 리포트 생성 실습



# 조인(JOIN)의 개념 및 필요성

- 조인(JOIN)이란?

  : 데이터 중복 방지를 위해 여러 테이블로 나뉘어 저장된 데이터를 공통 컬럼(Primary Key - Foreign Key)을 매개체로 하나로 합쳐서 조회하는 기술

- 조인의 종류

  1) INNER JOIN (내부 조인): 두 테이블 모두에 존재하는 교집합 데이터만 결합
  2) LEFT/RIGHT OUTER JOIN (외부 조인): 한쪽 테이블 기준 전체 데이터 + 매칭 데이터 결합
  3) CROSS JOIN (상호 조인): 모든 경우의 수 조합 (카테시안 곱)
  4) SELF JOIN (자체 조인): 자기 자신 테이블과의 조인 (조직도 등)

# 내부 조인 (INNER JOIN)

- 개념: 두 테이블 모두에 존재하는 교집합 데이터만 결과에 포함됨
- 기본 구문

```
SELECT A.컬럼1, B.컬럼2
FROM 테이블A AS A
INNER JOIN 테이블B AS B ON A.조인키 = B.조인키;
```

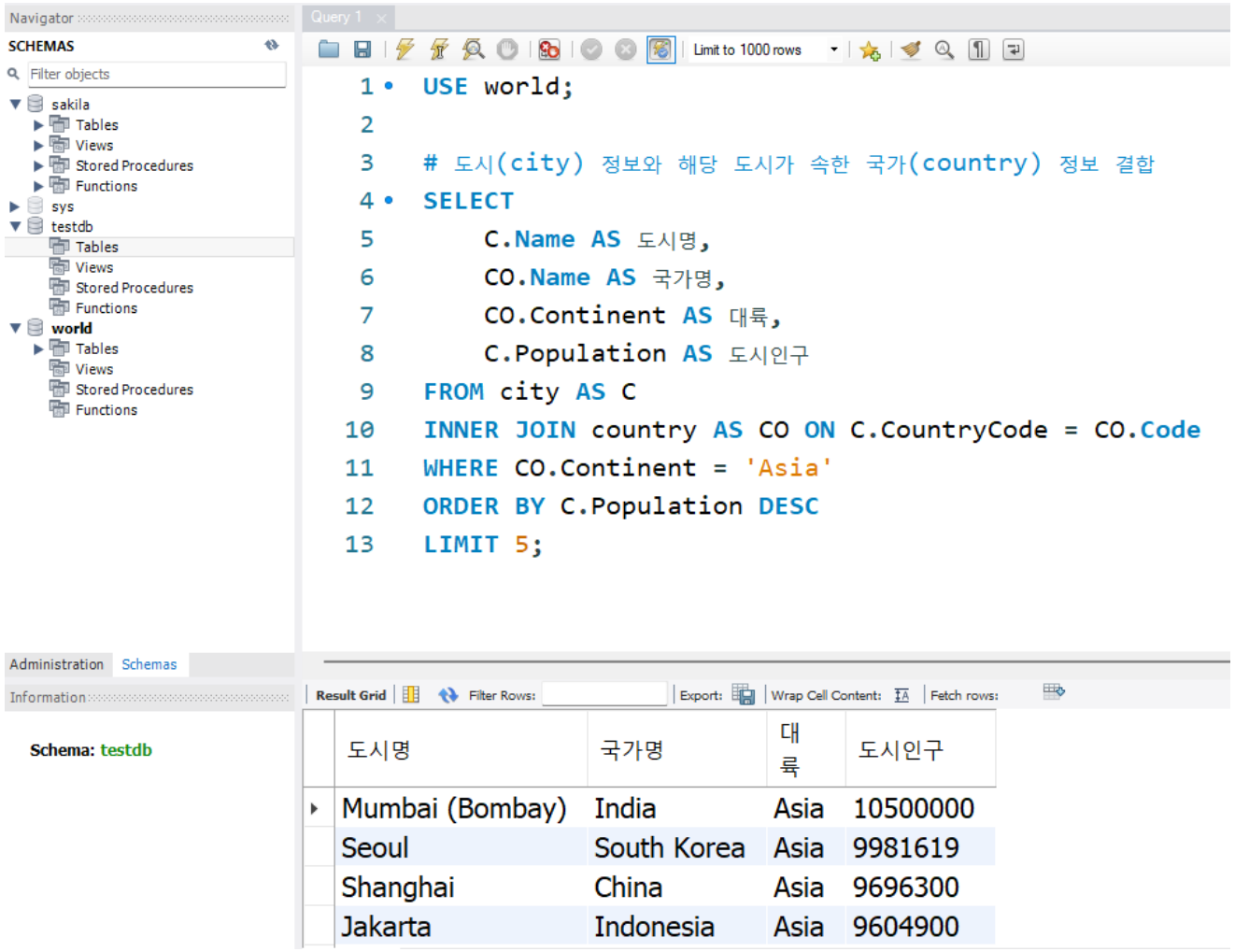
# 내부 조인 (INNER JOIN) 실습

- 실습 스크립트

```
USE world;


# 도시(city) 정보와 해당 도시가 속한 국가(country) 정보 결합
SELECT
    C.Name AS 도시명,
    CO.Name AS 국가명,
    CO.Continent AS 대륙,
    C.Population AS 도시인구
FROM city AS C
INNER JOIN country AS CO ON C.CountryCode = CO.Code
WHERE CO.Continent = 'Asia'
ORDER BY C.Population DESC
LIMIT 5;
```

# 내부 조인 (INNER JOIN) 실습



# 외부 조인 (LEFT / RIGHT OUTER JOIN)

- 개념

1) LEFT OUTER JOIN: 왼쪽(기준) 테이블의 모든 행을 출력하고, 오른쪽 테이블에 조건이 매칭되는 데이터가 없으면 NULL로 채워서 표시한다.

2) RIGHT OUTER JOIN: 오른쪽(기준) 테이블의 모든 행을 출력하고, 왼쪽 테이블에 조건이 매칭되는 데이터가 없으면 NULL로 채워서 표시한다.

- 기본 문법 (Syntax)

1) LEFT OUTER JOIN(LEFT JOIN)

```
SELECT A.컬럼1, B.컬럼2
FROM 테이블A(기준 테이블) AS A
LEFT OUTER JOIN 테이블B(상대 테이블) AS B ON A.조인키 = B.조인키;
```

2) RIGHT OUTER JOIN(RIGHT JOIN)

```
SELECT A.컬럼1, B.컬럼2
FROM 테이블A(상대 테이블) AS A
RIGHT OUTER JOIN 테이블B(기준 테이블) AS B ON A.조인키 = B.조인키;
```

# 외부 조인 (LEFT / RIGHT OUTER JOIN) 실습

## - 실습 스크립트

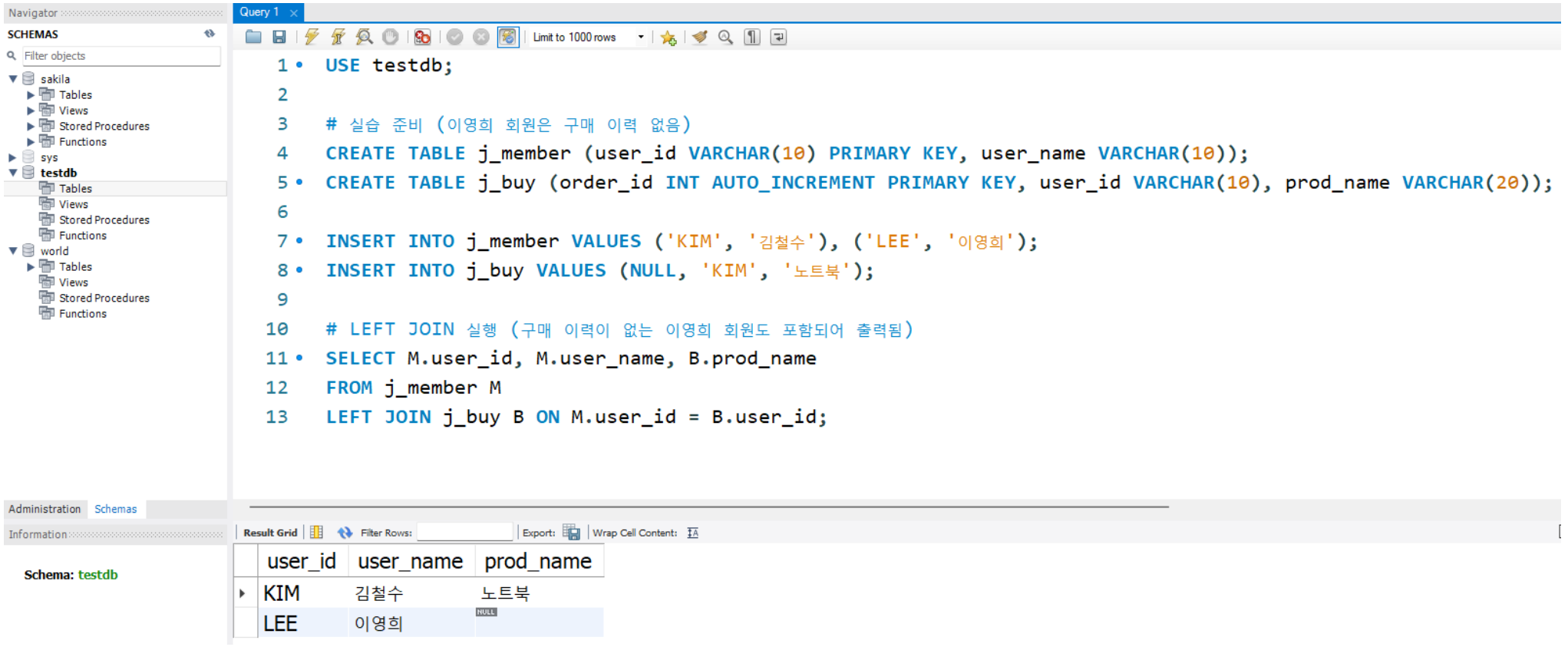
```
USE testdb;

# 실습 준비 (이영희 회원은 구매 이력 없음)
CREATE TABLE j_member (user_id VARCHAR(10) PRIMARY KEY, user_name VARCHAR(10));
CREATE TABLE j_buy (order_id INT AUTO_INCREMENT PRIMARY KEY, user_id VARCHAR(10), prod_name
VARCHAR(20));

INSERT INTO j_member VALUES ('KIM', '김철수'), ('LEE', '이영희');
INSERT INTO j_buy VALUES (NULL, 'KIM', '노트북');

# LEFT JOIN 실행 (구매 이력이 없는 이영희 회원도 포함되어 출력됨)
SELECT M.user_id, M.user_name, B.prod_name
FROM j_member M
LEFT JOIN j_buy B ON M.user_id = B.user_id;
```

# 외부 조인 (LEFT / RIGHT OUTER JOIN) 실습



# 상호 조인 (CROSS JOIN) 및 자체 조인 (SELF JOIN)

- 상호 조인 (CROSS JOIN)

1) 개념: 조인 조건절(ON) 없이 A 테이블의 모든 행과 B 테이블의 모든 행을 1:1로 무조건 조합하는 조인 기법이다.

2) 기본 문법 (Syntax):

```
SELECT A.컬럼1, B.컬럼2
FROM 테이블A AS A
CROSS JOIN 테이블B AS B;
```

3) 특징 및 용도

① 최종 출력 행 수: A 테이블 행 수 * B 테이블 행 수

② 주로 시스템 성능 테스트나 대량의 더미(Dummy) 데이터를 생성할 때 활용된다.

# 상호 조인 (CROSS JOIN) 및 자체 조인 (SELF JOIN)

- 자체 조인 (SELF JOIN)

1) 개념: 하나의 테이블 내에서 계층 구조나 연관 관계(예: 사원ID - 상사ID, 카테고리ID – 상위카테고리ID)를 조회하기 위해 자기 자신과 조인하는 기법이다.

2) 기본 문법 (Syntax):

```
SELECT E.emp_name AS 사원명, M.emp_name AS 상사명
FROM 테이블명([사원 역할] 테이블) AS E
INNER JOIN 테이블명([상사 역할] 동일 테이블) AS M
ON E.manager_id = M.emp_id;   # 조인 조건 (사원의 상사ID = 상사의 사원ID)
```

3) 핵심 주의사항

: 하나의 테이블을 두 번 참조하므로, 각 역할을 구분할 수 있도록 반드시 서로 다른 테이블 별칭(Alias, 예시: E, M)을 지정해야 한다.

# 상호 조인 (CROSS JOIN) 및 자체 조인 (SELF JOIN) 실습

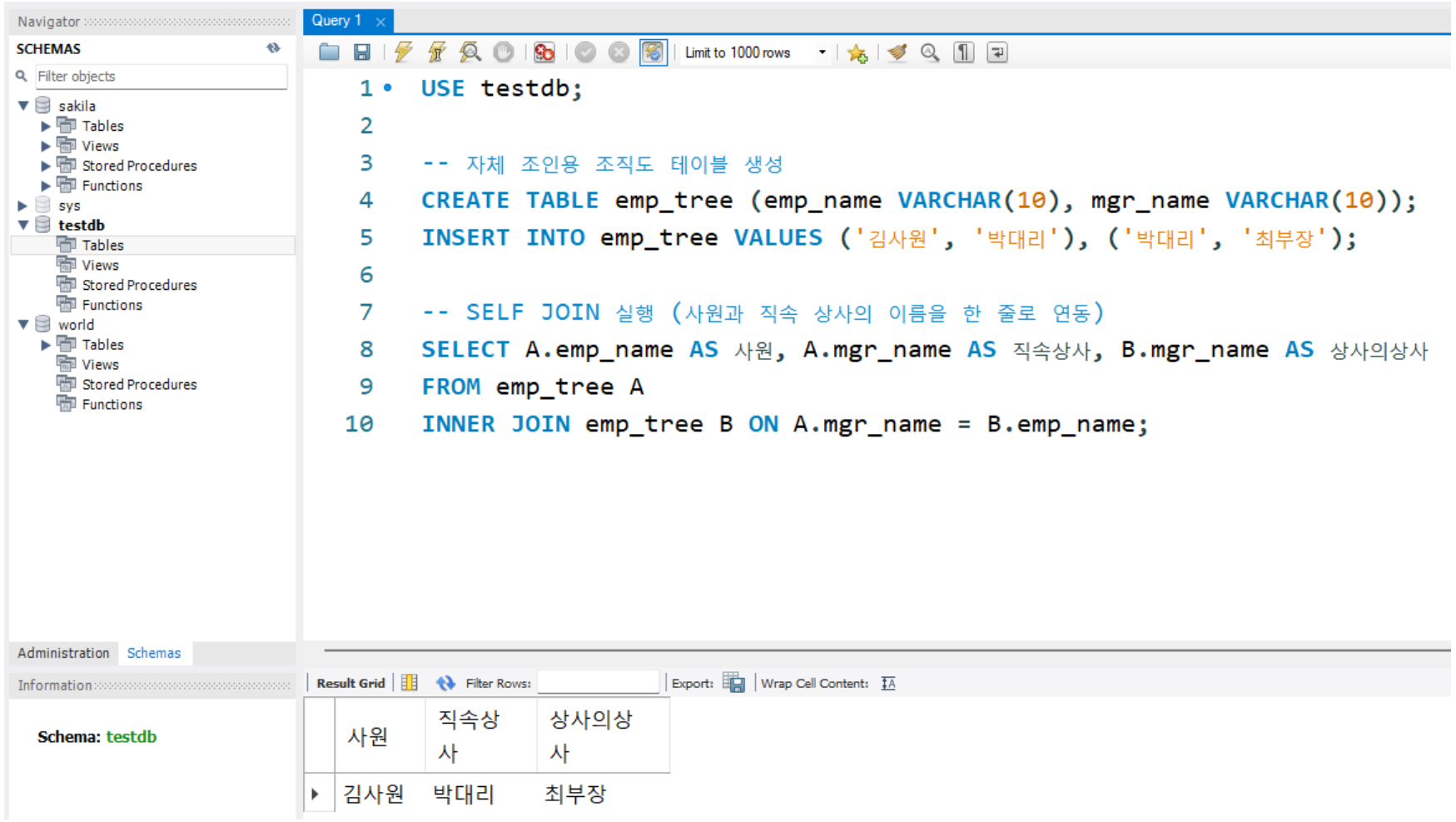
- 실습 스크립트

```
USE testdb;

# 자체 조인용 조직도 테이블 생성
CREATE TABLE emp_tree (emp_name VARCHAR(10), mgr_name VARCHAR(10));
INSERT INTO emp_tree VALUES ('김사원', '박대리'), ('박대리', '최부장');

# SELF JOIN 실행 (사원과 직속 상사의 이름을 한 줄로 연동)
SELECT A.emp_name AS 사원, A.mgr_name AS 직속상사, B.mgr_name AS 상사의상사
FROM emp_tree A
INNER JOIN emp_tree B ON A.mgr_name = B.emp_name;
```

# 상호 조인 (CROSS JOIN) 및 자체 조인 (SELF JOIN) 실습



# [종합 실습] SQL 기본 및 조인 다지기

- 과제: world 데이터베이스를 활용하여 다음 결과를 출력하는 단일 SQL 문을 작성하시오.
  1. 대륙(Continent)별로 도시 인구(city.Population)의 합계를 구하시오.
  2. 도시 인구 합계가 1억 명 이상인 대륙만 필터링하시오.
  3. 결과를 도시 인구 합계 내림차순으로 정렬하시오.

# [종합 실습] SQL 기본 및 조인 다지기

- 종합 실습 정답 스크립트

```
USE world;


SELECT
    CO.Continent AS 대륙,
    SUM(C.Population) AS 총도시인구합계
FROM country CO
INNER JOIN city C ON CO.Code = C.CountryCode
GROUP BY CO.Continent
HAVING 총도시인구합계 >= 100000000
ORDER BY 총도시인구합계 DESC;
```

# [종합 실습] SQL 기본 및 조인 다지기

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| 대륙 | 총도시인구합계 |
|---|---|
| Asia | 697604103 |
| Europe | 241942813 |
| South America | 172037859 |
| North America | 168250381 |
| Africa | 135838579 |

# 인덱스(Index)의 개념과 필요성

- 인덱스(Index) 개념
  1) 데이터를 쉽고 빠르게 찾기위해 특정 기준(예: ID 순, 알파벳 순)으로 정렬한 뒤, 일정 범위의 값으로 묶어 그룹(1~1000번은 A그룹, 1001~2000번은 B그룹, ...)으로 설정한 피라미드(Tree) 구조
  2) 인덱스가 지정된 컬럼을 검색하면 몇 번의 대소 비교만으로 원하는 번호의 행을 즉시 찾을 수 있음
- Full Table Scan vs Index Scan
  1) Full Table Scan (전체 테이블 스캔)
     ① 인덱스가 없을 때 발생
     ② 첫 번째 행부터 마지막 행까지 일일이 다 읽어서 조건에 맞는지 비교 (데이터가 많을수록 속도가 극도로 느려짐)
  2) Index Scan (인덱스 스캔)
     ① 인덱스가 설정되어 있을 때 발생
     ② 정렬되어 있는 인덱스 구조(Tree) 구조를 타고 내려가 해당 데이터만 콕 집어서 추출 (검색 속도 극대화)

# 뷰(View)의 개념과 동작 원리

- 뷰(View)의 정의

  : 자주 사용하는 조회 결과를 위해 별도의 테이블을 새로 만들어 디스크에 저장하지 않고, SELECT 쿼리문 텍스트 자체만 DB에 파이썬 딕셔너리(키-값) 형태로 보관한 뒤, 사용자가 호출할 때 일반 테이블과 동일하게 조회 기능을 제공하는 가상 테이블

- 뷰의 내부 동작 과정

  1) 뷰 생성: 개발자가 **CREATE VIEW 뷰이름 AS SELECT ...** 구문 실행

  2) 정의 저장: DBMS가 데이터(테이블)를 저장하지 않고, 해당 SELECT 구문 자체만 저장

  3) 뷰 호출: 사용자가 **SELECT * FROM 뷰이름**; 으로 실행

  4) 쿼리 실행: DBMS가 DB에 저장되어 있던 SELECT 구문을 꺼내와 원본 테이블(Base Table)을 대신 조회하여 최신 결과를 반환

# 뷰(View)의 개념과 동작 원리

- 뷰의 핵심 기술적 특징

1) 디스크 공간 절약: 조회 결과 데이터를 저장하지 않고 쿼리 정의만 보관하므로 디스크 용량을 추가로 차지하지 않음

2) 실시간 동기화: 원본 테이블의 데이터가 변경되면, 뷰를 통해 조회하는 결과에도 최신 데이터가 즉시 반영됨

# 뷰(View)의 활용 목적: 보안(접근 통제)과 코드의 재사용성

- 뷰를 통한 강제적 접근 통제
  1) 1단계 (원본 테이블 차단): 민감한 정보가 들어있는 원본 테이블에 대한 SELECT 권한을 사용자 계정에서 차단
  2) 2단계 (정제된 뷰 생성): 주민등록번호, 급여, 비밀번호 등 민감 컬럼을 제외하고 안전한 컬럼만 추출하여 뷰(가상 테이블) 생성
  3) 3단계 (뷰 접근 권한 부여): 사용자 계정에 오직 해당 뷰에 대해서만 SELECT 접근 권한을 부여
- 강제적 접근 통제가 가져오는 2가지 관점의 효과
  1) 관점 1 (사용자/개발자 측면 - Human Error 및 악의적 유출 차단)
     : 개발자의 실수나 악의적인 쿼리 작성 시도에도 원본 테이블 접근 권한 자체가 없으므로(Access Denied), 민감한 데이터의 유출이 구조적으로 원천 차단됨
  2) 관점 2 (시스템/보안 관리 측면 - 보안과 업무 편의성의 동시 확보)
     : 원본 데이터의 보안은 완벽하게 유지하면서도, 일반 사용자는 기존 SQL 조작 방식 그대로 허용된 데이터에 안전하게 접근하여 업무를 수행할 수 있음

# 뷰(View)의 활용 목적: 보안(접근 통제)과 코드의 재사용성

- 코드의 재사용성

: 다중 테이블 JOIN, WHERE 조건절, GROUP BY 등이 포함된 복잡한 쿼리를 매번 중복 작성할 필요 없이, 뷰로 한 번 등록해 두고 프로그램이나 쿼리에서 자유롭게 재사용 가능

# 뷰 생성(`CREATE VIEW`), 조회 및 삭제(`DROP VIEW`)

- 뷰 생성 기본 문법

```
CREATE VIEW [뷰이름] AS
SELECT [컬럼1, 컬럼2 ...]
FROM [테이블명]
WHERE [조건];
```

- 뷰 삭제 기본 문법

```
DROP VIEW [뷰이름];
```

# 뷰(VIEW) 실습

- 실습 스크립트

```
USE testdb;

# 실습용 원본 테이블 준비
CREATE TABLE real_user (
    user_id VARCHAR(10) PRIMARY KEY,
    user_name VARCHAR(20) NOT NULL,
    user_ssn VARCHAR(14),        # 주민번호 (민감정보)
    salary INT                   # 급여 (민감정보)
);

INSERT INTO real_user VALUES
('KIM', '김철수', '950101-1234567', 4000000),
('LEE', '이영희', '980202-2345678', 3500000);
```

# 뷰(VIEW) 실습

- 실습 스크립트

```
USE testdb;

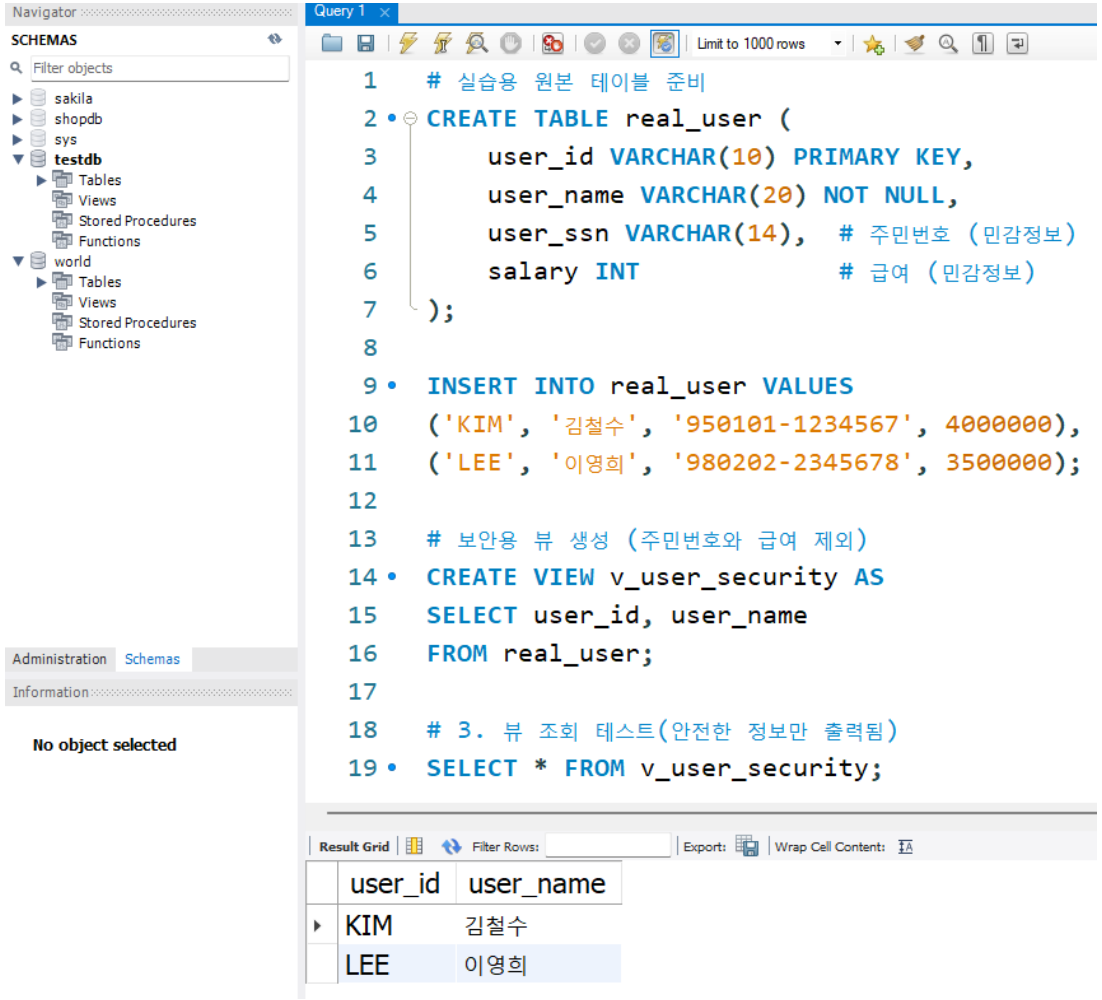
# 보안용 뷰 생성 (주민번호와 급여 제외)
CREATE VIEW v_user_security AS
SELECT user_id, user_name
FROM real_user;

# 3. 뷰 조회 테스트(안전한 정보만 출력됨)
SELECT * FROM v_user_security;

# 뷰 삭제 (뷰를 지워도 원본 real_user 데이터는 안전하게 유지됨)
DROP VIEW v_user_security;
SELECT * FROM real_user;
```

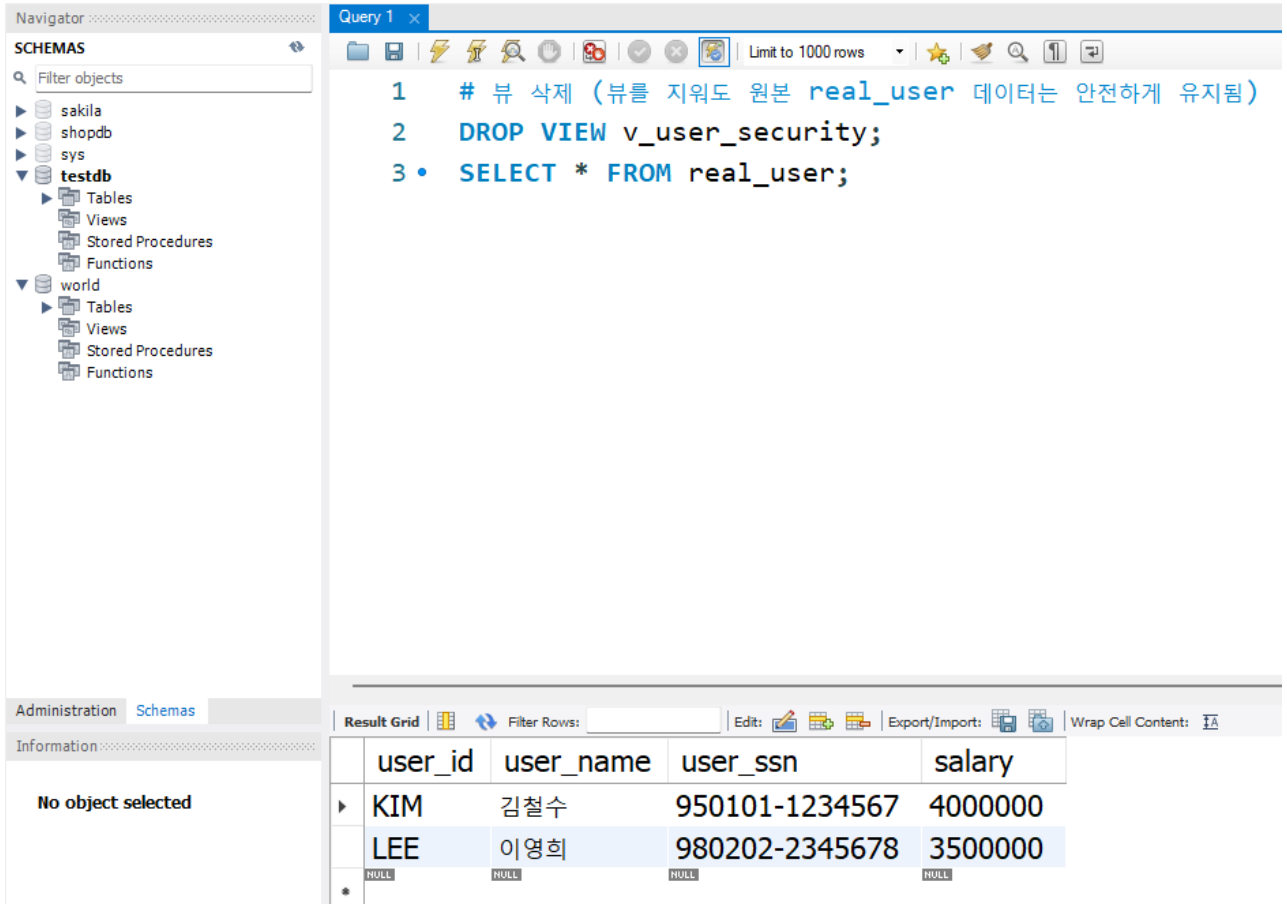
# 뷰(VIEW) 실습

- 보안용 뷰 생성, 뷰 조회 테스트(안전한 정보만 출력됨)



# 뷰(VIEW) 실습

- 뷰 삭제 (뷰를 지워도 원본 real_user 데이터는 안전하게 유지됨)



# 복잡한 조인(JOIN) 쿼리를 재사용하는 뷰 작성 실습

- 실습 스크립트

```
USE world;


# 도시(city)와 국가(country) 테이블을 조인하여 대륙이 'Asia'인 데이터만 추출하는 뷰 생성
CREATE VIEW v_asia_city_info AS
SELECT
    C.Name AS city_name,
    CO.Name AS country_name,
    C.Population AS city_population
FROM city AS C
INNER JOIN country AS CO ON C.CountryCode = CO.Code
WHERE CO.Continent = 'Asia';
```

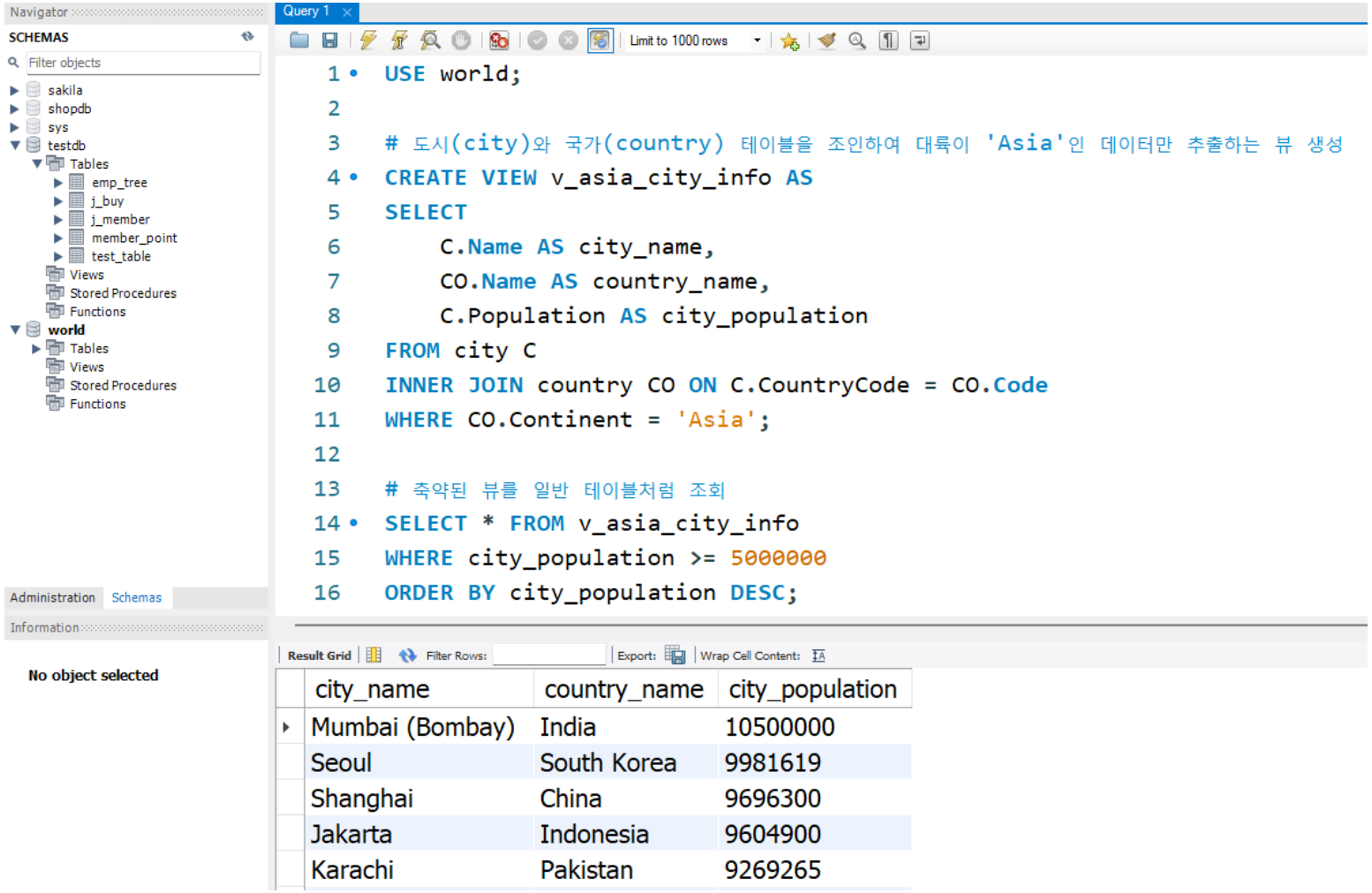
# 복잡한 조인(JOIN) 쿼리를 재사용하는 뷰 작성 실습

- 실습 스크립트

```
# 축약된 뷰를 일반 테이블처럼 조회
SELECT * FROM v_asia_city_info
WHERE city_population >= 5000000
ORDER BY city_population DESC;

# 결과: 길고 복잡한 INNER JOIN 문을 매번 작성할 필요 없이 단 한 줄의 SELECT문으로 정제된 결과를
   즉시 획득
```

# 복잡한 조인(JOIN) 쿼리를 재사용하는 뷰 작성 실습



# 데이터베이스 백업(Backup)과 복원(Restore)

- 백업과 복원의 필요성

  : 시스템 장애, 악성코드 감염, 개발자의 실수로 인한 데이터 삭제 시 원상 복구를 위해 필수적

- 백업의 종류

  1) 논리적 백업 (SQL Dump): 데이터베이스의 구조(DDL)와 데이터(DML)를 하나 이상의 SQL 텍스트 파일(`.sql`) 형태로 저장하는 방식 (본 실습 방식)

  2) 물리적 백업 (Physical Backup): 데이터베이스가 실제 저장되는 디스크 파일(Data File, Log File) 자체를 그대로 복사하는 방식

- MySQL Workbench 백업/복원 도구

  : Workbench 상단 메뉴 Server → Data Export(백업) / Data Import (복원) 지원

# 데이터베이스 백업(Backup)과 복원(Restore) 실습

- 데이터 백업 (Data Export) 단계
  1) Workbench 상단 메뉴에서 Server → Data Export 선택
  2) 좌측 Tables to Export 패널에서 백업 대상 데이터베이스(예: testdb) 체크
  3) Export Options 설정:
     ① Export to Self-Contained File 선택
     ② 저장될 파일 경로 지정 (예: `C:₩Users₩User₩publictestdb_dump.sql`)
  4) 우측 하단 Start Export 버튼 클릭 → Export Completed 확인
- 데이터 복원 (Data Import) 단계
  1) 기존 testdb를 지운 후 복원 테스트를 위해 `DROP DATABASE testdb;` 실행
  2) Workbench 상단 메뉴에서 Server → Data Import 선택
  3) Import from Self-Contained File 선택 → 저장했던 `testdb_dump.sql` 파일 지정
  4) 우측 하단 Start Import 버튼 클릭

# 복원 에러(ERROR 1046) 분석 및 해결

- 대표 복원 에러 현상

  ERROR 1046 (3D000) at line 22: No database selected

  Operation failed with exitcode 1

- 원인 정확히 파악하기

  1)·백업 파일(`testdb_dump.sql`) 내에 `CREATE DATABASE` 또는 `USE` 구문이 생략되어 있을 때 발생함

  2) 복원 시 MySQL 서버가 "이 테이블과 데이터들을 어느 데이터베이스에 집어넣어야 하는가?"를 알지 못해 작업을 중단하는 것임

# 복원 에러(ERROR 1046) 분석 및 해결

- 가장 확실한 대안 2가지 (선택 적용)

1) 대안 ① (GUI 방식 - 가장 추천)

① `Data Import` 화면 중간의 `Default Target Schema` 드롭다운 클릭

② 복원될 대상 데이터베이스를 선택 (없다면 `New...` 버튼을 눌러 `testdb` 생성 후 선택)

③ 이 상태에서 `Start Import`를 누르면 에러 없이 복원 완료!

2) 대안 ② (SQL 파일 수정 방식)

① 백업된 `.sql` 파일을 메모장으로 열고 최상단 첫 줄에 아래 코드 추가 작성 후 저장

②`CREATE DATABASE IF NOT EXISTS testdb; USE testdb;`

# 실습 환경 설정

1. 윈도우 시작(또는 찾기) → anaconda 입력 → Anaconda Prompt 실행

2. 가상 환경 설치 : conda create –n db python=3.12

```
(base) C:₩Users₩303megait>conda create -n db python=3.12
Retrieving notices: done
Channels:
 - defaults
Platform: win-64
Collecting package metadata (repodata.json): done
Solving environment: done

## Package Plan ##

  environment location: C:₩Users₩303megait₩anaconda3₩envs₩db

  added / updated specs:
    - python=3.12


The following NEW packages will be INSTALLED:

  bzip2              pkgs/main/win-64::bzip2-1.0.8-h2bbff1b_6
  ca-certificates    pkgs/main/win-64::ca-certificates-2025.2.25-haa95532_0
  expat              pkgs/main/win-64::expat-2.6.4-h8ddb27b_0
  libffi             pkgs/main/win-64::libffi-3.4.4-hd77b12b_1
  openssl            pkgs/main/win-64::openssl-3.0.16-h3f729d1_0
  pip                pkgs/main/win-64::pip-25.0-py312haa95532_0
  python             pkgs/main/win-64::python-3.12.9-h14ffc60_0
  setuptools         pkgs/main/win-64::setuptools-75.8.0-py312haa95532_0
  sqlite             pkgs/main/win-64::sqlite-3.45.3-h2bbff1b_0
  tk                 pkgs/main/win-64::tk-8.6.14-h0416ee5_0
  tzdata             pkgs/main/noarch::tzdata-2025a-h04d1e81_0
  vc                 pkgs/main/win-64::vc-14.42-haa95532_4
  vs2015_runtime     pkgs/main/win-64::vs2015_runtime-14.42.34433-he0abc0d_4
  wheel              pkgs/main/win-64::wheel-0.45.1-py312haa95532_0
  xz                 pkgs/main/win-64::xz-5.6.4-h4754444_1
  zlib               pkgs/main/win-64::zlib-1.2.13-h8cc25b3_1


Proceed ([y]/n)?
```

# 실습 환경 설정

3. 가상 환경 활성화 : conda activate db

4. 필요한 라이브러리 설치 : pip install PyMySQL streamlit

5. miniconda3 설치 폴더 → 새 폴더 생성 : work1

6. Visual Studio Code의 메뉴에서 'File' >> 'Open Folder'를 선택,  work1 폴더 선택

7. CTRL+SHIFT+P를 입력, 'Python: Select Interpreter' 선택, db 인터프리터 선택

# Python과 MySQL DB 연동

- Python에서 DB를 연동하는 이유
  : 데이터베이스에 저장된 데이터를 파이썬으로 가져와 가공・분석하고, 이를 웹 화면에 시각적으로 구현하기 위함

- 필수 라이브러리
  1) PyMySQL : 파이썬과 MySQL 데이터베이스 사이의 통신을 담당하는 라이브러리
  2) 설치(Anaconda Prompt 가상환경 db): pip install PyMySQL

# PyMySQL을 이용한 DB 접속

- PyMySQL 접속 및 데이터 로드 절차

1) conn=pymysql.connect(매개변수)
   ① 파이썬 프로그램과 MySQL 데이터베이스를 안전하게 연결해 주는 전용 통로
   ② 매개변수: host, user, password, db, charset, cursorclass
   ③ cursorclass
      • MySQL DBMS로부터 조회 결과를 받아올 때 결과값의 형식(자료형)을 결정하는 클래스
      • pymysql.cursors.DictCursor: 조회된 전체 행을 list[dict] 형식으로 수신
      • 예시: [{'id': 1, 'name': '홍길동'}, {'id': 2, 'name': '이순신'}]

2) cursor = conn.cursor()
   • 파이썬 코드로 MySQL DBMS에 명령을 내리고 결과를 받아오는 조작 도구

3) cursor.execute(sql): SQL 쿼리 실행

4) cursor.fetchall()
   ① 조회 결과 전체를 한 번에 파이썬으로 가져오는 함수
   ② 조회된 전체 행을 list[dict] 구조로 수신
   ③ 반환 데이터 구조: [{'col1': 값1, 'col2': 값2}, {'col1': 값3, 'col2': 값4}, ...]

# PyMySQL을 이용한 DB 접속 실습

- 실습 스크립트 작성 (pymysql_study.py)

```
(db) C:\Users\gon09\work2>python pymysql_study.py
조회된 데이터 (파이썬 list[dict] 구조):
[{'Name': 'Seoul', 'District': 'Seoul', 'Population': 9981619}, {'Name': 'Pusan', 'District': 'Pusan', 'Population': 3804522},
: 'Taegu', 'Population': 2548568}, {'Name': 'Taejon', 'District': 'Taejon', 'Population': 1425835}]
--------------------------------------------------------------------------------
파이썬 for문 및 dict 키 추출을 이용한 데이터 출력:
- 도시명: Seoul | 지역: Seoul | 인구수: 9,981,619명
- 도시명: Pusan | 지역: Pusan | 인구수: 3,804,522명
- 도시명: Inchon | 지역: Inchon | 인구수: 2,559,424명
- 도시명: Taegu | 지역: Taegu | 인구수: 2,548,568명
- 도시명: Taejon | 지역: Taejon | 인구수: 1,425,835명
```

# PyMySQL + Streamlit 연동 웹 대시보드 생성 실습

- 실습 스크립트 작성 (app.py)

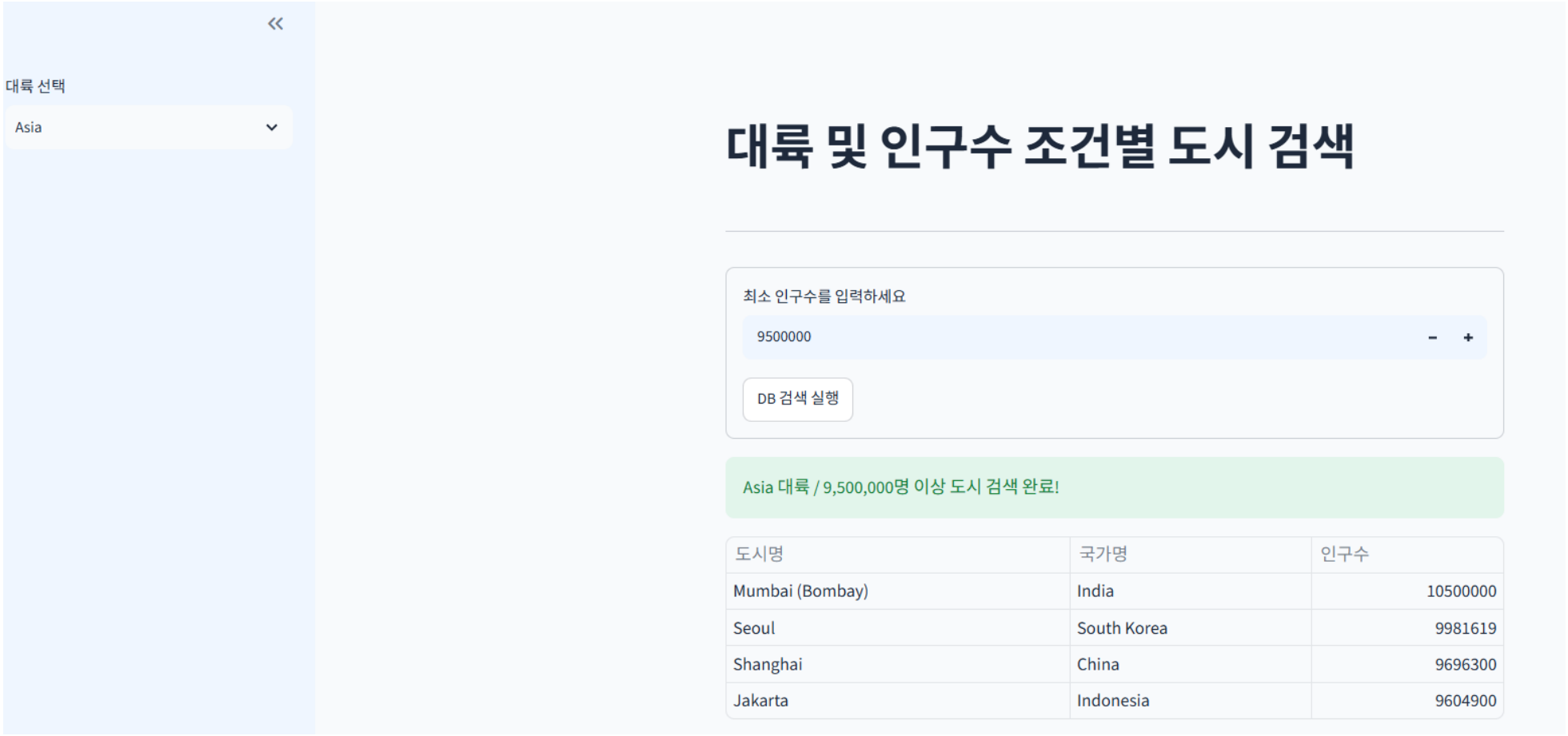
## world DB 한국 주요 도시 현황

### 한국 상위 10개 도시 목록

| 도시명 | 도_지역 | 인구수 |
|---|---|---|
| Seoul | Seoul | 9981619 |
| Pusan | Pusan | 3804522 |
| Inchon | Inchon | 2559424 |
| Taegu | Taegu | 2548568 |
| Taejon | Taejon | 1425835 |
| Kwangju | Kwangju | 1368341 |
| Ulsan | Kyongsangnam | 1084891 |
| Songnam | Kyonggi | 869094 |
| Puchon | Kyonggi | 779412 |
| Suwon | Kyonggi | 755550 |

# PyMySQL + Streamlit 위젯(st.selectbox, st.form) 동적 연동 실습

- 실습 스크립트 작성 (app_search.py)




MISSION:
COMPLETE